



INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE FÍSICA Y MATEMÁTICAS

MATEMÁTICAS EN REVERSA: FUERZA AXIOMÁTICA Y
FUNDAMENTOS LÓGICOS DE LOS SUBSISTEMAS RCA_0 ,
 WKL_0 Y ACA_0

T E S I S

QUE PARA OBTENER EL GRADO DE:

LICENCIADO EN MATEMÁTICA ALGORÍTMICA

PRESENTA:

OMAR PORFIRIO GARCÍA

DIRECTOR DE TESIS:

DAVID JOSÉ FERNÁNDEZ BRETÓN

Ciudad de México, 2026



Índice general

Índice de figuras	5
1. Teoría de la computabilidad	7
1.1. Computabilidad: Enfoque inductivo	7
1.1.1. Funciones primitivo-recursivas	7
1.1.2. Funciones parcial-recursivas	21
1.2. Computabilidad: Enfoque mecánico	26
1.2.1. Codificación de listas	39
1.2.2. Codificación de máquinas URM	43
1.3. La Máquina Universal, el problema de la detención y conjuntos c.e.	54
2. Aritmética de segundo orden	59
2.1. Sintaxis	59
2.2. Semántica	63
2.3. Resultados y definiciones esenciales	66
3. El subsistema RCA_0	75
3.1. Teoría de números elemental	76
3.2. Sistemas numéricos	80
3.2.1. Los números enteros	80
3.2.2. Los números racionales	91
3.2.3. Los números reales	98

Índice de figuras

1.1. Cinta y registros	27
1.2. Cinta inicial para la máquina M	28
1.3. Cinta después de ejecutar $S(1)$	28
1.4. Cinta después de ejecutar $S(3)$	28
1.5. Cinta después de ejecutar $S(1)$ por segunda vez	28
1.6. Cinta después de ejecutar $S(3)$ por segunda vez	29
1.7. Ejecución completa de la máquina M	30
1.8. Máquina URM de la composición	36
1.9. Máquina URM de la recursión primitiva	38
1.10. Máquina URM de la minimización no acotada	39

Capítulo 1

Teoría de la computabilidad

1.1. Computabilidad: Enfoque inductivo

En esta sección abordaremos el enfoque inductivo de la computabilidad. Este se caracteriza por la construcción de lo computable por medio de ciertas funciones básicas y operaciones que permiten combinarlas. Frente al enfoque mecánico, el cual concibe un algoritmo como una secuencia de pasos ejecutados en el tiempo, este enfoque nos describe lo computable de una manera estructural: un algoritmo deja de ser una lista finita de instrucciones y, en cambio, se vuelve una combinación de otros algoritmos más simples. Este enfoque tiene como ventaja el poder justificar la computabilidad de una función por medio de la verificación de propiedades de cerradura; en lugar de exhibir una lista de pasos explícitos para cada función, solamente es necesario verificar que la función puede derivarse de otras que ya conozcamos como computables y que se obtengan mediante operaciones admisibles.

1.1.1. Funciones primitivo-recursive

El primer tipo de funciones que abordaremos son las *funciones primitivo-recursive*. El interés sobre este tipo de funciones es, por una parte, debido a que abarcan la mayoría de los objetos computables que podemos encontrarnos en la práctica; y por otra, a que son la mayoría de objetos que construiremos para estudiar la computabilidad.

La característica principal de las funciones primitivo-recursive es que se obtienen de funciones básicas mediante composición y recursión primitiva. La composición corresponde a la idea de poder utilizar diferentes algoritmos como subrutinas auxiliares que permitan ejecutar un algoritmo más complejo. La recursión, por su parte, puede interpretarse como un ciclo *for*: el número de iteraciones queda determinado de antemano por los argumentos de la función, de modo que no solo tenemos la certeza de que el proceso ejecutará un número finito de pasos,

sino de cuántos de ellos exactamente. Estos conceptos se definen formalmente de la siguiente manera.

Definición 1.1 (Funciones primitivo-recursive). Sea $n \in \omega^+$. Una función $f : \omega^n \rightarrow \omega$ se denomina *primitivo-recursive* si satisface alguna de las siguientes condiciones:

1. (Funciones básicas) La función f es una de las siguientes funciones:
 - La función *cero*: $Z : \omega^n \rightarrow \omega$ tal que para cada $x \in \omega^n$, $Z(x) = 0$.
 - Si $n = 1$, la función *sucesor*: $S : \omega \rightarrow \omega$ tal que para cada $x \in \omega$, $S(x) = x + 1$.
 - Para algún $k \in \omega^+$ con $k \leq n$, la función *proyección*: $\pi_k^n : \omega^n \rightarrow \omega$ tal que para cada $x_1, \dots, x_n \in \omega$, $\pi_k^n(x_1, \dots, x_n) = x_k$.
2. (Composición) Para algún $m \in \omega^+$ existen funciones primitivo-recursive $g_1, \dots, g_m : \omega^n \rightarrow \omega$ y $h : \omega^m \rightarrow \omega$ tales que para cada $x \in \omega^n$, $f(x) = h(g_1(x), \dots, g_m(x))$.
3. (Recursión) Si $n > 1$ y existen funciones primitivo-recursive $g : \omega^{n-1} \rightarrow \omega$ y $h : \omega^{n+1} \rightarrow \omega$ tales que para cada $x \in \omega^{n-1}$ y cada $y \in \omega$ se satisface que:

$$\begin{aligned} f(x, 0) &= g(x) \\ f(x, y + 1) &= h(x, y, f(x, y)) \end{aligned}$$

Para cada $n \in \omega^+$, denotamos por Prim_n al conjunto de todas las funciones primitivo-recursive de aridad n . Definimos al conjunto de funciones primitivo-recursive como $\text{Prim} := \bigcup_{n \in \omega^+} \text{Prim}_n$.

Ejemplo de cómo es que estas funciones modelan una parte de la computabilidad es el siguiente teorema.

Teorema 1.1. La suma, producto, exponenciación y las funciones constantes son funciones primitivo-recursive.

Antes de ir a la demostración formal, veamos cómo es que podemos reescribir a estas funciones. Para el caso de la suma, notemos que si $\text{Suma}(x, y) = x + y$, entonces:

$$\text{Suma}(x, y + 1) = x + (y + 1) = (x + y) + 1 = \text{Suma}(x, y) + 1$$

por lo que la suma puede escribirse de manera recursiva sobre el segundo argumento. Notemos que para que la recursión esté bien definida necesitamos tener un caso base. En este caso tendríamos que $\text{Suma}(x, 0) = x + 0 = x$.

Similarmente para el caso del producto, si $\text{Prod}(x, y) = xy$, entonces:

$$\text{Prod}(x, y + 1) = x(y + 1) = xy + x = \text{Prod}(x, y) + x$$

con el caso base $\text{Prod}(x, 0) = x \cdot 0 = 0$.

Ahora para la exponenciación, si $\text{Exp}(x, y) = x^y$, entonces:

$$\text{Exp}(x, y + 1) = x^{y+1} = x^y x = \text{Exp}(x, y)x$$

con el caso base $\text{Exp}(x, 0) = x^0 = 1$.

Con esto vemos que estas operaciones pueden calcularse de manera recursiva, lo cual nos es de ayuda para poder utilizar recursión primitiva y así ver que estas funciones son primitivo-recursivas.

Demostración. Para el caso de la suma definamos la función $f : \omega^2 \rightarrow \omega$ como:

$$\begin{aligned} f(x, 0) &= \pi_1^1(x) \\ f(x, y + 1) &= S(\pi_3^3(x, y, f(x, y))) \end{aligned}$$

Puesto que π_1^1 , S y π_3^3 son funciones primitivo-recursivas, se tiene que f es primitivo-recursiva. Mostremos entonces que para cada $x, y \in \omega$ se satisface que $f(x, y) = x + y$. Mostremos por inducción sobre y .

- (Base) Se tiene que para $y = 0$ se sigue que $f(x, 0) = \pi_1^1(x) = x = x + y$.
- (Paso inductivo) Sea $y \in \omega$ tal que $f(x, y) = x + y$. Entonces mostremos que $f(x, y + 1) = x + (y + 1)$. Por la definición de f se tiene lo siguiente:

$$f(x, y + 1) = S(\pi_3^3(x, y, f(x, y))) = S(f(x, y)) = f(x, y) + 1 = (x + y) + 1 = x + (y + 1)$$

Probando así la proposición.

Ahora, para el caso del producto definamos la función auxiliar $g : \omega^3 \rightarrow \omega$ para cada $x, y, z \in \omega$ como:

$$g(x, y, z) = \pi_3^3(x, y, z) + \pi_1^3(x, y, z)$$

la cual es primitivo-recursiva debido a que π_1^3 , π_3^3 y la suma lo son. Ahora, el producto puede describirse como la función $f : \omega^2 \rightarrow \omega$ tal que:

$$\begin{aligned} f(x, 0) &= Z(x) \\ f(x, y + 1) &= g(x, y, f(x, y)) \end{aligned}$$

Puesto que Z y g son funciones primitivo-recursivas, se tiene que f es primitivo-recursiva. Mostremos entonces que para cada $x, y \in \omega$ se satisface que $f(x, y) = xy$. Mostremos por inducción sobre y .

- (Base) Se tiene que para $y = 0$ se sigue que $f(x, 0) = Z(x) = 0 = xy$.
- (Paso inductivo) Sea $y \in \omega$ tal que $f(x, y) = xy$. Entonces mostremos que $f(x, y + 1) = x(y + 1)$. Por la definición de f se tiene lo siguiente:

$$\begin{aligned}
 f(x, y + 1) &= g(x, y, f(x, y)) \\
 &= \pi_3^3(x, y, f(x, y)) + \pi_1^3(x, y) \\
 &= f(x, y) + \pi_1^2(x, y) \\
 &= f(x, y) + x \\
 &= xy + x \\
 &= x(y + 1)
 \end{aligned}$$

Probando así la proposición.

Para el caso de la exponenciación definamos a la función auxiliar $h : \omega^3 \rightarrow \omega$ tal que:

$$h(x, y, z) = \pi_3^3(x, y, z)\pi_1^3(x, y, z)$$

la cual podemos asegurar que es primitivo-recursiva ya que π_1^3 , π_3^3 y el producto lo son. Por lo tanto, la exponenciación puede describirse mediante la función $f : \omega^2 \rightarrow \omega$ tal que:

$$\begin{aligned}
 f(x, 0) &= S(Z(x)) \\
 f(x, y + 1) &= h(x, y, f(x, y))
 \end{aligned}$$

Puesto que S , Z y h son funciones primitivo-recursivas, se tiene que f es primitivo-recursiva. Mostremos entonces que para cada $x, y \in \omega^+$ se satisface que $f(x, y) = x^y$. Prosigamos por inducción sobre y .

- (Base) Para $y = 0$ se sigue que $f(x, 0) = S(Z(x)) = 1 = x^y$.
- (Paso inductivo) Sea $y \in \omega$ tal que $f(x, y) = x^y$. Entonces mostremos que $f(x, y + 1) = x^{y+1}$. Por la definición de f se tiene lo siguiente:

$$\begin{aligned}
 f(x, y + 1) &= h(x, y, f(x, y)) \\
 &= \pi_3^3(x, y, f(x, y))\pi_1^3(x, y, f(x, y)) \\
 &= f(x, y)x \\
 &= x^y x \\
 &= x^{y+1}
 \end{aligned}$$

Probando así la proposición.

Para cada $m \in \omega$ y cada $n \in \omega^+$ denotemos por $f_m^n : \omega^n \rightarrow \omega$ a la función constante m , es decir, tal que para cada $x \in \omega^n$ se satisface que $f_m^n(x) = m$. Mostremos por inducción sobre la constante m que f_m^n es primitivo-recursiva.

- (Base) Sea $n \in \omega^+$. Para la función f_0^n claramente que tiene que $f_0^n = Z \circ \pi_1^n$. Puesto que f_0^n es composición de funciones básicas, entonces es primitivo-recursiva.
- (Paso inductivo) Sean $n \in \omega^+$ y $m \in \omega$ tales que f_m^n es primitivo-recursiva. Notemos que $f_{m+1}^n = S \circ f_m^n$. Puesto que S es una función básica y f_m^n es primitivo-recursiva, entonces f_{m+1}^n es primitivo-recursiva.

Por lo tanto, para cada $n \in \omega^+$ y cada $m \in \omega$ se tiene que f_m^n es primitivo-recursiva. $\square$

De la demostración anterior puede apreciarse que usar recursión es equivalente al uso de un *for* algorítmicamente hablando. Para el caso de la suma se tendría que $x + y$ comprendería ejecutar un algoritmo que devuelve el valor x si $y = 0$ y, en caso contrario, suma uno al valor x un número y de veces.

Algoritmo 1 Suma

Entrada: x, y

Salida: $x + y$

```

para  $n = 0, \dots, y - 1$  hacer
     $x \leftarrow x + 1$ 
fin para
regresar  $x$ 

```

En cuanto al producto xy se trataría de un algoritmo el cual utiliza un ciclo *for* que va sumando repetidamente el valor x un número y de veces. De manera similar para el caso de la exponenciación, el cual utilizaría un ciclo **for** para ir multiplicando el valor x un número y de veces.

Algoritmo 2 Producto

Entrada: x, y

Salida: xy

```

 $z \leftarrow 0$ 
para  $n = 0, \dots, y - 1$  hacer
     $z \leftarrow \text{Suma}(z, x)$ 
fin para
regresar  $z$ 

```

Algoritmo 3 Exponenciación**Entrada:** x, y **Salida:** x^y

```

 $z \leftarrow 1$ 
para  $n = 0, \dots, y - 1$  hacer
     $z \leftarrow \text{Producto}(z, x)$ 
fin para
regresar  $z$ 

```

Para poder demostrar que otras funciones comunes (y particularmente las unarias) son funciones primitivo-recursivas debemos demostrar que es posible debilitar la recursión primitiva. Observemos que en la definición 1.1 se nos permite utilizar recursión primitiva únicamente cuando la función a calcular tiene al menos dos argumentos. Sin embargo, existen funciones que pueden calcularse recursivamente mediante un solo argumento (como el factorial). Por lo tanto, es necesario demostrar que utilizar este tipo de recursión más débil también genera funciones primitivo-recursivas.

Teorema 1.2 (Recursión simple). Sean $a \in \omega$ y $g : \omega^2 \rightarrow \omega$ primitivo-recursiva. Entonces la función $f : \omega \rightarrow \omega$ definida como sigue, para cada $x \in \omega$, es primitivo-recursiva:

$$\begin{aligned}
 f(0) &= a \\
 f(x+1) &= g(x, f(x))
 \end{aligned}$$

Demostración. Consideremos primero a las funciones auxiliares $h_0 : \omega^3 \rightarrow \omega$ tal que $h_0 = g(\pi_2^3, \pi_3^3)$ y $h : \omega^2 \rightarrow \omega$ definida como sigue para cada $x, y \in \omega$:

$$\begin{aligned}
 h(x, 0) &= f_a^1(x) \\
 h(x, y+1) &= h_0(x, y, h(x, y))
 \end{aligned}$$

Notemos que las funciones h_0 y h son primitivo-recursivas puesto que g y las funciones proyección son primitivo-recursivas. Afirmamos que h es una extensión de f donde su primer argumento no afecta a la recursión, de modo que $f = h(Z, \pi_1^1)$. Mostremos tal afirmación por inducción:

- (Base) Se tiene que $h(Z(0), \pi_1^1(0)) = h(0, 0) = f_a^1(0) = a = f(0)$.

- (Paso inductivo) Sea $x \in \omega$ tal que $f(x) = h(Z(x), \pi_1^1(x)) = h(0, x)$, entonces se tiene que:

$$\begin{aligned}
 h(Z(x+1), \pi_1^1(x+1)) &= h(0, x+1) \\
 &= h_0(0, x, h(0, x)) \\
 &= g(\pi_2^3(0, x, h(0, x)), \pi_3^3(0, x, h(0, x))) \\
 &= g(x, h(0, x)) \\
 &= g(x, f(x)) \\
 &= f(x+1)
 \end{aligned}$$

Por lo tanto, se cumple que $f = h(Z, \pi_1^1)$, la cual es primitivo-recursive ya que h es primitivo-recursive. $\square$

Con este teorema podemos entonces demostrar que las funciones factorial, predecesor y resta son primitivo-recursive.

Teorema 1.3. Las siguientes funciones son primitivo-recursive.

1. El factorial, es decir, la función $f : \omega \rightarrow \omega$ tal que para cada $x \in \omega$:

$$f(x) = \begin{cases} 1, & x = 0 \\ 1 \cdot 2 \cdot \dots \cdot x, & x > 0 \end{cases}$$

2. La función predecesor, es decir, la función $f : \omega \rightarrow \omega$ tal que para cada $x \in \omega$:

$$f(x) = \begin{cases} 0, & x = 0 \\ x - 1, & x > 0 \end{cases}$$

3. La función resta en ω , es decir, la función $f : \omega^2 \rightarrow \omega$ tal que para cada $x, y \in \omega$:

$$f(x, y) = \begin{cases} 0, & x \leq y \\ x - y, & x > y \end{cases}$$

Demostración. Para el factorial notemos que tal función se puede definir de la siguiente manera recursiva:

$$\begin{aligned}
 f(0) &= 1 \\
 f(x+1) &= (x+1) \cdot f(x)
 \end{aligned}$$

la cual es primitivo-recursive por los teoremas anteriores.

En el caso de la función predecesor, se puede definir recursivamente como sigue:

$$\begin{aligned} f(0) &= 0 \\ f(x+1) &= \pi_1^2(x, f(x)) \end{aligned}$$

la cual es primitivo-recursiva por los teoremas anteriores igualmente. Denotemos a la función predecesor como $\text{Pred}(x)$.

Para el caso de la resta, esta se puede definir recursivamente como sigue:

$$\begin{aligned} f(x, 0) &= \pi_1^1(x) \\ f(x, y+1) &= \text{Pred}(\pi_3^3(x, y, f(x, y))) \end{aligned}$$

Mostremos en este caso que realmente se trata de la resta restringida a ω por inducción sobre el segundo argumento. Para ello, dividamos en dos casos: cuando $y \leq x$ y cuando $y > x$.

Supongamos que $y \leq x \in \omega$ y mostremos por inducción sobre y que $f(x, y) = x - y$.

- (Base) Directamente se tiene que $f(x, 0) = \pi_1^1(x) = x = x - 0 = x - y$.
- (Paso inductivo) Sea $y < x$ para el cual $f(x, y) = x - y$ y mostremos que $f(x, y+1) = x - (y+1)$.

$$\begin{aligned} f(x, y+1) &= \text{Pred}(\pi_2^3(x, y, f(x, y))) \\ &= \text{Pred}(f(x, y)) \\ &= f(x, y) - 1 \\ &= (x - y) - 1 \\ &= x - (y+1) \end{aligned}$$

Notemos que en este caso, como $y < x$, entonces $f(x, y) = x - y > 0$, por lo que en efecto $\text{Pred}(f(x, y)) = f(x, y) - 1$.

Ahora, para el caso cuando $y > x$, notemos que existe $z \in \omega$ para el cual $y = x + 1 + z$. Por lo tanto, mostremos por inducción sobre z que $f(x, y) = 0$.

- (Base) Notemos directamente que $f(x, x+1) = \text{Pred}(f(x, x)) = \text{Pred}(0) = 0$.
- (Paso inductivo) Sea $z \in \omega$ para el cual se satisface que $f(x, x+1+z) = 0$, entonces mostremos que $f(x, x+1+(z+1)) = 0$.

$$\begin{aligned} f(x, x+1+(z+1)) &= f(x, (x+1+z)+1) \\ &= \text{Pred}(f(x, x+1+z)) \\ &= \text{Pred}(0) = 0 \end{aligned}$$

Por lo que, en efecto, se concluye que f es la función resta restringida a ω . $\square$

Podemos extender la definición de primitivo-recursión a subconjuntos y relaciones de números naturales. Esta extensión es fundamental puesto que saber si podemos computar un conjunto es lo que nos permite determinar qué problemas pueden o no ser resueltos por medio de algoritmos. Un conjunto se dice *primitivo-recursivo* si podemos determinar si un elemento pertenece a él por medio de una función primitivo-recursiva; lo cual se traduce en que su función característica es primitivo-recursiva. Más aún, esto se puede extender a propiedades. Una propiedad se denomina primitivo-recursiva si podemos *decidir* si un elemento la satisface o no mediante una función primitivo-recursiva. Notemos que cualquier propiedad sobre los naturales puede ser traducida al conjunto que define, y si este conjunto es primitivo-recursivo, entonces se tiene que siempre podemos determinar si cualquier elemento la satisface.

Definición 1.2 (Conjuntos primitivos recursivos). Sea $A \subseteq \omega^n$ para algún $n \in \omega^+$. El conjunto A se dice ser *primitivo-recursivo* si su función característica $\chi_A : \omega^n \rightarrow \omega^+$ es primitivo-recursiva.

Notación 1.1. Sea $R \subseteq \omega^n$ con $n \in \omega^+$, entonces para cada $x \in \omega^n$ se denotará por $R(x)$ si $x \in R$ y se denotará por $\neg R(x)$ si $x \notin R$.

Ejemplos de conjuntos que son primitivos recursivos están descritos en el siguiente teorema.

Teorema 1.4. Los conjuntos $>, <, = \subseteq \omega^2$, $\{0\}$ y ω^+ son primitivos recursivos.

Demostración. Notemos que $\chi_{\{0\}}(x) = 1 - x$ para cada $x \in \omega$, puesto que cuando $x = 0$, se tiene que $1 - x = 1$; y cuando $x > 0$, entonces $1 - x = 0$ por definición de resta restringida. Puesto que la resta es primitivo-recursiva, se sigue que $\chi_{\{0\}}$ es primitivo-recursiva.

Puesto que $\chi_{\omega^+}(x) = 1 - \chi_{\{0\}}(x)$ para cada $x \in \omega$ y $\chi_{\{0\}}$ es primitivo-recursiva, se sigue que χ_{ω^+} es primitivo-recursiva.

Finalmente, puesto que $\chi_{<}(x, y) = \chi_{\omega^+}(y - x)$, $\chi_{>}(x, y) = \chi_{\omega^+}(x - y)$ y $\chi_{=}(x, y) = \chi_{\{0\}}(\chi_{<}(x, y) + \chi_{>}(x, y))$ para cada $x, y \in \omega$, se concluye que $\chi_{<}$, $\chi_{>}$ y $\chi_{=}$ son primitivo-recursivos. $\square$

Teorema 1.5. Sean $n \in \omega^+$ y $A, B \subseteq \omega^n$. Si A y B son primitivo-recursivos, entonces $\omega^n \setminus A$, $A \cup B$ y $A \cap B$ son primitivo-recursivos.

Demostración. Se tiene que $\chi_{\omega^n \setminus A} = 1 - \chi_A$, $\chi_{A \cap B} = \chi_A \chi_B$ y $\chi_{A \cup B} = \chi_A + \chi_B - \chi_{A \cap B}$, y puesto que A y B son primitivo-recursivos, se sigue que $\omega^n \setminus A$, $A \cup B$ y $A \cap B$ son primitivo-recursivos. $\square$

Corolario 1.6. Sean $n \in \omega^+$ y φ, ψ fórmulas con n variables libres. Si φ y ψ definen conjuntos primitivos recursivos, entonces $\neg\varphi$, $\varphi \wedge \psi$ y $\varphi \vee \psi$ definen conjuntos primitivos recursivos en ω^n .

Lo siguiente es demostrar que la clase de funciones primitivo-recursivas es cerrada bajo ciertas operaciones, lo que será de mucha utilidad a lo largo del capítulo.

Teorema 1.7. Sean $n \in \omega^+$, y $f : \omega^{n+1} \rightarrow \omega$ y $g : \omega^n \rightarrow \omega$ funciones primitivo-recursivas. Entonces:

1. la función $h : \omega^n \rightarrow \omega$ dada por $h(x) = \sum_{z < g(x)} f(x, z)$ para cada $x \in \omega^n$ es primitivo-recursiva.
2. la función $h : \omega^n \rightarrow \omega$ dada por $h(x) = \prod_{z < g(x)} f(x, z)$ para cada $x \in \omega^n$ es primitivo-recursiva.

Demostración. Mostremos únicamente para el primer punto puesto que el segundo es análogo. Definamos entonces la función $p : \omega^{n+1} \rightarrow \omega$ tal que para cada $x \in \omega^n$:

$$\begin{aligned} p(x, 0) &= 0 \\ p(x, y + 1) &= p(x, y) + f(x, y) \end{aligned}$$

Mostremos que $p(x, y) = \sum_{z < y} f(x, z)$ para cada $x \in \omega^n$ y $y \in \omega^+$ por inducción sobre y .

- (Base) Se tiene que si $y = 0$, entonces:

$$p(x, 0) = 0 = \sum_{z < 0} f(x, z) = \sum_{z < y} f(x, z)$$

- (Paso inductivo) Sea $y > 0$ tal que $p(x, y) = \sum_{z < y} f(x, z)$, entonces mostremos que se cumple para $y + 1$.

$$p(x, y + 1) = p(x, y) + f(x, y) = \sum_{z < y} f(x, z) + f(x, y) = \sum_{z < y+1} f(x, z)$$

Probando así lo que se requería. Puesto que f es primitivo-recursiva, se sigue que p es primitivo-recursiva. De lo anterior se sigue entonces que $h(x) = p(x, g(x))$, y luego h es primitivo-recursiva ya que p y g lo son.

□

Notación 1.2. Sean $t \in \omega$, $n \in \omega^+$ y $R \subseteq \omega^{n+1}$, entonces para cada $x \in \omega^n$ denotamos por:

- $(\exists m < t)R(x, m)$ a la expresión $\exists m((m < t) \wedge R(x, m))$.
- $(\forall m < t)R(x)$ a la expresión $\forall m((m < t) \rightarrow R(x, m))$.
- $(\exists m \leq t)R(x)$ a la expresión $\exists m((m \leq t) \wedge R(x, m))$.
- $(\forall m \leq t)R(x)$ a la expresión $\forall m((m \leq t) \rightarrow R(x, m))$.

Corolario 1.8. Sean $n \in \omega^+$, $R \subseteq \omega^{n+1}$ primitivo-recursivo y $f : \omega^n \rightarrow \omega$ primitivo-recursiva, entonces:

- el conjunto $\{x \in \omega^n : (\exists z < f(x))R(x, z)\}$ es primitivo-recursivo.
- el conjunto $\{x \in \omega^n : (\forall z < f(x))R(x, z)\}$ es primitivo-recursivo.

Antes de demostrar este corolario es importante mencionar que este tipo de expresiones son fundamentales dentro de la Teoría de la Computabilidad, y particularmente cuando desarrollemos la teoría de la Aritmética de Segundo Orden. A estas expresiones les denominamos "búsquedas". Por ejemplo, para verificar que la expresión $(\exists z < f(x))R(x, z)$ es verdadera, se puede ir comprobando con todos los valores $z = 0, 1, \dots, f(x) - 1$ si se satisface $R(x, z)$, uno por uno, buscando cuál de esos valores satisface tal pertenencia; y la expresión sería falsa si nunca encontramos tal valor. Para el caso de $(\forall z < f(x))R(x, z)$, podemos en cambio verificar si la expresión es falsa buscando entre todos los valores $z = 0, 1, \dots, f(x) - 1$ si $\neg R(x, z)$; y si nunca encontramos tal contraejemplo, entonces la expresión original es verdadera.

Es importante notar que los procedimientos descritos siempre terminan, pues existe un número finito de candidatos (los valores entre 0 y $f(x)$) que podemos verificar. No importa si en la práctica tarda demasiado verificar todos y cada uno de ellos, teóricamente siempre podemos decidir si las expresiones son verdaderas o falsas.

Ahora, para demostrar que estos conjuntos son primitivo-recursivos la idea es justamente saber cuándo las fórmulas son verdaderas. Para el caso de $(\exists z < f(x))R(x, z)$, como mencionamos anteriormente, podemos verificar si alguno de los valores $\chi_R(x, 0), \chi_R(x, 1), \dots, \chi_R(x, f(x) - 1)$ es igual a uno; si al menos un término es igual a uno, entonces la fórmula se satisface. La forma de condensar esto en una única expresión es sumar todos los valores, pues la suma será mayor que cero si al menos un valor satisface la fórmula, y será cero si ninguno la satisface, de modo que podemos devolver uno si la suma es mayor que cero y devolvemos cero en caso contrario.

De manera similar, para la expresión $(\forall z < f(x))R(x, z)$ debemos verificar que todos los valores $\chi_R(x, 0), \chi_R(x, 1), \dots, \chi_R(x, f(x) - 1)$ son uno para que la fórmula se satisfaga. Eso podemos lograrlo multiplicando todos los valores; si todos son uno, el producto será uno (in-

dicando que la fórmula se satisface), y si al menos uno es cero, entonces el producto será cero (indicando que la fórmula es falsa).

Demostración. Denotemos a los conjuntos como $S = \{x \in \omega^n : (\exists z < f(x))R(x, z)\}$ y $T = \{x \in \omega^n : (\forall z < f(x))R(x, z)\}$, entonces se tiene que:

$$\begin{aligned}\chi_S(x) &= \chi_{\omega^+} \left(\sum_{z < f(x)} \chi_R(x, z) \right) \\ \chi_T(x) &= \prod_{z < f(x)} \chi_R(x, z)\end{aligned}$$

Puesto que f y R son primitivos recursivos, se sigue que S y T son primitivos recursivos. $\square$

Cuando hablamos de computación, una herramienta fundamental en la práctica es el control de flujo *if-then-else*. Varias veces necesitamos que un programa computacional sea capaz de distinguir entre diferentes escenarios y realizar entonces acciones específicas para cada uno. Resulta relevante que esta estructura pueda ser modelada dentro de esta teoría. Notemos que una expresión *if-then-else* tiene la siguiente forma:

Algoritmo 4 *if-then-else*

```

si  $R_1(x)$  entonces
    regresar  $g_1(x)$ 
si no si  $R_2(x)$  entonces
    regresar  $g_2(x)$ 
     $\vdots$ 
si no si  $R_m(x)$  entonces
    regresar  $g_m(x)$ 
si no
    regresar  $g_{m+1}(x)$ 
fin si

```

Donde las expresiones $R_1, \dots, R_m$ son las condiciones (disjuntas) que nos permiten discernir entre los diferentes escenarios y las cuales se evalúan para determinar si alguna se satisface para entonces ejecutar el código descrito por g_k para algún $k \leq m + 1$. Como hemos descrito anteriormente, una condición o propiedad se corresponde con el conjunto que define; por lo que las condiciones pueden verse como relaciones primitivo-recursivas disjuntas. Para el caso de los códigos de cada rama, estos pueden ser representados como funciones primitivo-recursivas (nuestra forma de modelar un algoritmo hasta ahora). Por lo tanto, una expresión *if-then-else* puede modelarse dentro de este marco como una función primitivo-recursiva definida a trozos.

Teorema 1.9. Sean $n, m \in \omega^+$, funciones primitivo-recursive $g_1, \dots, g_m, g_{m+1} : \omega^n \rightarrow \omega$ y relaciones primitivo-recursive $R_1, \dots, R_m \subseteq \omega^n$ disjuntas a pares, entonces la función $f : \omega^n \rightarrow \omega$ tal que para cada $x \in \omega^n$:

$$f(x) = \begin{cases} g_1(x), & R_1(x) \\ \vdots & \\ g_m(x), & R_m(x) \\ g_{m+1}(x), & \text{en otro caso} \end{cases}$$

es primitivo-recursive.

Demostración. Se sigue que:

$$f = \sum_{k=1}^m g_k \chi_{R_k} + g_{m+1} \chi_{\omega^n \setminus \bigcup_{k=1}^m R_k}$$

Por lo tanto, f es primitivo-recursive. □

Ya sabemos que la búsqueda existencial acotada es primitivo-recursive, pero en ciertas ocasiones no solamente necesitamos saber si tal existencia es afirmativa, sino obtener un valor específico que sea un testigo debido a que necesitamos operar con él. Por ejemplo, no solamente necesitamos saber si existen los números primos, sino obtener explícitamente un primo particular. Puesto que trabajamos dentro del marco de los números naturales, la forma más natural de elegir un testigo es tomar el mínimo de ellos. Esa es la razón por la que debemos verificar que computacionalmente podemos encontrar el mínimo elemento que pertenece a una relación.

Consideremos una relación $R(x, y)$ y $p \in \omega$. Para poder demostrar que encontrar el mínimo valor $z < p$ tal que $R(x, z)$ puede ser modelado por una función primitivo-recursive primero debemos justificar que es posible verificar primitivo-recursive que un elemento es mínimo o no de una relación. Para ello construimos una relación $S(x, z)$ la cual nos describa el hecho de que $z = \min\{y : y < p \wedge R(x, y)\}$. Dado que trabajamos con existencia acotada y podemos verificar si un elemento es mínimo, el procedimiento consiste en determinar para cada valor $z < p$ si lo es o no. Ya que ser mínimo es una propiedad de unicidad, si existe, se asegura que hay un único z para el cual $\chi_S(x, z) = 1$ y para cualquier valor $y \neq z$ se tiene $\chi_S(x, y) = 0$. Por lo tanto, se puede aislar a z realizando la suma $\sum_{y < p} y \cdot \chi_S(x, y)$.

Teorema 1.10 (Minimización acotada). Sean $n \in \omega^+$, $R \subseteq \omega^{n+1}$ una relación primitivo-recursive y $g : \omega^n \rightarrow \omega$ primitivo-recursive. Entonces la función $f : \omega^n \rightarrow \omega$ tal que para cada

$x \in \omega^n$:

$$f(x) = \begin{cases} \min\{y \in \omega : y < g(x) \wedge R(x, y)\}, & (\exists z < g(x))R(x, z) \\ 0, & \text{en otro caso} \end{cases}$$

es primitivo-recursive.

Demostración. Consideremos a la relación:

$$S = \{(x, z) \in \omega^{n+1} : R(x, z) \wedge (\forall y < z) \neg R(x, y)\}$$

Esta relación es primitivo-recursive puesto que:

$$\chi_S(x, z) = \chi_R(x, z) \left(\prod_{y < g(x)} \chi_{\omega^n \setminus R}(y, x) \right)$$

Afirmamos que $S(x, z)$ si y solo si $z = \min\{y \in \omega : R(x, y)\}$. Consideremos que $S(x, z)$ pero $z \neq \min\{y \in \omega : R(x, y)\}$, por lo que existe $w < z$ tal que $R(x, w)$. Obtenemos entonces que w es un testigo de la fórmula $(\exists y < z)R(x, y)$, la cual es la negación de $(\forall y < z) \neg R(x, y)$. Se sigue por la definición de la relación S que $\neg S(x, z)$; lo cual es una contradicción. Ahora, supongamos que $z = \min\{y \in \omega : R(x, y)\}$, entonces por definición de mínimo se satisface que $R(x, z)$ y para cada $y \leq z$ se satisface que $R(x, y)$ implica $y = z$. Luego, para cada $y < z$ se sigue que $\neg R(x, y)$ por contrarrecíproca, es decir, $S(x, z)$. Por lo tanto, la función f puede expresarse de la siguiente forma:

$$f(x) = \sum_{z < g(x)} z \chi_S(x, z)$$

Por lo tanto, f es primitivo-recursive. □

Notación 1.3. A la función definida por el teorema anterior dados $n \in \omega^+$, $R \subseteq \omega^{n+1}$ una relación primitivo-recursive y $g : \omega^n \rightarrow \omega$ primitivo-recursive se le denota por $f(x) = (\mu z < g(x))R(x, z)$ para cada $x \in \omega^n$ y se lee como “el mínimo $z < g(x)$ tal que $R(x, z)$ ”.

Estas propiedades de las funciones primitivo-recursive nos permiten mostrar que el conjunto de números primos es primitivo-recursive, así como también la función que regresa el $(n+1)$ -ésimo primo. Esto es importante pues nos permitirá demostrar la enumeración de los algoritmos más adelante.

Teorema 1.11. La relación “divide a”, el conjunto de números primos y la función que devuelve el $(n+1)$ -ésimo número primo para cada $n \in \omega$ son primitivo-recursive.

Demostración. La relación “divide a” puede verse como la relación:

$$S = \{(x, y) \in \omega^2 : (\exists k \leq y)(y = kx)\}$$

Puesto que usamos búsqueda acotada y la igualdad es primitivo-recursive, se tiene que S es primitivo-recursive.

Ahora, sea $P \subseteq \omega$ el conjunto de los números primos. Notemos que:

$$P = \{x \in \omega : x > 1 \wedge \neg(\exists k < x)(k > 1 \wedge S(k, x))\}$$

Puesto que el “mayor que” y S son primitivo-recursive, se sigue que P es primitivo-recursive.

Finalmente, sea $f : \omega \rightarrow \omega$ tal que $f(n)$ es el $(n + 1)$ -ésimo primo para cada $n \in \omega$. Por la prueba de Euclides de la infinitud de los números primos sabemos que $f(n + 1) \leq f(n)! + 1$, por lo que f está dada como sigue:

$$\begin{aligned} f(0) &= 2 \\ f(n + 1) &= (\mu p < f(n)! + 2)(p > f(n) \wedge P(p)) \end{aligned}$$

Puesto que el factorial y P son primitivo-recursive, se tiene que f es primitivo-recursive. $\square$

1.1.2. Funciones parcial-recursive

Una extensión de las funciones primitivo-recursive son las funciones parcial-recursive. Esta clase de funciones es más general, puesto que resuelven un problema el cual las funciones primitivo-recursive no: realizar iteraciones indefinidamente. Notemos que las funciones primitivo-recursive se caracterizan por utilizar ciclos *for*, formalizados por el uso de la recursión primitiva. Aunque estas son suficientes para modelar la mayoría de funciones en la práctica, tienen un problema y es que el número de iteraciones que realizan está determinado por un límite específico. Notemos en todos los resultados anteriores que las sumas y productos, así como la minimización, tienen cotas específicas para poder ser calculadas. Esto nos restringe para poder calcular otro tipo de funciones más complejas por medio de algoritmos.

Un ejemplo simple es el algoritmo de bisección (o en general casi cualquier algoritmo de búsqueda de raíces). Aunque podemos realizar el método un número determinado de veces, esto no nos asegura que el resultado sea una aproximación satisfactoria al valor real de la raíz de la función. En cambio, lo que se realiza habitualmente es ejecutar el método un número indefinido de veces hasta alcanzar una precisión deseada. El número de iteraciones es indefinido puesto que la naturaleza de la función y los límites del intervalo que estemos utilizando para encontrar la raíz no permiten definir en general el número de iteraciones necesarias para que el algoritmo acabe, o siquiera determinar si el algoritmo realmente acabará; pues existe el caso donde no exista raíz y el método se ejecute sin fin.

Entonces, aunque tener la opción de ejecutar ciclos un número indefinido de veces puede llevarnos a tener algoritmos que se ejecuten sin fin, nos da la flexibilidad de poder computar un mayor número de funciones que las primitivo-recursivas no pueden.

En la práctica lo que permite realizar iteraciones indefinidas son los ciclos *while*, pues estos ejecutan instrucciones hasta que cierta condición deje de satisfacerse. Si tal condición siempre se satisface, entonces el algoritmo se ejecutará para siempre. Formalmente lo que nos permitirá capturar la idea de un ciclo *while* es la minimización. Como vimos anteriormente, ya tenemos una forma de encontrar el mínimo elemento que satisface cierta condición, pero restringida a que tal valor mínimo sea menor a cierta cota dada. Si ahora debilitamos la condición eliminando la cota, tenemos la flexibilidad que requeríamos, pues en este caso ya no es sabido en qué momento se encontrará el elemento mínimo; o si siquiera lo encontraremos.

Las funciones que permiten este tipo de comportamiento se denominan *funciones parcial-recursivas*. Su nombre se debe a que pueden existir entradas de la función que nunca devuelvan un valor concreto debido a que el algoritmo se ejecuta por siempre. Si una función de esta clase siempre devuelve un resultado para cualquier entrada, entonces se denomina *total-recursiva*.

- Definición 1.3** (Funciones parciales). 1. Sea $\uparrow$ un conjunto tal que $\uparrow \notin \omega$. Definimos entonces al conjunto $\omega^\uparrow := \omega \cup \{\uparrow\}$.
2. Una *función parcial* es una función $f : \omega^n \rightarrow \omega^\uparrow$ para algún $n \in \omega$.
3. Sean $n \in \omega$, $f : \omega^n \rightarrow \omega^\uparrow$ parcial y $x \in \omega^n$. Si $f(x) = \uparrow$, entonces lo denotaremos por $f(x) \uparrow$; y si existe $y \in \omega$ tal que $f(x) = y$, entonces lo denotaremos por $f(x) \downarrow$.
4. Sean $n \in \omega$ y $f : \omega^n \rightarrow \omega^\uparrow$ parcial, entonces se define el *dominio* de f como el conjunto $\text{dom}(f) := \{x \in \omega^n : f(x) \downarrow\}$.
5. Sean $n \in \omega$ y $f : \omega^n \rightarrow \omega^\uparrow$ parcial, entonces f se denomina *total* si $\text{dom}(f) = \omega^n$.

Definición 1.4 (Funciones parcial-recursivas). Sea $n \in \omega^+$. Una función parcial $f : \omega^n \rightarrow \omega^\uparrow$ se denomina *parcial-recursiva* si satisface alguna de las siguientes condiciones:

1. (Funciones básicas) La función f es una de las siguientes funciones:
 - La función *cero*: $Z : \omega^n \rightarrow \omega^\uparrow$ tal que para cada $x \in \omega^n$, $Z(x) = 0$.
 - Si $n = 1$, la función *sucesor*: $S : \omega \rightarrow \omega^\uparrow$ tal que para cada $x \in \omega$, $S(x) = x + 1$.
 - Para algún $k \in \omega^+$ con $k \leq n$, la función *proyección*: $\pi_k^n : \omega^n \rightarrow \omega^\uparrow$ tal que para cada $x_1, \dots, x_n \in \omega$, $\pi_k^n(x_1, \dots, x_n) = x_k$.

2. (Composición) Para algún $m \in \omega^+$ existen funciones parcial-recursivas $g_1, \dots, g_m : \omega^n \rightarrow \omega^\uparrow$ y $h : \omega^m \rightarrow \omega^\uparrow$ tales que para cada $x \in \omega^n$:

$$f(x) = \begin{cases} h(g_1(x), \dots, g_m(x)), & g_1(x) \downarrow, \dots, g_m(x) \downarrow \\ \uparrow, & \text{en otro caso} \end{cases}$$

3. (Recursión) Si $n > 1$ y existen funciones parcial-recursivas $g : \omega^{n-1} \rightarrow \omega^\uparrow$ y $h : \omega^{n+1} \rightarrow \omega^\uparrow$ tales que para cada $x \in \omega^{n-1}$ y cada $y \in \omega$ se satisface que:

$$f(x, 0) = g(x) \\ f(x, y+1) = \begin{cases} h(x, y, f(x, y)), & f(x, y) \downarrow \\ \uparrow, & f(x, y) \uparrow \end{cases}$$

4. (Minimización) Existe una función parcial-recursiva $g : \omega^{n+1} \rightarrow \omega^\uparrow$ tal que:

$$f(x) = \begin{cases} \min\{y \in \omega : g(x, y) = 0 \wedge (\forall z < y)(g(x, z) \downarrow)\}, & \text{si existe} \\ \uparrow, & \text{en otro caso} \end{cases}$$

A esta función se le denota por $\mu z(g(x, z) = 0)$ leída como “el mínimo z tal que $g(x, z) = 0$ ”.

Para cada $n \in \omega^+$, denotamos por Parcial_n al conjunto de todas las funciones parcial-recursivas de aridad n . Definimos al conjunto de funciones parcial-recursivas como $\text{Parcial} := \bigcup_{n \in \omega^+} \text{Parcial}_n$.

Una función se denomina *total-recursiva* si es parcial-recursiva y total.

Observemos que la minimización no acotada efectivamente puede verse como un ciclo *while*. Para poder encontrar el valor mínimo z que verifica $g(x, z) = 0$ podemos ir iterando sobre todos los valores desde $z = 0$ en adelante *mientras* no se satisfaga $g(x, z) = 0$. Si eventualmente un valor z sí satisface la igualdad, devolvemos tal valor; pero si ningún valor satisface la igualdad, entonces seguiremos buscando indefinidamente.

Algoritmo 5 *While*

```

 $z \leftarrow 0$ 
mientras  $g(x, z) \neq 0$  hacer
     $z \leftarrow z + 1$ 
fin mientras
regresar  $z$ 

```

Se tiene directamente lo siguiente.

Teorema 1.12. Toda función primitivo-recursive es total-recursive. Se tiene entonces que $\text{Prim} \subsetneq \text{Parcial}$.

El hecho de que existen funciones parcial-recursive que no son primitivo-recursive es un ejercicio muy sencillo de demostrar, puesto que la función constante $f(x, y) = 1$ es primitivo-recursive y luego $\mu z(f(x, z) = 0) \uparrow$ para cada $x \in \omega^+$. Aunque existen ejemplos de funciones parcial-recursive más interesantes, el punto central aquí es que el conjunto de funciones parcial-recursive es una extensión de las funciones primitivo-recursive justamente por la minimización no acotada.

Las demostraciones de la mayoría de los siguientes teoremas son análogas a las de sus contrapartes primitivo-recursive recurriendo a la definición 1.4.

Definición 1.5 (Conjuntos recursivos). Sean $n \in \omega^+$ y $A \subseteq \omega^n$, entonces A se denomina *recursivo* si χ_A es una función total-recursive.

Teorema 1.13. Se satisfacen cada una de las siguientes afirmaciones:

1. Sean $a \in \omega^\uparrow$ y $g : \omega^2 \rightarrow \omega^\uparrow$ parcial-recursive, entonces la función $f : \omega \rightarrow \omega^\uparrow$ definida como sigue para cada $x \in \omega$ es parcial-recursive:

$$\begin{aligned} f(0) &= a \\ f(x+1) &= g(x, f(x)) \end{aligned}$$

2. Sean $n \in \omega^+$ y $A, B \subseteq \omega^n$. Si A y B son recursivos, entonces $\omega^n \setminus A$, $A \cup B$ y $A \cap B$ son recursivos.
3. Sean $n \in \omega^+$ y φ, ψ fórmulas con n variables libres. Si φ y ψ definen conjuntos recursivos, entonces $\neg\varphi$, $\varphi \wedge \psi$ y $\varphi \vee \psi$ definen conjuntos recursivos en ω^n .
4. Sean $n \in \omega^+$, y $f : \omega^{n+1} \rightarrow \omega^\uparrow$ parcial-recursive y $g : \omega^n \rightarrow \omega^\uparrow$ total-recursive. Entonces:
 - a) la función $h : \omega^n \rightarrow \omega^\uparrow$ dada por $h(x) = \sum_{z < g(x)} f(x, z)$ para cada $x \in \omega^n$ es parcial-recursive.
 - b) la función $h : \omega^n \rightarrow \omega^\uparrow$ dada por $h(x) = \prod_{z < g(x)} f(x, z)$ para cada $x \in \omega^n$ es parcial-recursive.
5. Sean $n \in \omega^+$, $R \subseteq \omega^{n+1}$ recursivo y $f : \omega^n \rightarrow \omega^\uparrow$ total-recursive, entonces:

- el conjunto $\{x \in \omega^n : (\exists m < f(x))R(x, m)\}$ es recursivo.
 - el conjunto $\{x \in \omega^n : (\forall m < f(x))R(x, m)\}$ es recursivo.
6. Sean $n, m \in \omega^+$, funciones parcial-recursivas $g_1, \dots, g_m, g_{m+1} : \omega^n \rightarrow \omega^\uparrow$ y relaciones recursivas $R_1, \dots, R_m \subseteq \omega^n$ disjuntas a pares, entonces la función $f : \omega^n \rightarrow \omega^\uparrow$ tal que para cada $x \in \omega^n$:

$$f(x) = \begin{cases} g_1(x), & R_1(x) \\ \vdots & \\ g_m(x), & R_m(x) \\ g_{m+1}(x), & \text{en otro caso} \end{cases}$$

es parcial-recursiva.

7. Sean $n \in \omega^+$, $R \subseteq \omega^{n+1}$ una relación recursiva y $g : \omega^n \rightarrow \omega^\uparrow$ total-recursiva. Entonces la función $f : \omega^n \rightarrow \omega^\uparrow$ tal que para cada $x \in \omega^n$, $f(x) = (\mu z < g(x))R(x, z)$ es parcial-recursiva.
8. Sean $n \in \omega^+$ y $R \subseteq \omega^{n+1}$ una relación recursiva. Entonces la función $f : \omega^n \rightarrow \omega^\uparrow$ tal que para cada $x \in \omega^n$, $f(x) = \mu z R(x, z)$ es parcial-recursiva.

Demostración. Los únicos dos puntos cuya demostración vale la pena detallar son (6) y (8).

Para el caso del punto (6), consideremos a $n, m \in \omega^+$, las funciones parcial-recursivas $g_1, \dots, g_m, g_{m+1} : \omega^n \rightarrow \omega^\uparrow$ y las relaciones recursivas $R_1, \dots, R_m \subseteq \omega^n$ disjuntas a pares. Definamos $R_{m+1} = \omega^n \setminus \bigcup_{k=1}^m R_k$. Si definimos a f como:

$$f = \sum_{k=1}^{m+1} g_k \chi_{R_k}$$

entonces se tendría que si para algún $x \in \omega^n$ y algún $k \leq m+1$ se satisface que $g_k(x) \uparrow$, entonces $f(x) \uparrow$ incluso cuando $\chi_{R_k}(x) = 0$. Para evitar ese comportamiento definamos para cada $k \leq m+1$ la función auxiliar $h_k : \omega^{n+1} \rightarrow \omega^\uparrow$ como sigue para cada $x \in \omega^n$ y cada $y \in \omega$:

$$\begin{aligned} h_k(x, 0) &= 0 \\ h_k(x, y+1) &= g_k(x) \end{aligned}$$

Notemos que $h_k(x, 0) = 0$ y $h_k(x, 1) = g_k(x)$. Definamos entonces para cada $k \leq m+1$ las funciones $g'_k : \omega^n \rightarrow \omega^\uparrow$ para cada $x \in \omega^n$ como $g'_k(x) = h_k(x, \chi_{R_k}(x))$, la cual es parcial-recursiva. Notemos que estas funciones son tales que:

$$g'_k(x) = \begin{cases} g_k(x), & R_k(x) \\ 0, & \neg R_k(x) \end{cases}$$

De esta manera obtenemos que:

$$f = \sum_{k=1}^{m+1} g'_k$$

Ahora, para el punto (8), como R es recursiva, entonces $S := \omega^{n+1} \setminus R$ es recursiva, de lo cual se sigue que para cada $x \in \omega^n$:

$$f(x) = \mu z (\chi_S(x, z) = 0)$$

□

El punto (8) del teorema anterior es el que justifica nuestra interpretación de la minimización como el ciclo *while*. Dados x y la relación $R(x, z)$, si queremos encontrar el mínimo valor z para el cual $R(x, z)$, entonces iteramos desde $z = 0$ en adelante mientras no se satisfaga $R(x, z)$. En el primer momento que obtengamos un valor z que satisfaga $R(x, z)$, lo devolvemos, y entonces ese valor es el mínimo. Esto puede resumirse en el siguiente pseudocódigo:

Algoritmo 6 *While*

```

 $z \leftarrow 0$ 
mientras  $\neg R(x, z)$  hacer
     $z \leftarrow z + 1$ 
fin mientras
regresar  $z$ 

```

1.2. Computabilidad: Enfoque mecánico

El otro enfoque que tenemos de la computabilidad es uno con una visión más relacionada con la ejecución de una máquina. Este considera a los algoritmos como los objetos que normalmente utilizamos en la práctica: una lista de instrucciones finitas que ejecutamos para obtener un resultado. Aunque el enfoque inductivo nos facilita más operar los algoritmos para obtener otros, asegurando que mantenemos la cerradura, este enfoque es mejor para poder entender intuitivamente cómo funcionan procedimentalmente las funciones computables. Uno de los resultados más importantes será demostrar que el enfoque inductivo (con las funciones parcial-recursivas) y este nuevo son equivalentes mediante el teorema de la forma normal de Kleene.

Aunque el modelo más conocido para hablar de este enfoque son las máquinas de Turing, en este texto usaremos un modelo diferente, pero equivalente, el cual es el modelo de las *máquinas sin límite de registros* o *URM* (*Unlimited Register Machines*). Este modelo alternativo nos

provee de una mayor ventaja conceptual y matemática, puesto que aquí los datos que entran y salen de la máquina son números naturales directamente, a diferencia de las máquinas de Turing que tratan con alfabetos para formar palabras (generalmente en binario) que después se interpretan como números naturales.

De manera simple podemos pensar en una URM como una lista de instrucciones que se ejecutan sobre una cinta compuesta de un número infinito de celdas, donde podemos escribir en cada una un número natural. A cada celda de la cinta se le denomina *registro*, y cada registro puede contener un único número natural. Por el momento denotaremos por R a la cinta, por R_n al n -ésimo registro de R y por $r_n \in \omega$ al contenido de R_n para cada $n \in \omega^+$. (Figura 1.1)

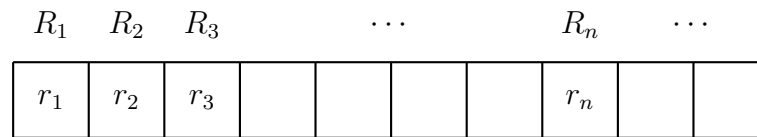


Figura 1.1: Cinta y registros

Ahora, las instrucciones que podemos utilizar se dividen en cuatro tipos:

1. Cero: Para cada $n \in \omega^+$ la instrucción $Z(n)$ indica que se anota 0 en el n -ésimo registro, es decir, en R_n se instancia $r_n \leftarrow 0$.
2. Sucesor: Para cada $n \in \omega^+$ la instrucción $S(n)$ indica que se le suma 1 al contenido del n -ésimo registro, es decir, en R_n se instancia $r_n \leftarrow r_n + 1$.
3. Transferir: Para cada $n, m \in \omega^+$ la instrucción $T(n, m)$ indica que se transfiere el contenido del n -ésimo registro al m -ésimo registro sin modificar el contenido del n -ésimo registro, es decir, en R_m se instancia $r_m \leftarrow r_n$.
4. Saltar: Para cada $n, m, k \in \omega^+$ la instrucción $J(n, m, k)$ indica que se compara el contenido de los registros n -ésimo y m -ésimo. Si sus contenidos son iguales, entonces se ejecuta la instrucción k ; y si no, entonces se sigue con la siguiente instrucción.

Una máquina URM es entonces una tupla de estas instrucciones, puesto que es importante conocer el orden en el cual se ejecutan las instrucciones.

Para entender mejor cómo se utilizan este tipo de máquinas, abordemos por ejemplo la ejecución de las instrucciones de la máquina $M = (J(2, 3, 5), S(1), S(3), J(1, 1, 1))$ tomando como punto de partida la cinta que inicialmente se instancia con $r_1 = 3$, $r_2 = 2$ y $r_n = 0$ para cada $n > 2$. (Figura 1.2)

La ejecución de la máquina M se realiza yendo en secuencia por cada uno de los elementos de la tupla. Por lo tanto, se ejecuta primero la instrucción $J(2, 3, 5)$, la cual compara los contenidos

3	2	0	0	0	0	0	0	0	
---	---	---	---	---	---	---	---	---	--

Figura 1.2: Cinta inicial para la máquina M

de los registros R_2 y R_3 . Puesto que $r_2 = 2$ y $r_3 = 0$, no se satisface que $r_2 = r_3$, por lo que se sigue con la siguiente instrucción. En esta iteración no se modificó ningún contenido de la cinta.

La siguiente instrucción en M es $S(1)$. En este caso se suma 1 al contenido en el primer registro, dando como resultado $r_1 = 3 + 1 = 4$. En este caso sí se modificó el contenido de la cinta, y el resultado se puede ver en la figura 1.3.

4	2	0	0	0	0	0	0	0	
---	---	---	---	---	---	---	---	---	--

Figura 1.3: Cinta después de ejecutar $S(1)$

Seguimos con la siguiente instrucción de la tupla que es $S(3)$, la cual indica que se debe sumar 1 al tercer registro, dando $r_3 = 0 + 1 = 1$, modificando la cinta como se ve en la figura 1.4.

4	2	1	0	0	0	0	0	0	
---	---	---	---	---	---	---	---	---	--

Figura 1.4: Cinta después de ejecutar $S(3)$

La siguiente instrucción es $J(1, 1, 1)$ la cual pide comparar los contenidos del registro 1. Puesto que trivialmente $r_1 = r_1$, debemos ejecutar la instrucción 1, es decir, se regresa al inicio de la tupla M y se realiza el proceso de nuevo. En esta iteración no se modificó la cinta.

Ahora, al ejecutar $J(2, 3, 5)$ de nuevo se comparan los contenidos en los registros R_2 y R_3 . Ya que $r_2 = 2 \neq 1 = r_3$, entonces proseguimos con la siguiente instrucción. En esta iteración tampoco se modificó la cinta.

Continuando con la instrucción $S(1)$, se suma 1 al contenido en el primer registro, dando como resultado $r_1 = 4 + 1 = 5$. La actualización de la cinta se puede ver en la figura 1.5.

5	2	1	0	0	0	0	0	0	
---	---	---	---	---	---	---	---	---	--

Figura 1.5: Cinta después de ejecutar $S(1)$ por segunda vez

Seguimos con la siguiente instrucción $S(3)$ obteniendo como resultado $r_3 = 1 + 1 = 2$, modificando la cinta como se ve en la figura 1.6.

5	2	2	0	0	0	0	0	0	
---	---	---	---	---	---	---	---	---	--

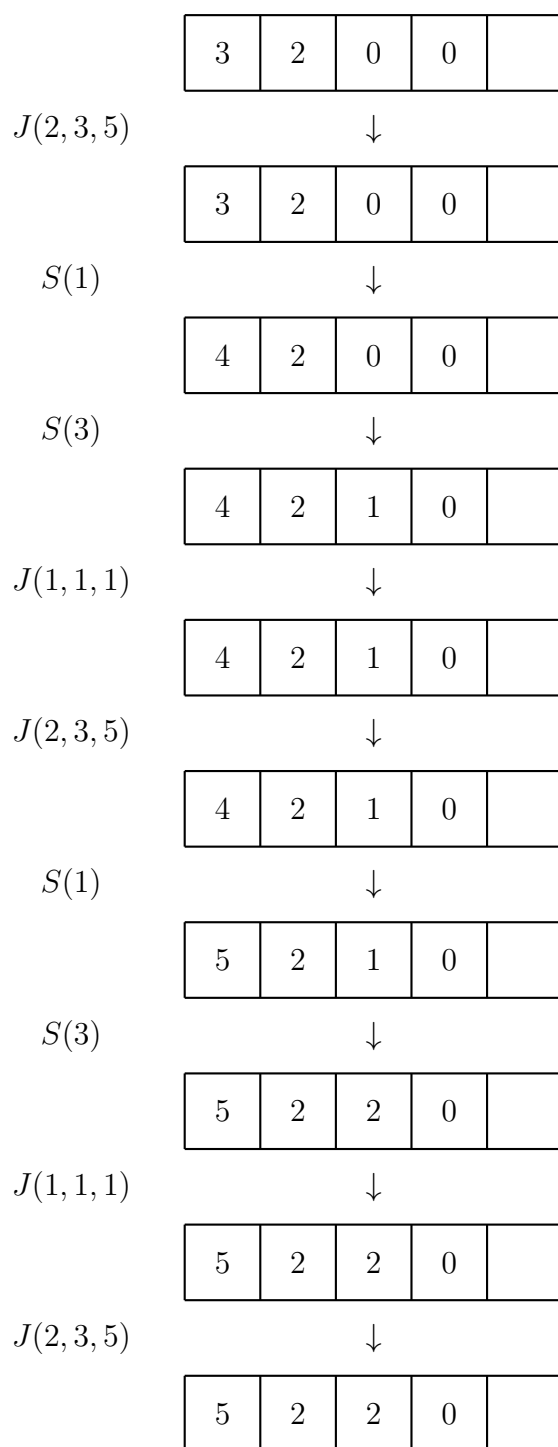
Figura 1.6: Cinta después de ejecutar $S(3)$ por segunda vez

La siguiente instrucción es $J(1, 1, 1)$. Dado que trivialmente $r_1 = r_1$, entonces se ejecuta la instrucción 1, volviendo al inicio de la tupla M . En esta iteración no se modificó la cinta. Cuando se ejecuta $J(2, 3, 5)$, de nuevo se comparan los contenidos en los registros R_2 y R_3 , y ya que $r_2 = 2 = r_3$, la siguiente instrucción a ejecutar es la número 5.

Notemos que la tupla de instrucciones M solamente tiene 4 elementos, luego no es posible realizar la instrucción 5 que se pidió en $J(2, 3, 5)$. Cuando esto pasa, se interpreta que el algoritmo ha terminado al no haber más instrucciones a ejecutar. Por lo tanto, la salida de este algoritmo es la cinta $r_1 = 5$, $r_2 = r_3 = 2$ y $r_n = 0$ para cada $n > 3$.

Por lo tanto, la ejecución completa de este algoritmo con la cinta de entrada $r_1 = 3$, $r_2 = 2$ y $r_n = 0$ para cada $n > 2$ es la que se muestra en la figura 1.7. En ella se puede ver cómo se van modificando los contenidos de los registros a medida que se ejecutan las instrucciones de la máquina M .

Este ejemplo nos permite entender cómo es la lógica de este tipo de objetos. Cada una de las cintas que se obtienen a lo largo de la ejecución del algoritmo puede verse como una sucesión de números naturales. Por ejemplo, la cinta de entrada del ejemplo se formaliza como la sucesión $(3, 2, 0, 0, 0, \dots)$. Luego, para cada iteración k del algoritmo se define la función $f_k : \omega \rightarrow \omega$ como la sucesión de números naturales contenidos en los registros de la cinta en esa iteración, de modo que para cada $n \in \omega^+$, $f_k(n) = r_n$. De esta manera la ejecución del algoritmo dado por una máquina URM es una sucesión de funciones de números naturales $(f_k)_{k \in \omega}$ donde cada f_k es la representación de los números de los registros de la cinta en la k -ésima iteración. Por convención la cinta de entrada del algoritmo se corresponde con la 0-ésima iteración. Por lo

Figura 1.7: Ejecución completa de la máquina M

tanto, como sucesiones, la ejecución de la máquina M puede verse de la siguiente manera:

$$\begin{array}{llllll}
f_0(1) = 3 & f_0(2) = 2 & f_0(3) = 0 & f_0(4) = 0 & \dots \\
f_1(1) = 3 & f_1(2) = 2 & f_1(3) = 0 & f_1(4) = 0 & \dots \\
f_2(1) = 4 & f_2(2) = 2 & f_2(3) = 0 & f_2(4) = 0 & \dots \\
f_3(1) = 4 & f_3(2) = 2 & f_3(3) = 1 & f_3(4) = 0 & \dots \\
f_4(1) = 4 & f_4(2) = 2 & f_4(3) = 1 & f_4(4) = 0 & \dots \\
f_5(1) = 4 & f_5(2) = 2 & f_5(3) = 1 & f_5(4) = 0 & \dots \\
f_6(1) = 5 & f_6(2) = 2 & f_6(3) = 1 & f_6(4) = 0 & \dots \\
f_7(1) = 5 & f_7(2) = 2 & f_7(3) = 2 & f_7(4) = 0 & \dots \\
f_8(1) = 5 & f_8(2) = 2 & f_8(3) = 2 & f_8(4) = 0 & \dots \\
f_9(1) = 5 & f_9(2) = 2 & f_9(3) = 2 & f_9(4) = 0 & \dots
\end{array}$$

Recordemos que el paso de un estado de la cinta a otro está completamente determinado por las instrucciones que tenga la máquina URM. Entonces, para poder manipular de manera adecuada tales instrucciones, y para poder describir de manera más explícita la sucesión de funciones $\{f_k\}_{k \in \omega}$, interpretamos a las instrucciones como tuplas de la siguiente forma:

$$Z(n) \rightarrow (0, n) \quad S(n) \rightarrow (1, n) \quad T(n, m) \rightarrow (2, n, m) \quad J(n, m, k) \rightarrow (3, n, m, k)$$

En este sentido se define al conjunto de *instrucciones* para las máquinas URM de la siguiente manera.

Definición 1.6 (Instrucciones). Se define al *conjunto de instrucciones* para las máquinas URM como el siguiente conjunto de tuplas:

$$\begin{aligned}
\text{Instr} := & \{(0, n) \in \omega^2 : n \in \omega^+\} \cup \{(1, n) \in \omega^2 : n \in \omega^+\} \\
& \cup \{(2, n, m) \in \omega^3 : n, m \in \omega^+\} \cup \{(3, n, m, k) \in \omega^4 : n, m, k \in \omega^+\}
\end{aligned}$$

Con ello se define ahora lo que es una máquina URM, entendida como una sucesión finita de instrucciones.

Definición 1.7 (Máquinas URM). Una *máquina URM* es una tupla no vacía $M \in \text{Instr}^{<\omega}$, es decir, es tal que existe $n \in \omega^+$ para el cual $M \in \text{Instr}^n$.

Recordando el ejemplo, al principio teníamos una cinta de entrada, la cual es una sucesión de elementos en ω , donde solo un número finito de ellos son diferentes de cero. Lo anterior es debido a que un algoritmo siempre se debe recibir como entrada un número finito de datos, lo cual podemos codificar como tal cinta con esos datos dispuestos a lo largo de ella. Entonces dar como entrada al algoritmo una tupla de datos $(x_1, \dots, x_k) \in \omega^k$ para algún $k \in \omega^+$ corresponde

a definir la función $f_0 : \omega \rightarrow \omega$ tal que para cada $n \in \omega^+$:

$$f_0(n) := \begin{cases} x_n, & n \leq k \\ 0, & \text{en otro caso} \end{cases}$$

Este concepto de datos de entrada es lo que justifica el hecho de que la cinta sea infinita, pues esa propiedad nos permite ingresar como entrada a un algoritmo un conjunto de datos arbitrariamente grande, e incluso poder usar los registros sobrantes como unidades de memoria que permitan almacenar datos auxiliares necesarios para computar lo que necesitamos.

Notemos que a lo largo de la descripción de la sucesión $\{f_k\}_{k \in \omega}$ no hemos descrito qué pasa con $f_k(0)$. La razón es porque se utiliza ese dato como auxiliar para poder determinar qué pasa entre cada iteración, almacenando el número de instrucción que se debe ejecutar de la máquina URM.

Con todo esto se define cómo es que se determina la sucesión de las cintas dadas por la ejecución de las instrucciones de la máquina URM.

Definición 1.8 (Ejecución de una máquina URM). Sean $M = (I_1, \dots, I_u)$ una máquina URM y $x = (x_1, \dots, x_n) \in \omega^n$ una tupla de entrada para algunos $n, u \in \omega^+$. Se define la *ejecución* de la máquina M con entrada x como la sucesión de funciones $\{f_k\}_{k \in \omega}$ definidas de la siguiente manera:

1. Para cada $m \in \omega$:

$$f_0(m) = \begin{cases} 1, & m = 0 \\ x_m, & 1 \leq m \leq n \\ 0, & \text{en otro caso} \end{cases}$$

2. Para cada $k \in \omega$, si $f_k(0) \leq u$ y además:

- a) $I_{f_k(0)} = (0, i)$ para algún $i \in \omega^+$, se tiene entonces que para cada $m \in \omega$:

$$f_{k+1}(m) = \begin{cases} f_k(0) + 1, & m = 0 \\ 0, & m = i \\ f_k(m), & \text{en otro caso} \end{cases}$$

- b) $I_{f_k(0)} = (1, i)$ para algún $i \in \omega^+$, se tiene entonces que para cada $m \in \omega$:

$$f_{k+1}(m) = \begin{cases} f_k(m) + 1, & m \in \{0, i\} \\ f_k(m), & \text{en otro caso} \end{cases}$$

c) $I_{f_k(0)} = (2, i, j)$ para algunos $i, j \in \omega^+$, se tiene entonces que para cada $m \in \omega$:

$$f_{k+1}(m) = \begin{cases} f_k(0) + 1, & m = 0 \\ f_k(i), & m = j \\ f_k(m), & \text{en otro caso} \end{cases}$$

d) $I_{f_k(0)} = (3, i, j, p)$ para algunos $i, j, p \in \omega^+$, se tiene entonces que para cada $m \in \omega$:

$$f_{k+1}(m) = \begin{cases} p, & m = 0 \text{ y } f_k(i) = f_k(j) \\ f_k(0) + 1, & m = 0 \text{ y } f_k(i) \neq f_k(j) \\ f_k(m), & \text{en otro caso} \end{cases}$$

3. Para cada $k > 0$, si $f_k(0) > u$, entonces $f_{k+1} = f_k$.

Decimos que la máquina M termina su ejecución dada la entrada x si existe $k \in \omega$ tal que $f_k(0) > u$.

Así formalmente hemos descrito el procedimiento que se había realizado en el ejemplo. Con esto dicho se define qué significa que una función sea *computable* desde este enfoque. De manera simple podemos definir el valor que produce una máquina URM, para cada entrada $x \in \omega^n$ con $n \in \omega^+$, como el contenido del primer registro si la ejecución acaba.

Por ejemplo, considerando el ejemplo que describimos anteriormente, las instrucciones generan un algoritmo tal que dada la entrada $(3, 2)$, al acabar su ejecución, devuelve el valor 5 en su primer registro. Resulta de hecho que la máquina que describimos, para cada par de entrada $(x, y) \in \omega^2$, el algoritmo termina y el valor del primer registro es su suma $x + y$. Por lo tanto, tal máquina es una implementación algorítmica de la función suma.

Sin embargo, dada la misma máquina $M = (J(2, 3, 5), S(1), S(3), J(1, 1, 1))$ podemos darle de entrada ternas $(x, y, z) \in \omega^3$, generando como resultado una función $f : \omega^3 \rightarrow \omega^+$ con un comportamiento diferente al de la suma. Consideremos por ejemplo la entrada $(1, 1, 2)$, entonces

se obtiene la siguiente ejecución:

$$\begin{array}{rcl}
 & & (1, 1, 2, 0, 0, \dots) \\
 J(2, 3, 5) & \rightarrow & (1, 1, 2, 0, 0, \dots) \\
 S(1) & \rightarrow & (2, 1, 2, 0, 0, \dots) \\
 S(3) & \rightarrow & (2, 1, 3, 0, 0, \dots) \\
 J(1, 1, 1) & \rightarrow & (2, 1, 3, 0, 0, \dots) \\
 J(2, 3, 5) & \rightarrow & (2, 1, 3, 0, 0, \dots) \\
 S(1) & \rightarrow & (3, 1, 3, 0, 0, \dots) \\
 S(3) & \rightarrow & (3, 1, 4, 0, 0, \dots) \\
 J(1, 1, 1) & \rightarrow & (3, 1, 4, 0, 0, \dots) \\
 J(2, 3, 5) & \rightarrow & (3, 1, 4, 0, 0, \dots) \\
 S(1) & \rightarrow & (4, 1, 4, 0, 0, \dots) \\
 & & \vdots
 \end{array}$$

Notemos que al seguir ejecutando las instrucciones el tercer registro siempre es mayor al valor del segundo registro, entonces la instrucción $J(2, 3, 5)$ nunca redirige a la instrucción 5; por lo que la ejecución nunca termina. En este caso se tiene que $f(1, 1, 2) \uparrow$. Se puede verificar también que si $(x, y, z) \in \omega^3$ es tal que $y \geq z$, entonces $f(x, y, z) = x + y - z$; y si $y < z$, entonces $f(x, y, z) \uparrow$.

Con esto podemos apreciar que la misma máquina URM puede darnos diferentes funciones únicamente variando el número de datos de entrada. Por lo tanto, la aridad de la función es esencial para determinar su comportamiento al ser computada mediante una máquina URM.

Definición 1.9 (Funciones URM-computables). Para cada máquina URM $M \in \text{Instr}^k$ para algún $k \in \omega^+$ y cada $n \in \omega^+$ se define la *función URM-computable* de M con aridad n como la función parcial $\varphi_M^{(n)} : \omega^n \rightarrow \omega^\uparrow$ tal que para cada $x \in \omega^n$ con ejecución $\left\{ f_m^{(x)} \right\}_{m \in \omega}$:

$$\varphi_M^{(n)}(x) = \begin{cases} f_p^{(x)}(1), & \exists u \left(f_u^{(x)}(0) > k \right) \wedge \left(p = \min \left\{ u : f_u^{(x)}(0) > k \right\} \right) \\ \uparrow, & \text{en otro caso} \end{cases}$$

Una función parcial $f : \omega^n \rightarrow \omega^\uparrow$ para algún $n \in \omega^+$ se denomina *URM-computable* si existe una máquina URM M tal que $f = \varphi_M^{(n)}$.

Definimos al conjunto URMComp como el conjunto de las funciones parciales que son URM-computables.

Notación 1.4. Dada una máquina URM M , denotaremos por φ_M a la función URM-computable $\varphi_M^{(1)}$.

Como mencionamos antes, el objetivo de esta sección será demostrar que ambos enfoques de la computabilidad son equivalentes, o más formalmente, demostrar que se tiene la igualdad de conjuntos $\text{Parcial} = \text{URMComp}$. La contención más sencilla de demostrar es que toda función parcial-recursiva es URM-computable.

Teorema 1.14. Toda función parcial-recursiva es URM-computable.

Demostración. Probemos por inducción sobre el conjunto de funciones parcial-recursivas. Mostremos primero que las funciones básicas son URM-computables. Sean $n, k \in \omega^+$ con $k \leq n$ y definamos las máquinas $M_1 = ((0, 1))$, $M_2 = ((1, 1))$ y $M_3 = ((2, k, 1))$, entonces se sigue directamente que $Z = \varphi_{M_1}^{(n)}$, $S = \varphi_{M_2}$ y $\pi_k^n = \varphi_{M_3}^{(n)}$.

Considérese ahora el caso de la **composición**. Sean $n, k \in \omega$, $g_1, \dots, g_k : \omega^n \rightarrow \omega^\uparrow$ y $h : \omega^k \rightarrow \omega^\uparrow$ funciones parcial-recursivas, y supongamos que existen máquinas URM $G_1, \dots, G_k$ y H tales que $h = \varphi_H^{(k)}$ y para cada $j \leq k$, $g_j = \varphi_{G_j}^{(n)}$. Mostremos entonces que la función $f : \omega^n \rightarrow \omega^\uparrow$ definida por $f = h(g_1, \dots, g_k)$ es URM-computable.

Para ello, describiremos cual será el proceso que debe realizar la máquina URM F que computa a f . Para cada $j \leq k$ denotemos por $M_j \in \omega^+$ al registro máximo al cual hace referencia la máquina G_j y por M_0 al registro máximo al cual hace referencia la máquina H . Defínase entonces $M = \max\{M_0, \dots, M_k, n, k\}$. El valor M nos servirá como un parámetro auxiliar para poder definir de manera adecuada la máquina F . La idea es que la máquina F primero computará los valores de $g_1, \dots, g_k$ y los almacenará en los registros $R_1, \dots, R_k$, para luego computar el valor de h con esos valores como entrada.

Dada la entrada $x \in \omega^n$, el primer paso es transferir sus valores en los registros desde R_{M+k+1} en adelante. La razón es guardar los datos de entrada en estos registros que funcionan como memoria auxiliar, ya que es necesario conocerlos para computar cada una de las funciones g_j ($j \leq k$). Además, cada uno de los registros $R_{M+1}, \dots, R_{M+k}$ se utilizarán para almacenar los resultados de cada una de las funciones g_j , y finalmente se computará la función h con estos valores como entrada.

Luego, después de transferir cada uno de los datos de entrada, se ejecutará la máquina G_1 (ya que los datos de entrada originales no se modificaron en los registros iniciales). Después de ejecutarse, se transferirá el valor del primer registro a R_{M+1} . Después, se inicializará en 0 cada uno de los registros desde R_1 hasta R_M , y entonces se transferirán los datos de entrada guardados en $R_{M+k+1}, \dots, R_{M+k+n}$ en los registros $R_1, \dots, R_n$. Repetiremos este mismo proceso para cada máquina G_j ($j \leq k$) guardando sus resultados en los registros R_{M+j} .

Al finalizar lo anterior, se tienen guardados los valores de $g_1(x), \dots, g_k(x)$ en los regis-

tros $R_{M+1}, \dots, R_{M+k}$ por hipótesis de inducción. El siguiente paso es inicializar los registros $R_1, \dots, R_M$ en 0 y transferir los valores de los registros $R_{M+1}, \dots, R_{M+k}$ a los registros $R_1, \dots, R_k$. Finalmente, se ejecutará la máquina H con estos valores como entrada, y el resultado final estará en el primer registro.

Por hipótesis de inducción se satisface que en el primer registro de la se aloja el valor $h(g_1(x), \dots, g_k(x)) = f(x)$. Puesto que $x \in \omega^n$ fue arbitrario, se concluye que $f = \varphi_F^{(n)}$. Este proceso puede visualizarse en la figura 1.8.

	zona de trabajo		resultados parciales	memoria auxiliar
	R_1	$R_2, \dots, R_M$	$R_{M+1}, \dots, R_{M+k}$	$R_{M+k+1}, \dots, R_{M+k+n}$
Entrada	x_1	$x_2 \cdots x_n \ 0 \cdots 0$	$0 \cdots 0$	$0 \cdots 0$
Se copia la entrada	x_1	$x_2 \cdots x_n \ 0 \cdots 0$	$0 \cdots 0$	$x_1 \cdots x_n$
Se ejecuta G_1	$g_1(x)$	residuos de G_1	$g_1(x) \ 0 \cdots 0$	$x_1 \cdots x_n$
Se restaura la entrada	x_1	$x_2 \cdots x_n \ 0 \cdots 0$	$g_1(x) \ 0 \cdots 0$	$x_1 \cdots x_n$
Se repite con $G_2, \dots, G_k$	$g_k(x)$	residuos de G_k	$g_1(x) \cdots g_k(x)$	$x_1 \cdots x_n$
Se cargan los resultados	$g_1(x)$	$g_2(x) \cdots g_k(x) \ 0 \cdots 0$	$g_1(x) \cdots g_k(x)$	$x_1 \cdots x_n$
Se ejecuta H	$f(x)$	residuos de H	$g_1(x) \cdots g_k(x)$	$x_1 \cdots x_n$

Figura 1.8: Evolución de la cinta de la máquina F al computar $f = h(g_1, \dots, g_k)$ sobre la entrada $x = (x_1, \dots, x_n)$

Continuemos con el caso de la **recursión primitiva**. Sean $n \in \omega$, $g : \omega^n \rightarrow \omega^\uparrow$ y $h : \omega^{n+2} \rightarrow \omega^\uparrow$ funciones parcial-recursivas, y supongamos que existen máquinas URM G y H tales que $g = \varphi_G^{(n)}$ y $h = \varphi_H^{(n+2)}$. Mostremos entonces que la función $f : \omega^{n+1} \rightarrow \omega^\uparrow$ definida como sigue para cada $x \in \omega^n$ y $y \in \omega$ es URM-computable:

$$\begin{aligned} f(x, 0) &= g(x) \\ f(x, y + 1) &= h(x, y, f(x, y)) \end{aligned}$$

De igual forma describamos cuál es el proceso que debe seguir la máquina URM F la cual satisface $f = \varphi_F^{(n+1)}$. Sean $x \in \omega^n$, $y \in \omega$ y $z = (x, y)$, entonces la idea es que la máquina

F computará primero el valor $f(x, 0) = g(x)$ como caso base y añadiremos instrucciones que nos permitan tener un contador j el cual se comparará con el valor y . Si $j < y$, entonces se computará $f(x, j + 1) = h(x, j, f(x, j))$, y si $j + 1 = y$, entonces acaba el proceso; si no, entonces repetimos lo mismo.

Sean M_g y M_h los registros máximos a los cuales hacen referencia las máquinas G y H , respectivamente. Definase $M = \max\{M_g, M_h, n + 2\}$. Al ingresar la entrada $z = (x, y)$, transferiremos únicamente los datos x en los registros $R_{M+1}, \dots, R_{M+n}$. Para el caso del valor y , lo transferiremos en el registro R_{M+n+3} , ya que el registro R_{M+n+1} alojará el valor del contador j y el registro R_{M+n+2} alojará el valor $f(x, j)$.

Después, inicializaremos el valor y en el registro R_{n+1} con 0. Entonces se ejecutará la máquina G , el cual por hipótesis devuelve el valor $g(x)$ en el primer registro.

Sea j el valor en el registro R_{M+n+1} . El siguiente bloque de instrucciones será comparar el valor de los registros R_{M+n+1} (el valor j) y R_{M+n+3} (el valor y); terminamos la ejecución si son iguales, y realizamos lo siguiente si no: transferimos el valor del primer registro ($f(x, j)$) al registro R_{M+n+2} , sumamos uno al valor del registro R_{M+n+1} ($j \leftarrow j + 1$), inicializamos con 0 los registros $R_1, \dots, R_M$, transferimos los valores de los registros $R_{M+n+1}, \dots, R_{M+n+2}$ en los registros $R_1, \dots, R_{n+2}$, y ejecutamos las instrucciones de la máquina H para obtener el valor $f(x, j + 1) = h(x, j, f(x, j))$ en el primer registro (lo cual se asegura por hipótesis de inducción). Repetimos este mismo bloque.

De esta manera, cuando $j + 1 = y$, al haber calculado $f(x, j + 1)$, se tendrá al final de la ejecución en el primer registro el valor $f(x, y)$. Puesto que $x \in \omega^n$ y $y \in \omega$ fueron arbitrarios, se satisface que $f = \varphi_F^{(n+1)}$. Este proceso puede visualizarse en la figura 1.9.

Finalicemos con el caso de la **minimización no acotada**. Sean $n \in \omega$ y $g : \omega^{n+1} \rightarrow \omega^\uparrow$ una función parcial-computable, y supongamos que existe una máquina URM G la cual satisface que $g = \varphi_G^{(n+1)}$. Mostremos entonces que la función $f : \omega^n \rightarrow \omega^\uparrow$ definida como sigue para cada $x \in \omega^n$ es URM-computable:

$$f(x) = \mu y (g(x, y) = 0)$$

De igual manera, describiremos el procedimiento que debe seguir una máquina URM F de modo que se satisfaga $f = \varphi_F^{(n)}$. Sea $x \in \omega^n$, entonces la idea es tener un contador $j \in \omega$ de modo que se calculará el valor $g(x, j)$ y verificaremos si es igual a cero; si es igual a cero, terminamos la ejecución y devolvemos el valor j , y si no, entonces aumentamos en uno a j ($j \leftarrow j + 1$) y repetimos el proceso.

Sea M_G el registro máximo al cual hace referencia la máquina G , entonces defínase $M = \max\{M_G, n + 1\}$. El primer paso es transferir los datos de entrada en el contenido de los registros $R_{M+1}, \dots, R_{M+n}$. El contenido del registro R_{M+n+1} nos servirá como el contador j y el contenido del registro R_{M+n+2} (el cual contiene el valor de cero) para comparar si la función g se anuló.

Sea j el contenido del registro R_{M+n+1} . El bloque a realizar es, primero, ejecutar las ins-

	zona de trabajo		copia de x	contador	valor previo	copia de y
	R_1	$R_2, \dots, R_M$	$R_{M+1}, \dots, R_{M+n}$	R_{M+n+1}	R_{M+n+2}	R_{M+n+3}
Entrada $z = (x, y)$	x_1	$x_2 \cdots x_n \ y \ 0 \cdots 0$	$0 \cdots 0$	0	0	0
Se copia la entrada y se borra y de R_{n+1}	x_1	$x_2 \cdots x_n \ 0 \cdots 0$	$x_1 \cdots x_n$	0	0	y
Se ejecuta G (caso base)	$f(x, 0)$	residuos de G	$x_1 \cdots x_n$	0	0	y
Si $j \neq y$: se guarda $f(x, j)$ y se carga la entrada de H	x_1	$x_2 \cdots x_n \ j \ f(x, j) \ 0 \cdots 0$	$x_1 \cdots x_n$	j	$f(x, j)$	y
Se ejecuta H	$f(x, j+1)$	residuos de H	$x_1 \cdots x_n$	j	$f(x, j)$	y
Se incrementa el contador	$f(x, j+1)$	residuos de H	$x_1 \cdots x_n$	$j+1$	$f(x, j)$	y
Al alcanzarse $j = y$, termina	$f(x, y)$	residuos de H	$x_1 \cdots x_n$	y	$f(x, y-1)$	y

 mientras $j \neq y$

Figura 1.9: Evolución de la cinta de la máquina F al computar por recursión primitiva f sobre la entrada $z = (x, y)$

trucciones de la máquina G para obtener en el primer registro el valor $g(x, j)$ por hipótesis de inducción, después se comparan los contenidos de los registros R_1 y R_{M+n+2} : si sí son iguales, entonces se ejecuta la última instrucción que describiremos más adelante; si no, entonces sumamos uno al contenido del registro R_{M+n+1} ($j \leftarrow j + 1$), inicializamos con cero los registros $R_1, \dots, R_M$, y transferimos el contenido de los registros $R_{M+1}, \dots, R_{M+n+1}$ en los registros $R_1, \dots, R_{n+1}$. Repetimos el bloque.

La última instrucción de la máquina es transferir el contenido del registro R_{M+n+1} (el valor j) en el primer registro y se termina la ejecución.

Si en efecto se satisface que $g(x, j) = 0$, entonces eso únicamente ocurrió debido a que para cada $k < j$, $g(x, k) \downarrow$ y (por construcción) $g(x, k) \neq 0$. Por lo tanto, efectivamente se satisface que, al finalizar la ejecución, el primer registro contiene el valor $\mu j(g(x, j) = 0) = f(x)$. Puesto que $x \in \omega^n$ es arbitrario, se satisface que $f = \varphi_F^{(n+1)}$. Este proceso puede visualizarse en la figura 1.10.

Por lo tanto, por inducción sobre el conjunto de funciones parcial-recursivas, se satisface que para toda función $f : \omega^n \rightarrow \omega^\uparrow$, para algún $n \in \omega$, parcial-recursiva existe una máquina URM F la cual satisface que $f = \varphi_F^{(n)}$. $\square$

La demostración de este teorema no solamente nos asegura que toda función parcial-

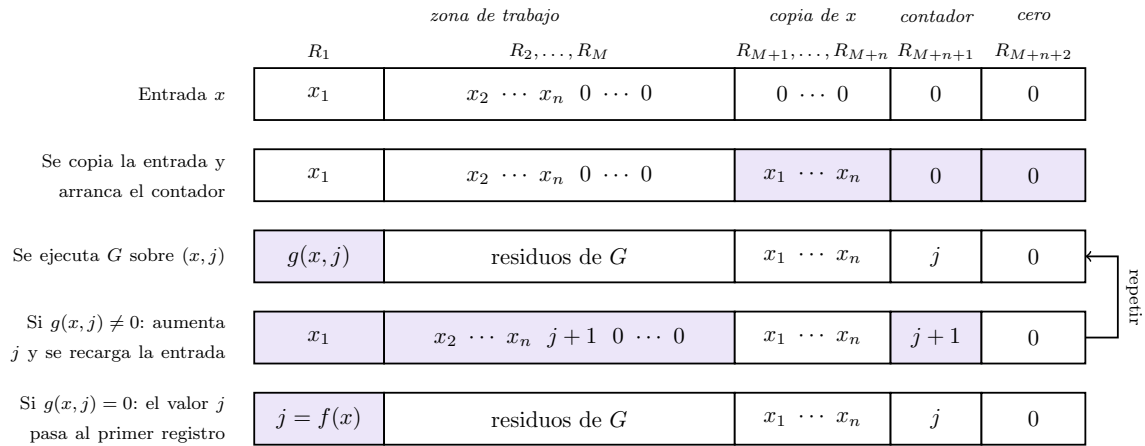


Figura 1.10: Evolución de la cinta de la máquina F al computar $f(x) = \mu y(g(x, y) = 0)$ sobre la entrada $x = (x_1, \dots, x_n)$

recursiva es URM-computable, sino que también nos da un método constructivo de cómo describir las instrucciones que debe seguir una máquina URM para computarla. Ese enfoque constructivo en casi todas las demostraciones de las propiedades de los objetos computables es la gran ventaja de toda la teoría, ya que todo objeto es teóricamente construible por un ser humano.

1.2.1. Codificación de listas

La demostración de $\text{URMComp} \subseteq \text{Parcial}$ necesita una herramienta esencial que es la *enumeración*. Esto se refiere a que cada máquina se corresponde con un único número natural que lo identifica y que codifica toda su información, lo cual permite tratar como entradas de funciones parcial-recursivas a los algoritmos mismos. Para poder realizarlo debemos corresponder primero las tuplas de números naturales con números naturales que, igualmente, codifiquen toda su información.

Aunque existen varias formas de poder codificar listas de naturales, la forma más simple es por medio del teorema fundamental de la aritmética. La idea es generar una inyección entre las listas y los naturales tratando a cada elemento de la lista como potencias de productos de primos. Por ejemplo, si tenemos la lista $(1, 2, 3) \in \omega^3$, entonces podemos mapearla al número $2^1 3^2 5^3$. La ventaja de este método es que el teorema fundamental de la aritmética nos asegura que tal producto de primos es único, por lo que podríamos recuperar la lista únicamente factorizando el número que lo representa y recopilando sus potencias.

Sin embargo, un problema de esta codificación es cuando tenemos listas con ceros, como por

ejemplo las listas $(1, 2, 0, 0)$ y $(1, 2)$. Notemos en este caso que ambas listas serían mapeadas al número $2^1 3^2$ justamente porque existen ceros al final de la primera lista. Una forma para poder arreglar esto es sumar 1 a cada exponente en el mapeo. En este caso, dadas las listas anteriores, podríamos mapear $(1, 2, 0, 0)$ a $2^{1+1} 3^{2+1} 5^{0+1} 7^{0+1}$ y mapear $(1, 2)$ a $2^{1+1} 3^{2+1}$, donde las listas ahora están mapeadas a diferentes números naturales. Esta forma de mapear listas ya asegura una inyección debido a que ya no tendríamos potencias nulas al trabajar con listas que contengan ceros como elementos.

Teorema 1.15. Sea la función $f : \omega^{<\omega} \rightarrow \omega$ tal que para cada $n \in \omega$ y $x = (x_1, \dots, x_n) \in \omega^n$ definida por:

$$f(x) = \prod_{k=1}^n p_k^{x_k+1}$$

donde p_k es el k -ésimo número primo, entonces f es inyectiva.

Demostración. Sean $x, y \in \omega^{<\omega}$ tales que $f(x) = f(y)$, entonces mostremos que $x = y$.

Si $x = \emptyset$ o $y = \emptyset$, entonces $f(x) = 1 = f(y)$, por lo que $x = y = \emptyset$, debido a que la única lista que se mapea a 1 es la lista vacía. Por lo tanto, supongamos que $x, y \neq \emptyset$.

Sean $n, m, x_1, \dots, x_n, y_1, \dots, y_m \in \omega$ tales que $x = (x_1, \dots, x_n)$ y $y = (y_1, \dots, y_m)$, entonces se tiene que:

$$f(x) = \prod_{k=1}^n p_k^{x_k+1} ; f(y) = \prod_{k=1}^m p_k^{y_k+1}$$

Ya que $f(x) = f(y)$, por el teorema fundamental de la aritmética sabemos que $n = m$ y para cada $k \leq n$ se tiene que $p_k^{x_k+1} = p_k^{y_k+1}$, de lo cual se sigue que $x_k = y_k$. Por lo tanto, $x = y$. $\square$

Definición 1.10 (Codificación de listas). Para cada $n \in \omega^+$ se define la función de *codificación* $f_n : \omega^n \rightarrow \omega$ tal que para cada $x = (x_1, \dots, x_n) \in \omega^n$:

$$f_n(x) = \prod_{k=1}^n p_k^{x_k+1}$$

Para cada $n \in \omega^+$ y cada $x = (x_1, \dots, x_n) \in \omega^n$ se denotará por $\langle x \rangle = \langle x_1, \dots, x_n \rangle$ a $f_n(x)$.

Teorema 1.16. Las funciones de codificación son primitivo-recursive.

Demostración. Denotemos por $f : \omega \rightarrow \omega$ a la función que devuelve el $(n+1)$ -ésimo número primo, entonces se sigue para cada $n \in \omega^+$ que para cada $x \in \omega^n$:

$$\langle x \rangle = \prod_{k < n} (f(k))^{\pi_{k+1}^n(x)+1}$$

Ya que f , la función proyección, la potencia, suma y producto acotado son primitivo-recursivas, se sigue que las funciones codificación son primitivo-recursivas. $\square$

De esta forma podemos definir entonces al conjunto de *códigos* o *secuencias* de listas como el conjunto de números naturales que son código de una lista. De manera auxiliar definiremos a 1 como la codificación de la lista vacía.

Definición 1.11 (Secuencias). Definimos al conjunto de *secuencias* o *códigos* como el siguiente subconjunto de ω :

$$\text{Seq} := \{1\} \cup \{\langle x \rangle \in \omega : x \in \omega^{<\omega}\}$$

Teorema 1.17. El conjunto de códigos Seq es primitivo-recursivo.

Demostración. Denotemos por $P \subseteq \omega$ al conjunto de números primos y $D \subseteq \omega^2$ a la relación “divide a”. Notemos que la forma de caracterizar los códigos es que son productos de potencias de números primos consecutivos, lo cual puede verse como el conjunto de números n tales que si p es primo y divide a n , entonces todo primo menor a p también divide a n . Definamos entonces la fórmula:

$$\varphi(n) = (n > 0 \wedge (\forall p \leq n)((P(p) \wedge D(p, n)) \rightarrow (\forall q \leq p)(P(q) \rightarrow D(q, n))))$$

Se tiene entonces que $\text{Seq} = \{n \in \omega : \varphi(n)\}$. Para el caso de 1 se satisface también que $\varphi(1)$ por vacuidad ya que ningún primo es menor o igual que 1. Ya que P y D son primitivo-recursivas, por los teoremas 1.6 y 1.8 sabemos entonces que Seq es primitivo-recursiva. $\square$

El objetivo ahora entonces es verificar que ciertas operaciones que podemos realizar sobre esta codificación son primitivo-recursivas, ya que nos permitirán trabajar con ellas de manera efectiva.

Definición 1.12 (Máxima potencia, acceso y longitud). Definimos las siguientes funciones:

1. la función *máxima potencia* $\text{MaxP} : \omega^2 \rightarrow \omega$ definida como sigue para cada $x, y \in \omega$:

$$\text{MaxP}(x, y) = \begin{cases} \max\{n \in \omega : x^n \mid y\}, & x > 1, y > 0 \\ 0, & \text{en otro caso} \end{cases}$$

2. para cada $k \in \omega$, la función *acceso* al $(k + 1)$ -ésimo elemento $f_k : \omega \rightarrow \omega$ definida como

sigue para cada $e \in \omega$:

$$f_k(e) = \begin{cases} x_k, & \exists n(\exists x_0, \dots, x_{n-1} \in \omega)(k < n \wedge e = \langle x_0, \dots, x_{n-1} \rangle) \\ 0, & \text{en otro caso} \end{cases}$$

Denotaremos por $e[k]$ a $f_k(e)$.

3. la función *longitud* $\text{Len} : \omega \rightarrow \omega$ definida como sigue para cada $e \in \omega$:

$$\text{Len}(e) = \begin{cases} n, & \exists n \exists x \in \omega^n (e = \langle x \rangle) \\ 0, & \text{en otro caso} \end{cases}$$

Teorema 1.18. Las funciones máxima potencia, acceso y longitud son primitivo-recursivas.

Demostración. Para la función máxima potencia la estrategia es utilizar su negación: la máxima potencia es equivalente a la mínima potencia que ya no divide a la secuencia (menos uno). Si $D \subseteq \omega^2$ es la relación “divide a”, se tiene lo siguiente para cada $x, y \in \omega$:

$$\text{MaxP}(x, y) = \chi_{>}(x, 1) \chi_{>}(y, 0) ((\mu n < y + 1) \neg D(x^n, y) - 1)$$

donde la resta se entiende como la resta en ω . Ya que D es primitivo-recursiva, se sigue que MaxP es primitivo-recursiva.

Sea ahora $k \in \omega$, entonces para la función de acceso al $(k + 1)$ -ésimo elemento, si denotamos por $f : \omega \rightarrow \omega$ a la función que devuelve el $(n + 1)$ -ésimo número primo, el $(k + 1)$ -ésimo elemento es la potencia m más grande que divide a la secuencia e , es decir, tal que $(f(k))^m \mid e$, pero la cual debemos restarle uno debido a que al codificar las listas sumamos 1 para asegurar la inyectividad. Se tiene entonces lo siguiente para cada $e \in \omega$:

$$e[k] = \chi_{\text{Seq}}(e) (\text{MaxP}(f(k), e) - 1)$$

Finalmente, para la función longitud la estrategia es buscar el primo más grande que divide la secuencia (ya que usamos primos consecutivos). Para ello usamos su negación: buscar el primo más grande es equivalente a buscar el menor primo que no divide a la secuencia (menos uno). Se tiene entonces que para cada $e \in \omega$:

$$\text{Len}(e) = \chi_{\text{Seq}}(e) (\mu n < e + 1) \neg D(f(n), e)$$

□

Teorema 1.19. Para cada $e \in \text{Seq}$ y cada $k < \text{Len}(e)$, se satisface que $e[k] < e$.

Demostración. Se sabe que para cada número natural $n \in \omega$ se satisface que $n < 2^n$, entonces para cada $e \in \text{Seq}$ y cada $k < \text{Len}(e)$ se satisface lo siguiente:

$$e[k] < 2^{e[k]} \leq p_{k+1}^{e[k]+1} \leq e$$

□

Teorema 1.20 (Máximo). Definimos a la función *máximo* $\text{Max} : \omega \rightarrow \omega$ de la siguiente manera:

$$\text{Max}(e) := \begin{cases} \text{máx}\{x_1, \dots, x_n\}, & \text{Seq}(e) \text{ y } e = \langle x_1, \dots, x_n \rangle \\ 0, & \text{en otro caso} \end{cases}$$

Entonces Max es primitivo-recursive.

Demostración. Definimos como ayuda a la relación $M \subseteq \omega^2$ de la siguiente forma:

$$M(e, m) \iff (\exists n < \text{Len}(e))(e[n] = m \wedge (\forall k < \text{Len}(e))(e[k] \leq e[n]))$$

Esta relación en esencia nos dice que un par (e, m) está en M si y solo si m es elemento de la lista codificada por e y es el elemento más grande de ella. Puesto que la lista codificada es finita, siempre tiene un único elemento máximo (considerando que no sea vacía). Gracias a los teoremas 1.6 y 1.8, afirmamos que M es primitivo-recursive.

De esta manera se recupera el elemento máximo iterando sobre todos los números m menores o iguales a e , por el teorema 1.19, verificando si $M(e, m)$. Ya que el máximo es único, entonces aseguramos que a lo más un único número m satisface que $M(e, m)$, y por lo tanto podemos definir a la función máximo como sigue:

$$\text{Max}(e) = \chi_{\text{Seq}}(e) \sum_{m=0}^e m \chi_M(e, m)$$

Puesto que Seq y M son primitivo-recursive, entonces Max es primitivo-recursive. □

Esta función es necesaria para poder demostrar que diferentes funciones son primitivo-recursive cuando codifiquemos las máquinas URM.

1.2.2. Codificación de máquinas URM

Con todo lo anterior desarrollado, ahora podemos codificar las máquinas URM como números naturales. Recordemos que definimos a las máquinas URM como una secuencia de instrucciones, y cada una de ellas es a su vez representada como una lista. Con todo esto, a cada

máquina URM podemos directamente traducirla como la codificación de tales listas. Por ejemplo, si tomamos la máquina URM $M = (J(2, 3, 5), S(1), S(3), J(1, 1, 1))$ que calculaba la suma, tendríamos la siguiente secuencia de listas:

$$M = ((3, 2, 3, 5), (1, 1), (1, 3), (3, 1, 1, 1))$$

La primera lista $(3, 2, 3, 5)$ se codificaría como $\langle 3, 2, 3, 5 \rangle = 2^4 3^3 5^4 7^6$ por ejemplo. Las demás listas se codificarían de la siguiente forma:

$$\begin{aligned} (3, 2, 3, 5) &\rightarrow \langle 3, 2, 3, 5 \rangle = 2^4 3^3 5^4 7^6 \\ (1, 1) &\rightarrow \langle 1, 1 \rangle = 2^2 3^2 \\ (1, 3) &\rightarrow \langle 1, 3 \rangle = 2^2 3^4 \\ (3, 1, 1, 1) &\rightarrow \langle 3, 1, 1, 1 \rangle = 2^4 3^2 5^2 7^2 \end{aligned}$$

Entonces podríamos codificar a M de manera similar de la siguiente forma:

$$\begin{aligned} e &= \langle \langle 3, 2, 3, 5 \rangle, \langle 1, 1 \rangle, \langle 1, 3 \rangle, \langle 3, 1, 1, 1 \rangle \rangle \\ &= 2^{\langle 3, 2, 3, 5 \rangle + 1} 3^{\langle 1, 1 \rangle + 1} 5^{\langle 1, 3 \rangle + 1} 7^{\langle 3, 1, 1, 1 \rangle + 1} \\ &= 2^{2^4 3^3 5^4 7^6 + 1} 3^{2^2 3^2 + 1} 5^{2^2 3^4 + 1} 7^{2^4 3^2 5^2 7^2 + 1} \end{aligned}$$

Aunque este número es gigantescamente grande y computacionalmente en la práctica sería muy difícil de calcular, teóricamente es posible, pero el punto principal es que este número es único y con él podemos recuperar cuáles son las instrucciones exactas de M usando todas las herramientas que hemos desarrollado. Por ejemplo, con esta máquina se tendría lo siguiente:

$$\begin{aligned} e[0] &= \langle 3, 2, 3, 5 \rangle \\ e[1] &= \langle 1, 1 \rangle \\ e[2] &= \langle 1, 3 \rangle \\ e[3] &= \langle 3, 1, 1, 1 \rangle \end{aligned}$$

y de igual forma se tendría entonces:

$$\begin{array}{llll} e[0][0] = 3 & e[1][0] = 1 & e[2][0] = 1 & e[3][0] = 3 \\ e[0][1] = 2 & e[1][1] = 1 & e[2][1] = 3 & e[3][1] = 1 \\ e[0][2] = 3 & & & e[3][2] = 1 \\ e[0][3] = 5 & & & e[3][3] = 1 \end{array}$$

Luego, se recupera cada instrucción por medio del acceso a la lista. Sin embargo, notemos que $e[0]$ corresponde a la *primera* instrucción de la máquina URM. Para facilitar la lectura e identificación de los elementos de la lista y las instrucciones correspondientes que codifican, la codificación de la máquina M iniciará en 0 con el objetivo de que los accesos a la lista y

las instrucciones de M correspondan directamente. En el ejemplo que estábamos analizando se sigue que su código es el siguiente:

$$\langle 0, \langle 3, 2, 3, 5 \rangle, \langle 1, 1 \rangle, \langle 1, 3 \rangle, \langle 3, 1, 1, 1 \rangle \rangle$$

En este caso se obtiene ahora que $e[1] = \langle 3, 2, 3, 5 \rangle$ es el código de la primera instrucción $J(2, 3, 5)$, $e[2] = \langle 1, 1 \rangle$ el de la segunda instrucción $S(1)$, $e[3] = \langle 1, 3 \rangle$ el de la tercera instrucción $S(3)$ y $e[4] = \langle 3, 1, 1, 1 \rangle$ el de la cuarta instrucción $J(1, 1, 1)$. Se define entonces lo siguiente.

Definición 1.13 (Codificación de instrucciones y máquinas URM). Se definen los siguientes conjuntos de codificaciones de instrucciones y máquinas URM

$$\begin{aligned} \text{InstrCod} := & \{ \langle 0, n \rangle \in \omega : n \in \omega^+ \} \cup \{ \langle 1, n \rangle \in \omega : n \in \omega^+ \} \\ & \cup \{ \langle 2, n, m \rangle \in \omega : n, m \in \omega^+ \} \cup \{ \langle 3, n, m, k \rangle \in \omega : n, m, k \in \omega^+ \} \end{aligned}$$

$$\text{URMCod} := \{ \langle 0, x_1, \dots, x_n \rangle \in \omega : n \in \omega^+, x_1, \dots, x_n \in \text{InstrCod} \}$$

Teorema 1.21. Los conjuntos InstrCod y URMCod son primitivos recursivos.

Demostración. Para demostrar que InstrCod es primitivo-recursivo solo es necesario demostrar que cada uno de los conjuntos que lo componen son primitivos recursivos. Definamos los conjuntos $A_1 = \{ \langle 0, n \rangle \in \omega : n \in \omega^+ \}$, $A_2 = \{ \langle 1, n \rangle \in \omega : n \in \omega^+ \}$, $A_3 = \{ \langle 2, n, m \rangle \in \omega : n, m \in \omega^+ \}$ y $A_4 = \{ \langle 3, n, m, k \rangle \in \omega : n, m, k \in \omega^+ \}$. Notemos lo siguiente:

$$\begin{aligned} A_1(e) & \iff (\text{Seq}(e) \wedge \text{Len}(e) = 2 \wedge e[0] = 0 \wedge e[1] > 0) \\ A_2(e) & \iff (\text{Seq}(e) \wedge \text{Len}(e) = 2 \wedge e[0] = 1 \wedge e[1] > 0) \\ A_3(e) & \iff (\text{Seq}(e) \wedge \text{Len}(e) = 3 \wedge e[0] = 2 \wedge e[1] > 0 \wedge e[2] > 0) \\ A_4(e) & \iff (\text{Seq}(e) \wedge \text{Len}(e) = 4 \wedge e[0] = 3 \wedge e[1] > 0 \wedge e[2] > 0 \wedge e[3] > 0) \end{aligned}$$

Ya que Seq, Len y la función de acceso son primitivo-recursivas, por el teorema 1.6 se sigue que A_1 , A_2 , A_3 y A_4 son primitivo-recursivas, y luego $\text{InstrCod} = A_1 \cup A_2 \cup A_3 \cup A_4$ es primitivo-recursiva.

Para URMCod, un elemento está en él si y solo si es un código y cada elemento de la lista es elemento de InstrCod. Ya que Len es primitivo-recursiva, se tiene lo siguiente:

$$\begin{aligned} \text{URMCod}(e) & \iff (\text{Seq}(e) \wedge \text{Len}(e) > 1 \wedge e[0] = 0 \\ & \quad \wedge (\forall k < \text{Len}(e))(k > 0 \rightarrow \text{InstrCod}(e[k]))) \end{aligned}$$

Puesto que ya sabemos que InstrCod es primitivo-recursiva, entonces por los teoremas 1.6 y 1.8 se sigue que URMCod es primitivo-recursiva. $\square$

Demostramos ahora las siguientes funciones que nos serán de utilidad para codificar la ejecución de las máquinas URM.

Teorema 1.22. Las siguientes funciones son primitivo-recursivas:

1. La función *número de instrucciones* $\text{NumInstr} : \omega \rightarrow \omega$ que devuelve el número de instrucciones de una máquina URM.
2. La función *número de registro máximo* $\text{MaxReg} : \omega \rightarrow \omega$ que devuelve el registro más grande utilizado por la máquina URM.

Demostración. Para la función número de instrucciones, ya que el número de instrucciones es simplemente la longitud de la secuencia menos uno (ya que agregamos 0 al inicio de manera artificial), tenemos lo siguiente:

$$\text{NumInstr}(e) = \chi_{\text{URMCod}}(e)(\text{Len}(e) - 1)$$

Para la función del registro máximo, la estrategia es recorrer todos los elementos de la lista, que son elementos de InstrCod , y, dependiendo del tipo de instrucción, obtener el máximo de los registros de la instrucción actual y el registro máximo anterior. Podemos codificarlo de manera recursiva definiendo primero la función $g : \omega^2 \rightarrow \omega$ como sigue para cada $x, e \in \omega$:

$$g(e, x) = \begin{cases} \text{Max}(\langle x, e[1] \rangle), & e[0] \leq 1 \\ \text{Max}(\langle x, e[1], e[2] \rangle), & e[0] > 1 \end{cases}$$

la cual es primitivo-recursiva y encuentra el máximo de los registros actuales. Por lo tanto, podemos definir la función $f : \omega^2 \rightarrow \omega$ como sigue de manera recursiva para cada $e, n \in \omega$:

$$\begin{aligned} f(e, 0) &= 0 \\ f(e, n + 1) &= g(e[n + 1], f(e, n)) \end{aligned}$$

la cual es primitivo-recursiva y recupera el máximo de los registros de la instrucción actual y el máximo anterior. Luego se tiene que la función número máximo de registros es primitivo-recursiva ya que satisface que para cada $e \in \omega$:

$$\text{MaxReg}(e) = \chi_{\text{URMCod}}(e)f(e, \text{Len}(e) - 1)$$

□

El objetivo ahora es poder codificar la ejecución de una máquina URM de manera primitivo-recursiva. Para ello, primero debe codificarse el contenido de los registros en una iteración de la máquina. Para ello, recordemos que las ejecuciones de la máquina son una sucesión

de funciones $\{f_k\}_{k \in \omega}$ que representan el estado de la cinta en cada iteración $k \in \omega$ (sin considerar los valores $f_k(0)$). Por ejemplo, considerando de nuevo el ejemplo de la máquina $M = (J(2, 3, 5), S(1), S(3), J(1, 1, 1))$, tenemos que la sucesión de funciones que representan el estado de la cinta es la siguiente:

$$\begin{aligned} f_0 &: (3, 2, 0, 0, 0, 0, \dots) \\ f_1 &: (3, 2, 0, 0, 0, 0, \dots) \\ f_2 &: (4, 2, 0, 0, 0, 0, \dots) \\ f_3 &: (4, 2, 1, 0, 0, 0, \dots) \\ f_4 &: (4, 2, 1, 0, 0, 0, \dots) \\ f_5 &: (4, 2, 1, 0, 0, 0, \dots) \\ f_6 &: (5, 2, 1, 0, 0, 0, \dots) \\ f_7 &: (5, 2, 2, 0, 0, 0, \dots) \\ &\vdots \qquad \qquad \qquad \vdots \end{aligned}$$

Aunque el paso natural podría ser codificar cada función f_k como una lista, esto no es posible debido a que son infinitas. Por lo tanto, la idea es codificar únicamente la parte de la función que realmente se utiliza en la ejecución de la máquina URM.

Debido a que en la ejecución de una máquina URM solamente hacemos uso de un número finito de registros en cada iteración, los cuales pueden tomar valores diferentes de cero, podemos codificar únicamente esa parte de la función. Luego, hacemos uso de la función MaxReg , la cual por definición nos da esa cota del registro más grande que podemos utilizar.

Luego, podemos considerar únicamente listas de longitud mayor a $\text{MaxReg}(e)$ para codificar cada función f_k . La longitud se considera mayor a $\text{MaxReg}(e)$ para tener acceso a esos registros que necesita la máquina para ser ejecutada, además de considerar el caso cuando la entrada f_0 requiera una lista más larga que $\text{MaxReg}(e)$. En nuestro ejemplo, si $e = \langle 0, \langle 3, 2, 3, 5 \rangle, \langle 1, 1 \rangle, \langle 1, 3 \rangle, \langle 3, 1, 1, 1 \rangle \rangle$, entonces tenemos que $\text{MaxReg}(e) = 3$. Como la entrada consta de solo dos valores, podemos considerar una lista de longitud $5 > \text{MaxReg}(e)$. Entonces

cada una de las funciones f_k se codificaría de la siguiente manera:

k	$f_k(0)$	$f_k(1)$	$f_k(2)$	$f_k(3)$	$f_k(4)$
0	$\langle 1, 3, 2, 0, 0 \rangle$				
1	$\langle 2, 3, 2, 0, 0 \rangle$				
2	$\langle 3, 4, 2, 0, 0 \rangle$				
3	$\langle 4, 4, 2, 1, 0 \rangle$				
4	$\langle 1, 4, 2, 1, 0 \rangle$				
5	$\langle 2, 4, 2, 1, 0 \rangle$				
6	$\langle 3, 5, 2, 1, 0 \rangle$				
7	$\langle 4, 5, 2, 2, 0 \rangle$				
8	$\langle 5, 5, 2, 2, 0 \rangle$				

Estas codificaciones de las funciones f_k como listas finitas se denominan *configuraciones*. Notemos que las características principales de las configuraciones es que al menos tienen un elemento (ya que $\text{MaxReg}(e) > 0$) y que su primer elemento siempre es diferente de cero, ya que en esa posición se almacenan las instrucciones a ejecutar.

Definición 1.14 (Configuraciones). El conjunto de configuraciones es el conjunto:

$$\text{Config} := \{c \in \omega : \text{Seq}(c) \wedge \text{Len}(c) > 0 \wedge c[0] > 0\}$$

Lo que sigue es determinar cómo es que verificamos que el paso de una configuración a otra realmente está determinada por la máquina URM codificada. La idea es construir una relación primitivo-recursiva $\text{Computo} \subseteq \omega^3$ para la cual una terna $(e, c, d) \in \omega^3$ pertenece Computo si la configuración d surge de la configuración c cuando se ejecuta la máquina URM codificada por e , o, más formalmente, si existe $k \in \omega$ tal que c codifica a f_k y d codifica a f_{k+1} .

Primero debemos asegurar que los números c y d son configuraciones válidas. Lo anterior se traduce al hecho que $c, d \in \text{Config}$, $\text{Len}(c) = \text{Len}(d)$ y $\text{Len}(c), \text{Len}(d) > \text{MaxReg}(e)$. Después debemos traducir la definición 1.8 a este contexto para asegurar que c genera a d según las instrucciones de la máquina URM codificada por e .

Notación 1.5. Sean $f : \omega \rightarrow \omega$ y $n \in \omega$, entonces se denota:

$$f \upharpoonright n = \langle f(0), \dots, f(n-1) \rangle$$

Definición 1.15 (Cómputo). Definimos la relación de *cómputo* $\text{Computo} \subseteq \omega^3$ de modo que para cada terna $(e, c, d) \in \omega^3$ se satisfaga $\text{Computo}(e, c, d)$ si y solo si:

1. $\text{URMCod}(e)$,

2. $\text{Config}(c)$ y $\text{Config}(d)$,
3. $\text{Len}(c) = \text{Len}(d)$,
4. $\text{Len}(c) > \text{MaxReg}(e)$, y
5. dada la ejecución $\{f_k\}_{k \in \omega}$ de la máquina codificada por e , si existe $k \in \omega$ tal que $c = f_k \upharpoonright \text{Len}(c)$, entonces $d = f_{k+1} \upharpoonright \text{Len}(d)$

Teorema 1.23. La relación $\text{Computo} \subseteq \omega^3$ es primitivo-recursiva.

Demostración. Para demostrar que Computo es primitivo-recursiva, probaremos que esta se puede definir por una fórmula primitivo-recursiva.

Sea $(e, c, d) \in \omega^3$, entonces se debe satisfacer lo siguiente:

$$\text{URMCod}(e) \wedge \text{Config}(c) \wedge \text{Config}(d) \wedge \text{Len}(c) = \text{Len}(d) \wedge \text{Len}(c) > \text{MaxReg}(e)$$

Ahora, traduzcamos qué es lo que debe satisfacer la terna para cada tipo de instrucción. Recordemos que no tenemos nada que ejecutar si el valor de la ejecución en cero es mayor al número de instrucciones, y por tanto la cinta se queda estática en el tiempo. Puesto que c codifica a la cinta antes de la ejecución y d codifica el resultado de tal ejecución, entonces $d = c$ si $c[0] > \text{NumInstr}(e)$. Por lo tanto, debe satisfacerse lo siguiente:

$$c[0] > \text{NumInstr}(e) \rightarrow d = c$$

Consideremos entonces ahora los casos cuando $c[0] \leq \text{NumInstr}(e)$. Siempre tengamos en cuenta que $c[0]$ codifica el número de instrucción a ejecutar actualmente y $d[0]$ el número de instrucción a ejecutar en el siguiente paso. Para el caso de la instrucción cero $Z(n)$ para algún $n \leq \text{MaxReg}(e)$, recordemos que el valor de la cinta en el n -ésimo registro se actualiza a cero, la siguiente instrucción a ejecutar es la actual más uno, y los demás registros se mantienen invariantes. Que la instrucción actual a ejecutar sea $Z(n)$ significa que $e[c[0]][0] = 0$. El valor n es el valor $e[c[0]][1]$, por lo que asignar cero al n -ésimo registro es realizar $d[e[c[0]][1]] = 0$. Actualizar la instrucción a ejecutar como la siguiente es asignar $d[0] = c[0] + 1$. Por último, mantener invariantes los demás registros diferentes del n -ésimo es asignar $d[k] = c[k]$ para cada $k < \text{Len}(c)$ tal que $k > 0$ (pues el primer valor es la instrucción) y $k \neq n$. Todo lo anterior se condensa en la siguiente fórmula:

$$(c[0] \leq \text{NumInstr}(e) \wedge e[c[0]][0] = 0) \rightarrow (d[e[c[0]][1]] = 0 \wedge d[0] = c[0] + 1 \wedge (\forall k < \text{Len}(c))((k > 0 \wedge k \neq e[c[0]][1]) \rightarrow d[k] = c[k]))$$

Para el caso de la instrucción sucesor $S(n)$ se debe satisfacer que el valor del n -ésimo registro debe sumársele uno, el número de instrucción se actualiza a ser el siguiente al actual, y todo

valor de la cinta diferente al n -ésimo registro debe mantenerse invariante. Esto se condensa en la siguiente fórmula:

$$(c[0] \leq \text{NumInstr}(e) \wedge e[c[0]][0] = 1) \rightarrow (d[e[c[0]][1]] = c[e[c[0]][1]] + 1 \wedge d[0] = c[0] + 1 \\ \wedge (\forall k < \text{Len}(c))((k > 0 \wedge k \neq e[c[0]][1]) \rightarrow d[k] = c[k]))$$

En el caso de la instrucción transferencia $T(n, m)$ debemos asignar al m -ésimo registro el valor del n -ésimo registro, el valor de la instrucción se actualiza a ser la siguiente a la actual, y todo valor diferente al m -ésimo registro se mantiene invariante. Notemos que los valores n y m son los números $e[c[0]][1]$ y $e[c[0]][2]$ respectivamente, entonces todo lo anterior se condensa en la siguiente fórmula:

$$(c[0] \leq \text{NumInstr}(e) \wedge e[c[0]][0] = 2) \rightarrow (d[e[c[0]][2]] = c[e[c[0]][1]] \wedge d[0] = c[0] + 1 \\ \wedge (\forall k < \text{Len}(c))((k > 0 \wedge k \neq e[c[0]][2]) \rightarrow d[k] = c[k]))$$

Finalmente, para el caso de la instrucción salto $J(n, m, k)$, simplemente se verifica si los registros n -ésimo y m -ésimo son iguales; si lo son, entonces la instrucción se actualiza a ser la k -ésima, y si no, entonces se actualiza a ser la siguiente a la actual. Todos los demás valores de la cinta se mantienen invariantes. Notemos que el valor k corresponde al número $e[c[0]][3]$, entonces todo lo anterior se condensa en la siguiente fórmula:

$$(c[0] \leq \text{NumInstr}(e) \wedge e[c[0]][0] = 3) \rightarrow ((c[e[c[0]][1]] = c[e[c[0]][2]] \rightarrow d[0] = e[c[0]][3]) \\ \wedge (c[e[c[0]][1]] \neq c[e[c[0]][2]] \rightarrow d[0] = c[0] + 1) \\ \wedge (\forall k < \text{Len}(c))(k > 0 \rightarrow d[k] = c[k]))$$

Por lo tanto, una terna $(e, c, d) \in \omega^3$ satisface $\text{Computo}(e, c, d)$ si y solo si satisface la conjunción de todas las fórmulas anteriores, y luego es primitivo-recursiva. $\square$

Con esta relación ya tenemos todas las herramientas necesarias para codificar cuando una máquina URM se detiene. Consideremos una máquina URM codificada por $e \in \omega$, una entrada codificada por $z \in \omega$ y una lista de configuraciones codificada por $p \in \omega$. Recordemos que una máquina URM acaba cuando existe una ejecución que sobrepasa el número de instrucciones que se pueden ejecutar. El trabajo del número p es codificar esa situación. El valor p es una lista de configuraciones que representa esa sucesión de funciones que formalizan la ejecución de la máquina.

En primer lugar, como p codifica todas las configuraciones de la ejecución de la máquina, como mínimo debe contener a la configuración que codifica la cinta de entrada. Por lo tanto, p debe satisfacer que $\text{Len}(p) > 0$ y para cada $k < \text{Len}(p)$ se satisface que $\text{Config}(p[k])$.

En segundo lugar, la primera configuración corresponde a ingresar los datos de entrada a la cinta; la lista z . Para que tengamos espacio suficiente para registrar todos los datos de z en la

cinta inicial, la longitud de la cinta debe ser al menos la longitud de z (más uno para guardar la instrucción a ejecutar). Por lo tanto, debe satisfacerse que $\text{Len}(p[0]) > \text{Len}(z)$, para cada $k < \text{Len}(z)$ se cumple $p[0][k+1] = z[k]$ y para cada $k < \text{Len}(p[0])$, si $k > \text{Len}(z)$, entonces $p[0][k] = 0$. Además, como la primera instrucción a ejecutar siempre debe ser la primera, se debe cumplir también que $p[0][0] = 1$.

En tercer lugar, cada una de las configuraciones debe codificar una ejecución válida de la máquina codificada por e . Por lo tanto, debe satisfacerse para cada $k < \text{Len}(p) - 1$ que $\text{Computar}(e, p[k], p[k+1])$.

Finalmente, para que p codifique una ejecución que termina, recordemos que eventualmente una de las ejecuciones deba indicar que se realizará una instrucción inexistente. En este caso, la última configuración de p debe indicar que se ejecutará una instrucción cuya posición supera a $\text{NumInstr}(e)$, es decir, $p[\text{Len}(p) - 1][0] > \text{NumInstr}(e)$.

La relación que se define como la conjunción de todas estas condiciones que describen cuando una ejecución de una máquina URM termina se denomina T-predicado de Kleene, y será la pieza fundamental para demostrar que las funciones URM-computables son parcial-computables.

Definición 1.16 (T-predicado de Kleene). Se define el *T-predicado de Kleene* como la relación $T \subseteq \omega^3$ la cual satisface que una terna $(e, z, p) \in \omega^3$ pertenece a T si y solo si:

1. $e \in \text{URMCod}$, $z \in \text{Seq}$, $p \in \text{Seq}$,
2. $\text{Len}(p) > 0$, $p[0][0] = 1$,
3. para cada $k < \text{Len}(p)$ se satisface $\text{Config}(p[k])$,
4. $\text{Len}(p[0]) > \text{Len}(z)$,
5. para cada $k < \text{Len}(z)$ se satisface $p[0][k+1] = z[k]$,
6. para cada $k < \text{Len}(p[0])$, si $k > \text{Len}(z)$, entonces se satisface $p[0][k] = 0$,
7. para cada $k < \text{Len}(p) - 1$ se satisface $\text{Computar}(e, p[k], p[k+1])$, y
8. $p[\text{Len}(p) - 1][0] > \text{NumInstr}(e)$.

Directamente por definición se satisface lo siguiente.

Teorema 1.24. El T-predicado de Kleene es primitivo-recursivo.

Para finalizar, recordemos que dada una máquina URM, la función parcial que genera devuelve para cada entrada el valor del primer registro cuando si termina la ejecución. Considerando al valor p que codifica las configuraciones de la ejecución de la máquina, el resultado

del cómputo sería el valor $p[\text{Len}(p) - 1][1]$.

Definición 1.17 (Función U). Se define la función $U : \omega \rightarrow \omega$ tal que para cada $p \in \omega$, $U(p) = p[\text{Len}(p) - 1][1]$.

Notemos que se sigue lo siguiente por definición.

Teorema 1.25. La función U es primitivo-recursive.

Ahora, recordemos que el valor de la función URM-computable que genera una máquina URM no se calcula técnicamente con cualquier ejecución que indique que el cómputo ya terminó, sino con la mínima de ellas. Por lo tanto, el valor de tal función sería el valor de U de la mínima ejecución p para la cual la máquina e con entrada z termina, es decir, $U(\mu p T(e, z, p))$. A esta función que calcula de manera alternativa una función URM-computable se denomina *Forma Normal de Kleene*.

Teorema 1.26 (Teorema de la Forma Normal de Kleene). Sean M una máquina URM con codificación $e \in \text{URMCod}$ y $n \in \omega^+$, entonces para cada $x \in \omega^n$, con $z = \langle x \rangle$, se satisface lo siguiente:

$$\varphi_M^{(n)}(x) = U(\mu p T(e, z, p))$$

Demostración. Sean $n \in \omega^+$, $x \in \omega^n$ y $p \in \text{Seq}$ para los cuales solamente se satisfagan los primeros siete puntos de la definición 1.16, y sea $\{f_k\}_{k \in \omega}$ la ejecución de la máquina M con entrada x .

Por definición de p sabemos que es de la forma $p = \langle c_0, \dots, c_t \rangle$ de modo que $t = \text{Len}(p) - 1$ y para cada $k < \text{Len}(p)$, $\text{Config}(c_k)$. Además, notemos que para cada $k < \text{Len}(p) - 1$ se satisface que $\text{Computo}(e, c_k, c_{k+1})$, por lo que debido a la definición de Computo y de p sabemos que $\text{Len}(c_0) = \text{Len}(c_1) = \dots = \text{Len}(c_t) > \text{Len}(z)$, $\text{MaxReg}(e)$. Sea entonces $N := \text{Len}(c_0)$.

Por definición de p sabemos que:

$$c_0 = \langle 1, z[0], \dots, z[\text{Len}(z) - 1], \underbrace{0, \dots, 0}_{N - \text{Len}(z) - 1} \rangle$$

Por lo que se satisface que $f_0 \upharpoonright N = c_0$. Mostremos por inducción que para cada $k \leq t$ se satisface que $f_k \upharpoonright N = c_k$. El caso base ya lo mostramos, entonces sea $k < t$ tal que $f_k \upharpoonright N = c_k$. Por la definición 1.15, si $f_k \upharpoonright N = c_k$, entonces $f_{k+1} \upharpoonright N = c_{k+1}$.

Ahora, si p satisface el punto (8) de la definición 1.16, entonces se tiene que $c_t[0] > \text{NumInstr}(e)$, de lo cual se implica que $f_t(0) > \text{NumInstr}(e)$, es decir, $\varphi_M^{(n)}(x) \downarrow$. Por lo tanto, si $T(e, z, p)$, entonces $\varphi_M^{(n)}(x) \downarrow$.

Por otro lado, si $\varphi_M^{(n)}(x) \downarrow$, entonces existe $m \in \omega$ para el cual $f_m(0) > \text{NumInstr}(e)$. Si tomamos $N = \max\{\text{Len}(z), \text{MaxReg}(e)\} + 1$, entonces asignando $p = \langle f_0 \upharpoonright N, \dots, f_m \upharpoonright N \rangle$ se

satisface directamente que $T(e, z, p)$. Con esto hemos probado entonces que:

$$\exists p T(e, z, p) \iff \varphi_M^{(n)}(x) \downarrow$$

Si $p \in \text{Seq}$ es tal que $T(e, z, p)$, entonces notemos que por la definición de la función U se satisface que $U(p) = p[\text{Len}(p) - 1][1] = f_{\text{Len}(p)-1}(1) = \varphi_M^{(n)}(x)$; donde la última igualdad se sigue por el punto (3) de la definición 1.8. Por lo tanto:

$$\varphi_M^{(n)}(x) = U(\mu p T(e, z, p))$$

□

Este teorema es la pieza clave que nos faltaba para demostrar la equivalencia entre los enfoques, pues este nos da explícitamente la forma en la que se traduce una función URM-computable en una función parcial-recursiva.

■ **Corolario 1.27.** Toda función URM-computable es parcial-computable.

Demostración. Sean $n \in \omega^+$ y $f : \omega^n \rightarrow \omega^\uparrow$ una función URM-computable para alguna máquina URM M . Si $e \in \text{URMCod}$ es la codificación de M , entonces por el teorema de la forma normal de Kleene obtenemos que para cada $x \in \omega^n$:

$$f(x) = \varphi_M^{(n)}(x) = U(\mu p T(e, \langle x \rangle, p))$$

Puesto que el T-predicado de Kleene y la función U son primitivo-recursivas, entonces f es parcial-recursiva. □

■ **Corolario 1.28.** Una función es parcial-recursiva si y solo si es URM-computable, o equivalentemente, $\text{Parcial} = \text{URMComp}$.

Otro resultado interesante que se deriva del teorema de la forma normal de Kleene, y que se basa en nuestra interpretación de ver las funciones primitivo-recursivas como aquellos algoritmos que solamente utilizan ciclos *for* y las funciones parcial-recursivas que utilizan minimización no acotada como aquellos algoritmos que utilizan ciclos *while*, es el hecho que en la forma normal de Kleene de una función URM-computable solamente utilizamos una única vez el operador minimización, pues el T-predicado de Kleene y la función U son primitivo recursivas, lo cual puede interpretarse como que todo algoritmo puede reescribirse utilizando a lo más una vez un ciclo *while*.

■ **Corolario 1.29.** Todo algoritmo puede reescribirse para utilizar a lo más una vez un ciclo *while*.

1.3. La Máquina Universal, el problema de la detención y conjuntos c.e.

Ya que se demostró que ambos enfoques de la computabilidad son equivalentes, ya no es necesario hacer distinción cuando una función es URM-computable o cuando es parcial-recursiva. Esto no es un hecho aislado de estos dos modelos que hemos desarrollado para describir la computabilidad. Se ha demostrado que otros modelos diferentes de lo computable como las *máquinas de Turing* o el *cálculo lambda* de Church (y varios otros modelos) siempre son a lo más equivalentes a los modelos que desarrollamos en este texto.

Por esa razón es que la computabilidad no se basa específicamente en un modelo para formalizarla, sino que permite utilizar cualquier modelo que se nos acomode para cada problema específico. Pues todos los modelos que se han desarrollado modelan los mismos objetos, solo desde diferentes puntos de vista.

Debido a eso, de ahora en adelante en este texto, un objeto que pueda ser computado mediante cualquiera de los modelos que hemos trabajado le asignaremos la etiqueta de *computable*.

- Definición 1.18** (Computabilidad). 1. Sean $n \in \omega^+$ y $f : \omega^n \rightarrow \omega^\uparrow$, entonces f se denomina *computable* si es parcial-recursiva o URM-computable.
2. Sean $n \in \omega^+$ y $f : \omega^n \rightarrow \omega^\uparrow$, entonces f se denomina *total-computable* si es computable y $\text{dom}(f) = \omega^n$.
3. Sean $n \in \omega^+$ y $A \subseteq \omega^n$, entonces A se denomina *computable* si su función característica $\chi_A : \omega^n \rightarrow \omega^\uparrow$ es total-computable.
4. Sean $n \in \omega^+$ y $\varphi(x_1, \dots, x_n)$ una fórmula con variables libres $x_1, \dots, x_n$, entonces φ se denomina *computable* si el conjunto $\{(x_1, \dots, x_n) \in \omega^n : \varphi(x_1, \dots, x_n)\}$ es computable.

Una consecuencia importante del teorema de la forma normal de Kleene es la existencia de la llamada *máquina universal*. Se trata de una función computable que puede “simular” el comportamiento de cualquier otra función computable si le damos su código. Este resultado lo utilizamos en nuestra vida común con demasiada frecuencia, pues nuestras computadoras modernas (con el suficiente tiempo) pueden ejecutar cualquier programa que queramos si le facilitamos el código. Esto fue históricamente importante, pues significó la gran ventaja de no necesitar construir una computadora específica para cada algoritmo que quisieramos ejecutar, sino que era posible construir una única computadora que pudiera ejecutar todos los algoritmos que necesitáramos desde un mismo lugar.

Teorema 1.30 (Existencia de la Máquina Universal). Existe una máquina URM N la cual satisface lo siguiente para cada $e, z \in \omega$:

$$\varphi_N^{(2)}(e, z) = \begin{cases} \varphi_M^{(n)}(x), & e \text{ codifica a } M \in \text{URM} \text{ y } z = \langle x \rangle \text{ para algunos } n \in \omega^+ \text{ y } x \in \omega^n \\ 0, & \text{en otro caso} \end{cases}$$

Demostración. Definamos la siguiente función $\varphi : \omega^2 \rightarrow \omega^\dagger$ para cada $e, z \in \omega$:

$$\varphi(e, z) = \begin{cases} U(\mu p T(e, z, p)), & e \in \text{URMCod}, z \in \text{Seq}, \text{Len}(z) > 0 \\ 0, & \text{en otro caso} \end{cases}$$

Notemos que φ es parcial-recursive, y por ende URM-computable, por lo que existe $N \in \text{URM}$ tal que $\varphi = \varphi_N^{(2)}$. Entonces, por el teorema de la forma normal de Kleene se satisface que:

$$\begin{aligned} \varphi_N^{(2)}(e, z) &= \varphi(e, z) \\ &= \begin{cases} U(\mu p T(e, z, p)), & e \in \text{URMCod}, z \in \text{Seq}, \text{Len}(z) > 0 \\ 0, & \text{en otro caso} \end{cases} \\ &= \begin{cases} \varphi_M^{(n)}(x), & e \text{ codifica a } M \in \text{URM} \text{ y } z = \langle x \rangle \text{ para algunos } n \in \omega^+ \text{ y } x \in \omega^n \\ 0, & \text{en otro caso} \end{cases} \end{aligned}$$

□

Notación 1.6. Sea $N \in \text{URM}$ la máquina universal, entonces denotaremos por $\Phi_e(z)$ a $\varphi_N^{(2)}(e, z)$ para cada $e, z \in \omega$.

La existencia de la máquina universal nos permite desvincularnos de las máquinas URM cuando hablamos de códigos. Puesto que existe al menos una máquina universal, y esta puede simular cualquier otra máquina teniendo su codificación, mas no la máquina que codifica explícitamente, entonces los algoritmos se corresponden directamente con números naturales. Además, la codificación de algoritmos que construimos mediante máquinas URM no es la única que existe. Existen, por ejemplo, maneras de codificar algoritmos mediante funciones parcial-recursive, máquinas de Turing, o algún otro modelo de computabilidad, que nos permiten construir el T-predicado de Kleene también; y luego demostrar la existencia de la máquina universal. Más aún, todas estas codificaciones pueden corresponderse de manera biyectiva mediante funciones computables. Por lo tanto, de ahora en adelante los algoritmos se representarán únicamente por números naturales.

Ahora, debido a la definición de las máquinas URM (o de las funciones parcial-recursive) sabemos que el conjunto de funciones computables es numerable debido a la numeración de

estas mismas. Puesto que el conjunto de todas las posibles funciones parciales no es numerable ya que $(\omega^\dagger)^\omega \approx \aleph_1$, se sigue que existe una función parcial que no es computable. Aunque este argumento es suficiente para demostrar la existencia de funciones no computables, no nos exhibe un ejemplo explícito de una función que no lo sea.

De esta manera, para verificar la existencia de funciones no computables, abordaremos la idea de Turing sobre otro problema equivalente: la existencia de conjuntos no computables. Recordemos la interpretación de las funciones computables como algoritmos los cuales pueden ser ejecutados mediante operaciones básicas y ciclos *for* y *while*. Además, recordemos que trabajamos con funciones parciales debido al comportamiento indeseado de los ciclos *while*: la función puede nunca devolver un valor concreto, o la ejecución puede nunca terminar. Por lo tanto, una pregunta natural a realizarse es: ¿existe un proceso algorítmico que nos permita determinar si un algoritmo termina dada una entrada específica?

La pregunta anterior fue nombrada por Turing como el *problema de la detención*. Este problema puede enunciarse formalmente de la siguiente manera: ¿el conjunto $\{\langle e, z \rangle \in \omega : \Phi_e(z) \downarrow\}$ es computable? La respuesta a esta pregunta es negativa, y la demostración de ello usa como argumento principal la existencia de la máquina universal. Para poder demostrarlo, utilizaremos otro conjunto que será de ayuda, denominado *conjunto de detención*.

Definición 1.19 (Conjunto de detención). Se define el *conjunto de detención* como el conjunto:

$$K := \{e \in \omega : \Phi_e(e) \downarrow\}$$

Este conjunto en cambio colecciona a todos los algoritmos que terminan cuando se les da su propio código como entrada. Esa autorreferencia nos permitirá demostrar que no es computable usando un argumento de diagonalización.

Teorema 1.31 (Teorema de Turing). El conjunto de detención K no es computable.

Demostración. Probemos por contradicción. Supongamos que K es computable y definamos a la función $f : \omega \rightarrow \omega^\dagger$ tal que para cada $e \in \omega$:

$$f(e) = \begin{cases} \Phi_e(e) + 1, & e \in K \\ 0, & e \notin K \end{cases}$$

Puesto que por hipótesis K es computable, se sigue que f es una función computable. Por lo tanto, existe $z \in \omega$ tal que para cada $x \in \omega$, $f(x) = \Phi_z(x)$. Supongamos que $z \in K$, entonces se satisface lo siguiente:

$$\Phi_z(z) = f(z) = \Phi_z(z) + 1$$

Lo cual es una contradicción. Por lo que $z \notin K$, pero eso implica que:

$$0 = f(z) = \Phi_z(z) \uparrow$$

Lo cual también es una contradicción. Las contradicciones surgen de la suposición que K es computable. Por lo tanto, K no es un conjunto computable. $\square$

Dentro de esta demostración se utiliza la función auxiliar $f : \omega \rightarrow \omega^\uparrow$ tal que para cada $e \in \omega$:

$$f(e) = \begin{cases} \Phi_e(e) + 1, & e \in K \\ 0, & e \notin K \end{cases}$$

Con ayuda de este conjunto se puede demostrar que el problema de la detención tiene solución negativa.

| Corolario 1.32. El conjunto $K' := \{\langle e, z \rangle \in \omega : \Phi_e(z) \downarrow\}$ no es computable.

Demostración. Notemos que para cada $e \in \omega$ se satisface que $\langle e, e \rangle \in K'$ si y solo si $e \in K$. Se sigue entonces que $\chi_K(e) = \chi_{K'}(\langle e, e \rangle)$. Si K' fuera computable, entonces $\chi_{K'}$ sería computable, y también χ_K por consiguiente. Puesto que K no es computable por el teorema de Turing, se sigue que K' no es computable. $\square$

Capítulo 2

Aritmética de segundo orden

2.1. Sintaxis

El lenguaje de la aritmética de segundo orden será el marco donde podremos desarrollar toda la teoría de las Matemáticas en Reversa. Cuando hablamos de segundo orden, no nos referimos a lo que comúnmente se refiere a la cuantificación sobre funciones o relaciones (o al menos no directamente); sino más bien al hecho de que este lenguaje es un lenguaje de dos *tipos*. Con *tipos* (o *sorts* en inglés) nos referimos a que los elementos que podemos estudiar dentro del lenguaje son de dos clases de elementos diferentes: números y conjuntos. El tener dos clasificaciones diferentes de elementos nos permite hablar más fácil y elegantemente sobre los objetos que vamos a manipular.

Utilizar, por ejemplo, el lenguaje estándar de la Teoría de Números (L_1) no nos da suficiente riqueza o fuerza para poder hablar de manera directa sobre subconjuntos de números naturales, sino solamente de las propiedades de los números tal cual. Si en cambio utilizáramos el lenguaje de la Teoría de Conjuntos (ZF), este sería un sistema mucho más fuerte que generaría un problema, y es que no podríamos distinguir tan fácilmente cuándo hablamos de números y cuándo de conjuntos de números, pues en este lenguaje ambas serían indistinguibles hasta cierto punto.

Por esos motivos es que el lenguaje de la aritmética de segundo orden es la que mejor se ajusta a nuestros objetivos sobre hablar de manera independiente de números y conjuntos de números sin ser tan débil como el lenguaje de la Teoría de Números, ni tan fuerte como el de la Teoría de Conjuntos.

La característica principal de este lenguaje es el hecho que se trata de un lenguaje de *dos tipos*. Esto significa que hacemos una distinción explícita en el lenguaje de los dos tipos de elementos con los cuales trabajaremos: tipo número y tipo conjunto. Un elemento de tipo número es un elemento que se interpreta como un número natural, mientras que un elemento

de tipo conjunto es un elemento que se interpreta como un subconjunto de números naturales.

Es necesario recalcar que los elementos de tipo conjunto, al interpretarse, solamente pueden contener a elementos de tipo número. Esto es importante debido a que esta es una restricción que ya no nos permite expresar conjuntos que contienen a otros conjuntos. Formalmente no podríamos, por ejemplo, expresar lo que es un conjunto cociente de una clase de equivalencia sobre los números naturales, o el conjunto potencia de otro conjunto.

Definición 2.1 (Lenguaje de la aritmética de segundo orden). El lenguaje de la aritmética de segundo orden es el lenguaje de dos tipos $L_2 = (0, 1, +, \cdot, =, <, \in, V_N, V_C, \text{Log})$ donde:

1. los símbolos 0 y 1 son de constante (de tipo número).
2. los símbolos $+$ y $\cdot$ son de función binaria (de tipo número).
3. los símbolos $=$ y $<$ son de relación binaria (de tipo número).
4. el símbolo $\in$ es de relación binaria (relaciona tipo número con tipo conjunto).
5. los conjuntos V_N y V_C son infinitos, no vacíos y disjuntos denominados *conjuntos de variables de tipo número* y *de tipo conjunto* respectivamente.
6. el conjunto $\text{Log} = \{\neg, \wedge, \vee, \exists, \}, \{\}$ se denomina *conjunto de símbolos lógicos*.

Definimos al conjunto total de variables como $V := V_N \cup V_C$.

Notación 2.1. Denotaremos a las variables de tipo número (elementos de V_N) mediante letras minúsculas $(n, m, p, \dots)$, y a las variables de tipo conjunto (elementos de V_C) mediante letras mayúsculas $(X, Y, Z, \dots)$. En ambos casos, pueden utilizarse subíndices si es necesario (por ejemplo, n_1, n_2, X_1, X_2).

Dentro de este lenguaje es que podemos definir los *términos*, los cuales son palabras que se interpretarán como *elementos de los números naturales*, o simplemente *números*. La construcción de los términos se basa en el hecho de que podemos generar nuevos números utilizando como base los números que ya conocemos (0 y 1), además de variables de tipo número, con las cuales podremos expresar propiedades generales de los números más adelante.

Definición 2.2 (Términos). Se define el conjunto de *términos* de L_2 como el mínimo conjunto $\text{Term} \subseteq L_2^*$ que satisface los siguientes puntos:

1. Las palabras 0 y 1 pertenecen a Term .

2. $V_N \subseteq \text{Term}$

3. para cada t_1 y t_2 en Term , se satisface que las palabras $(t_1 + t_2)$ y $(t_1 \cdot t_2)$ pertenecen a Term .

Ejemplos de términos de L_2 son los siguientes:

$$\begin{array}{cccc} 1 & 0 & n & m_2 \\ (1 + 0) & (n \cdot m) & (1 \cdot (n_1 + m_2)) & ((1 + n) \cdot (0 \cdot m_1)) \end{array}$$

Las *fórmulas atómicas* son aquellas palabras las cuales hablan sobre las relaciones directas entre los términos. Estas son las que pueden expresar las propiedades más básicas sobre los elementos de los naturales, como lo es si dos términos o elementos son iguales o uno menor que otro; y debido a que podemos hablar de objetos de tipo conjunto, también sobre si un término pertenece o no a cierto subconjunto de los naturales.

Estas fórmulas son con las que podemos hablar, por ejemplo, de ecuaciones e inecuaciones.

Definición 2.3 (Fórmulas atómicas). Se define el conjunto de fórmulas atómicas de L_2 como el conjunto $\text{Atom} \subseteq L_2^*$ que satisface los siguientes puntos:

1. para cada t_1 y t_2 en Term se cumple que las palabras $(t_1 = t_2)$ y $(t_1 < t_2)$ pertenecen a Atom
2. para cada t en Term y cada X en V_C , la palabra $(t \in X)$ pertenece a Atom .

Ejemplos de fórmulas atómicas de L_2 son las siguientes:

$$\begin{array}{cccc} (1 = 0) & (n = 1) & (0 = m_1) & (n_1 = m_2) \\ (((n + 1) = (n + n))) & ((n \cdot m) + m < (n \cdot (n + 1))) & ((n + n) \in X) & (((n \cdot n) + 1) \in Y) \end{array}$$

Ahora, las *fórmulas* son las afirmaciones o proposiciones que nos permiten la mayor expresividad. Con estas ahora podemos hablar sobre las relaciones entre las fórmulas atómicas, e incluso con otras fórmulas más complejas. Lo importante sobre estas fórmulas es que además de tener conectores lógicos, también disponemos del uso de los cuantificadores. Con los cuantificadores ahora podemos hablar sobre propiedades generales de los números y subconjuntos de números naturales.

Definición 2.4 (Fórmulas). Se define el conjunto de fórmulas de L_2 como el mínimo conjunto $\text{Form} \subseteq L_2^*$ que satisface los siguientes puntos:

1. $\text{Atom} \subseteq \text{Form}$
2. para cada $\varphi \in \text{Form}$ se satisface que $\neg\varphi \in \text{Form}$.

3. para cada φ_1 y φ_2 en Form se satisface que la palabra $(\varphi_1 \wedge \varphi_2)$ pertenece a Form.
4. para cada $x \in V$ y cada $\varphi \in \text{Form}$ se satisface que las palabras $\forall x\varphi$ y $\exists x\varphi$ pertenecen a Form.

Notación 2.2. Para cada φ_1 y φ_2 en Form se realizan las siguientes abreviaciones:

1. se escribe $(\varphi_1 \vee \varphi_2)$ en lugar de $\neg(\neg\varphi_1 \wedge \neg\varphi_2)$.
2. se escribe $(\varphi_1 \rightarrow \varphi_2)$ en lugar de $(\neg\varphi_1 \vee \varphi_2)$.
3. se escribe $(\varphi_1 \leftrightarrow \varphi_2)$ en lugar de $((\varphi_1 \rightarrow \varphi_2) \wedge (\varphi_2 \rightarrow \varphi_1))$.

Ejemplos de fórmulas de L_2 son las siguientes:

$$\begin{aligned} & ((0 < 1) \wedge (1 = (0 + 1))) & \neg((n + 1) < 0) \\ & ((0 < n) \rightarrow \exists m((m + 1) = n)) & \forall X \exists n((n \cdot n) \in X) \end{aligned}$$

Definición 2.5 (Conjunto de variables). Sea $\varphi \in \text{Term} \cup \text{Form}$. Se define el *conjunto de variables* de φ de la siguiente forma inductiva:

1. Si $\varphi \in \{0, 1\}$, entonces $\text{Var}[\varphi] = \emptyset$.
2. Si $\varphi \in V_N$, entonces $\text{Var}[\varphi] = \{\varphi\}$.
3. Si φ está dada por $(t_1 \alpha t_2)$ con $\alpha \in \{+, \cdot, <, =\}$ y $t_1, t_2 \in \text{Term}$, entonces $\text{Var}[\varphi] = \text{Var}[t_1] \cup \text{Var}[t_2]$.
4. Si φ está dada por $(t \in X)$ con $X \in V_C$ y $t \in \text{Term}$, entonces $\text{Var}[\varphi] = \text{Var}[t] \cup \{X\}$.
5. Si φ está dada por $\neg\psi$ con $\psi \in \text{Form}$, entonces $\text{Var}[\varphi] = \text{Var}[\psi]$.
6. Si φ está dada por $(\psi_1 \wedge \psi_2)$ con $\psi_1, \psi_2 \in \text{Form}$, entonces $\text{Var}[\varphi] = \text{Var}[\psi_1] \cup \text{Var}[\psi_2]$.
7. Si φ está dada por $Qx\psi$ con $Q \in \{\exists, \forall\}$, $x \in V$ y $\psi \in \text{Form}$, entonces $\text{Var}[\varphi] = \text{Var}[\psi] \cup \{x\}$.

Definición 2.6 (Conjunto de variables libres). Sea $\varphi \in \text{Form}$. Se define el *conjunto de variables libres* de φ de la siguiente forma inductiva:

1. Si $\varphi \in \text{Atom}$, entonces $\text{Free}[\varphi] = \text{Var}[\varphi]$.

2. Si φ está dada por $\neg\psi$ con $\psi \in \text{Form}$, entonces $\text{Free}[\varphi] = \text{Free}[\psi]$.
3. Si φ está dada por $(\psi_1 \wedge \psi_2)$ con $\psi_1, \psi_2 \in \text{Form}$, entonces $\text{Free}[\varphi] = \text{Free}[\psi_1] \cup \text{Free}[\psi_2]$.
4. Si φ está dada por $Qx\psi$ con $Q \in \{\exists, \forall\}$, $x \in V$ y $\psi \in \text{Form}$, entonces $\text{Free}[\varphi] = \text{Free}[\psi] \setminus \{x\}$.

Definición 2.7 (Fórmulas cerradas). Se define el conjunto de *fórmulas cerradas* de L_2 como sigue:

$$\text{Sent} := \{\varphi \in \text{Form} : \text{Free}[\varphi] = \emptyset\}$$

2.2. Semántica

Definición 2.8 (Estructuras). Una *estructura* de L_2 , o L_2 -*estructura*, es una tupla $\mathcal{M} = (M_N, M_C, 0^{\mathcal{M}}, 1^{\mathcal{M}}, +^{\mathcal{M}}, \cdot^{\mathcal{M}}, =^{\mathcal{M}}, <^{\mathcal{M}}, \in^{\mathcal{M}})$ donde:

1. M_N y M_C son conjuntos no vacíos, denominados *conjuntos de tipo número* y *de tipo conjunto* respectivamente.
2. $0^{\mathcal{M}}, 1^{\mathcal{M}} \in M_N$
3. $+^{\mathcal{M}}, \cdot^{\mathcal{M}} : M_N^2 \rightarrow M_N$
4. $<^{\mathcal{M}}, =^{\mathcal{M}} \subseteq M_N^2$ donde $=^{\mathcal{M}}$ es la relación identidad.
5. $\in^{\mathcal{M}} \subseteq M_N \times M_C$

Notación 2.3. Para simplificar la notación, una L_2 -estructura se denota por una letra mayúscula en estilo caligráfico, su conjunto de tipo número se denota por esa letra en estilo plano con subíndice N , similarmente su conjunto de tipo conjunto se denota con subíndice C , y la misma letra sin subíndices es la unión de ambos.

Por ejemplo, si $\mathcal{A}$ es una L_2 -estructura, entonces A_N es su conjunto de tipo número, A_C es su conjunto de tipo conjunto y $A = A_N \cup A_C$.

Definición 2.9 (ω -estructura). Sea $\mathcal{M}$ una L_2 -estructura. $\mathcal{M}$ se denomina ω -*estructura* si $M_N = \omega$, $M_C \subseteq 2^\omega$, y $+^{\mathcal{M}}, \cdot^{\mathcal{M}}$ y $<^{\mathcal{M}}$ se interpretan de la forma estándar en ω , y $\in^{\mathcal{M}}$ se interpreta de la forma estándar de la teoría de conjuntos.

Identificaremos una ω -estructura por su conjunto de tipo conjunto, por lo que para cada $S \subseteq 2^\omega$, al decir que S es una ω -estructura nos referiremos en realidad a la única ω -estructura $\mathcal{M}$ tal que $M_C = S$.

Falta de con-
do

Falta agregar conceptos de sintaxis y semántica, como valuaciones, interpretación, implicación lógica, equivalencia lógica y deducción lógica

Definición 2.10 (Valuación de variables). Sea $\mathcal{M}$ una L_2 -estructura. Una *valuación de variables* de L_2 es una función $s : V \rightarrow M$ tal que para cada $n \in V_N$ se tiene que $s[n] \in M_N$ y para cada $X \in V_C$ se tiene que $s[X] \in M_C$.

Definición 2.11 (Interpretación de términos). Sean $\mathcal{M}$ una L_2 -estructura y $s : V \rightarrow M$ una valuación de variables. La *interpretación* de Term en $\mathcal{M}$ dada s es la función $s^* : \text{Term} \rightarrow M$ definida como sigue:

1. Para cada $c \in \{0, 1\}$, $s^*[c] = c^{\mathcal{M}}$.
2. Para cada $n \in V_N$, $s^*[n] = s[n]$.
3. Para cada $t_1, t_2 \in \text{Term}$ se tiene que $s^*[(t_1 + t_2)] = +^{\mathcal{M}}(s^*[t_1], s^*[t_2])$ y $s^*[(t_1 \cdot t_2)] = \cdot^{\mathcal{M}}(s^*[t_1], s^*[t_2])$

Notación 2.4. Sean $A_1, \dots, A_n$ conjuntos no vacíos con $n \in \omega$ y $R \subseteq \prod_{k=1}^n A_k$ una relación. Para cada $(x_1, \dots, x_n) \in \prod_{k=1}^n A_k$, escribiremos $R(x_1, \dots, x_n)$ en lugar de $(x_1, \dots, x_n) \in R$.

Notación 2.5. Sean A, B conjuntos no vacíos, $f : A \rightarrow B$, $a \in A$ y $b \in B$. denotamos por $f[a \rightsquigarrow b]$ a la función $g : A \rightarrow B$ dada como sigue:

$$g(x) = \begin{cases} f(x), & x \neq a \\ b, & x = a \end{cases}$$

Definición 2.12 (Interpretación de fórmulas). Sean $\mathcal{M}$ una L_2 -estructura y $s : V \rightarrow M$ una valuación de variables. Para cada $\varphi \in \text{Form}$, decimos que φ es *verdadera* en $\mathcal{M}$ (o que $\mathcal{M}$ *modela* a φ) con la valuación s , denotado por $(\mathcal{M}, s) \models \varphi$, si se satisface una de las siguientes condiciones:

1. Si φ está dada por $(t_1 R t_2)$ con $t_1, t_2 \in \text{Term}$ y $R \in \{<, =\}$, entonces $(\mathcal{M}, s) \models \varphi$ si y solo si $R^{\mathcal{M}}(s^*[t_1], s^*[t_2])$.
2. Si φ está dada por $(t \in X)$ con $t \in \text{Term}$ y $X \in V_C$, entonces $(\mathcal{M}, s) \models \varphi$ si y solo si

$$\in^{\mathcal{M}} (s^*[t], s[X]).$$

3. Si φ está dada por $\neg\psi$ con $\psi \in \text{Form}$, entonces $(\mathcal{M}, s) \models \varphi$ si y solo si no es cierto que $(\mathcal{M}, s) \models \psi$.
4. Si φ está dada por $(\psi_1 \wedge \psi_2)$ con $\psi_1, \psi_2 \in \text{Form}$, entonces $(\mathcal{M}, s) \models \varphi$ si y solo si $(\mathcal{M}, s) \models \psi_1$ y $(\mathcal{M}, s) \models \psi_2$.
5. Si φ está dada por $\exists x\psi$ con $x \in V$ y $\psi \in \text{Form}$, entonces $(\mathcal{M}, s) \models \varphi$ si y solo si para algún elemento $a \in M$ ($a \in M_N$ si $x \in V_N$, y $a \in M_C$ si $x \in V_C$) se satisface que $(\mathcal{M}, s[x \rightsquigarrow a]) \models \psi$.
6. Si φ está dada por $\forall x\psi$ con $x \in V$ y $\psi \in \text{Form}$, entonces $(\mathcal{M}, s) \models \varphi$ si y solo si para cualquier elemento $a \in M_N$ si $x \in V_N$ y $a \in M_C$ si $x \in V_C$, se satisface que $(\mathcal{M}, s[x \rightsquigarrow a]) \models \psi$.

Notación 2.6. Sea $\varphi \in \text{Form}$.

1. Si $\mathcal{M}$ es una L_2 -estructura, se denotará $\mathcal{M} \models \varphi$ si $(\mathcal{M}, s) \models \varphi$ para cualquier valuación de variables s .
2. Se denotará $\models \varphi$ si $(\mathcal{M}, s) \models \varphi$ para cualquier L_2 -estructura $\mathcal{M}$ y cualquier valuación de variables s .

Definición 2.13 (Definibilidad). Sean $\mathcal{M}$ una L_2 -estructura, $X \in M_C$ y $x_1, \dots, x_m \in M$ para algún $m \in \omega$. X se dice *definible* en L_2 con parámetros $x_1, \dots, x_m$ si existe $\varphi(n, \bar{x}_1, \dots, \bar{x}_m) \in \text{Form}$ con $n \in \text{Free}[\varphi]$ tal que para cada valuación de variables s :

$$X = \{k \in M_N : (\mathcal{M}, s_0[n \rightsquigarrow k]) \models \varphi(n, \bar{x}_1, \dots, \bar{x}_m)\}$$

donde $\bar{x}_1, \dots, \bar{x}_m \in V$ y $s_0 : V \rightarrow M$ es tal que $s_0 = s[\bar{x}_1 \rightsquigarrow x_1, \dots, \bar{x}_m \rightsquigarrow x_m]$.

Definición 2.14 (Equivalencia lógica). Sean $\varphi, \psi \in \text{Form}$, entonces φ y ψ se denominan *lógicamente equivalentes* si $\models (\varphi \leftrightarrow \psi)$.

En tal caso se denota por $\varphi \equiv \psi$.

Definición 2.15 (Implicación semántica). Sean $\Gamma \subseteq \text{Form}$ y $\varphi \in \text{Form}$. Decimos que φ se *implica semánticamente* de Γ si para cada L_2 -estructura $\mathcal{M}$ y valuación de variables s tales que $(\mathcal{M}, s) \models \psi$ para cada $\psi \in \Gamma$ se sigue que $(\mathcal{M}, s) \models \varphi$.

En tal caso se denotará por $\Gamma \models \varphi$.

2.3. Resultados y definiciones esenciales

Definición 2.16 (Cuantificadores acotados). Una *fórmula acotadamente cuantificada* es toda aquella de la forma $\exists n ((n < t) \wedge \varphi)$ o $\forall n ((n < t) \rightarrow \varphi)$ donde $\varphi \in \text{Form}$, $n \in V_N$, $t \in \text{Term}$ y $n \notin \text{Var}[t]$.

Notación 2.7. Sean $\varphi \in \text{Form}$, $n \in V_N$ y $t \in \text{Term}$ tales que $n \notin \text{Var}[t]$. Entonces:

1. escribiremos $(\exists n < t) \varphi$ en lugar de $\exists n ((n < t) \wedge \varphi)$.
2. escribiremos $(\forall n < t) \varphi$ en lugar de $\forall n ((n < t) \rightarrow \varphi)$.

Definición 2.17 (Jerarquía aritmética). Se define al conjunto de fórmulas $\Delta_0^0 \subseteq \text{Form}$ como el mínimo conjunto que satisface lo siguiente:

1. $\text{Atom} \subseteq \Delta_0^0$
2. para cada $\varphi \in \Delta_0^0$ se satisface que $\neg \varphi \in \Delta_0^0$.
3. para cada $\varphi_1, \varphi_2 \in \Delta_0^0$ se satisface que $(\varphi_1 \wedge \varphi_2) \in \Delta_0^0$.
4. para cada $\varphi \in \Delta_0^0$, $n \in V_N$ y $t \in \text{Term}$ tales que $n \notin \text{Var}[t]$ se satisface que $(\exists n < t) \varphi, (\forall n < t) \varphi \in \Delta_0^0$.

Para cada $k \in \omega^+$ se definen los siguientes conjuntos:

1. $\Sigma_1^0 \subseteq \text{Form}$ es el conjunto de todas las fórmulas $\varphi \in \text{Form}$ tales que $\varphi \equiv \exists n \psi$ para algunos $n \in V_N$ y $\psi \in \Delta_0^0$.
2. $\Pi_1^0 \subseteq \text{Form}$ es el conjunto de todas las fórmulas $\varphi \in \text{Form}$ tales que $\varphi \equiv \forall n \psi$ para algunos $n \in V_N$ y $\psi \in \Delta_0^0$.
3. $\Sigma_{k+1}^0 \subseteq \text{Form}$ es el conjunto de todas las fórmulas $\varphi \in \text{Form}$ tales que $\varphi \equiv \exists n \psi$ para algunos $n \in V_N$ y $\psi \in \Pi_k^0$.
4. $\Pi_{k+1}^0 \subseteq \text{Form}$ es el conjunto de todas las fórmulas $\varphi \in \text{Form}$ tales que $\varphi \equiv \forall n \psi$ para algunos $n \in V_N$ y $\psi \in \Sigma_k^0$.

Teorema 2.1. Se satisface que $\Delta_0^0 \subseteq \Sigma_1^0$ y $\Delta_0^0 \subseteq \Pi_1^0$.

Demostración. Consideremos una fórmula $\varphi \in \Delta_0^0$ y $n \in V_N$ con $n \notin \text{Var}[\varphi]$. Mostremos que $\varphi \in \Sigma_1^0$ y $\varphi \in \Pi_1^0$.

Consideremos a la fórmula ψ dada por $\exists n ((n = n) \wedge \varphi)$. Notemos que $((n = n) \wedge \varphi) \in \Delta_0^0$ ya que $(n = n) \in \text{Atom} \subseteq \Delta_0^0$ y, por hipótesis, $\varphi \in \Delta_0^0$.

Obtenemos lo siguiente:

$$\psi \equiv \exists n ((n = n) \wedge \varphi) \equiv \exists n (\top \wedge \varphi) \equiv \exists n \varphi \equiv \varphi$$

Por lo que $\varphi \equiv \psi$, y entonces $\varphi \in \Sigma_1^0$.

Ahora, para el caso de Π_1^0 , si θ está dada por $\forall n ((n = n) \rightarrow \varphi)$, similarmente se tiene que $((n = n) \rightarrow \varphi) \in \Delta_0^0$. Luego:

$$\theta \equiv \forall n ((n = n) \rightarrow \varphi) \equiv \forall n (\top \rightarrow \varphi) \equiv \forall n \varphi \equiv \varphi$$

Por lo que $\varphi \equiv \theta$, y en conclusión $\varphi \in \Pi_1^0$. □

Lema 2.2. Sean $S \subseteq 2^\omega$ una ω -estructura, $Y \in S$ y $X \in \omega^m$ para algún $m \in \omega$. Si X es definible por una fórmula Δ_0^0 con parámetro Y , entonces $X \leq_T Y$.

Demostración. Sea $\varphi(n_1, \dots, n_m) \in \Delta_0^0$ la fórmula con parámetro Y y $n_1, \dots, n_m \in V_N$ tal que $X = \{(n_1, \dots, n_m) : \varphi(n_1, \dots, n_m)\}$. Probemos por inducción sobre el conjunto Δ_0^0 .

- (Base) Si $\varphi \in \text{Atom}$, entonces tenemos dos casos:
 - Si φ está dada por $(t_1 = t_2)$ o $(t_1 < t_2)$ para algunos $t_1, t_2 \in \text{Term}$, ya que la suma, producto, la relación “menor que” y la relación identidad son computables en ω , se sigue que X es computable. Ya que $X \leq_T \emptyset$ y $\emptyset \leq_T Y$, entonces $X \leq_T Y$.
 - Si φ está dada por $(t \in Y)$ para algún $t \in \text{Term}$, ya que la suma y producto son computables en ω , entonces se tiene claramente que $X \leq_T Y$.
- (Paso inductivo) Supongamos que $\psi(n_1, \dots, n_m), \theta(n_1, \dots, n_m) \in \Delta_0^0$ fórmulas con parámetro Y y $n_1, \dots, n_m \in V_N$ definen conjuntos $U, W \in 2^{\omega^m}$ respectivamente tales que $U, W \leq_T Y$. Entonces tenemos tres casos:

- Supongamos que φ está dada por $\neg\psi$. Ya que $U \leq_T Y$, entonces por propiedades de la computabilidad relativa, se tiene que $\omega^m \setminus U \leq_T Y$. Puesto que:

$$X = \{(n_1, \dots, n_m) : \varphi(n_1, \dots, n_m)\} = \{(n_1, \dots, n_m) : \neg\psi(n_1, \dots, n_m)\} = \omega^m \setminus U$$

se sigue que $X \leq_T Y$.

- Supongamos que φ está dada por $(\psi \wedge \theta)$. Ya que $U, W \leq_T Y$, entonces $U \cap W \leq_T Y$. Puesto que:

$$\begin{aligned} X &= \{(n_1, \dots, n_m) : \varphi(n_1, \dots, n_m)\} \\ &= \{(n_1, \dots, n_m) : (\psi(n_1, \dots, n_m) \wedge \theta(n_1, \dots, n_m))\} = U \cap W \end{aligned}$$

se sigue entonces que $X \leq_T Y$.

- Supongamos que φ está dada por $(\exists n < t) \psi$ o $(\forall n < t) \psi$ para algunos $n \in V_N$ y $t \in \text{Term}$ con $n \notin \text{Var}[t]$. Sabemos para cualquier caso que como la suma y producto son computables, las búsquedas acotadas también son computables en Y , por lo que $X \leq_T Y$.

Por lo tanto, podemos afirmar que $X \leq_T Y$. □

Teorema 2.3 (Post). Sean S una ω -estructura y $X, Y \in S$.

1. X es c.e. en Y si y solo si X es definible por una fórmula Σ_1^0 con parámetro Y .
2. X es co-c.e. en Y si y solo si X es definible por una fórmula Π_1^0 con parámetro Y .
3. X es computable en Y si y solo si X es definible por una fórmula Σ_1^0 y por una fórmula Π_1^0 , ambos con parámetro Y .

Demostración. 1. Demostremos la suficiencia. Sea $\varphi(n) \in \Sigma_1^0$ con parámetro Y tal que $X = \{n : \varphi(n)\}$, y mostremos que X es c.e. en Y .

Como $\varphi \in \Sigma_1^0$, existen $m \in V_N$ y $\psi(n, m) \in \Delta_0^0$ tales que $\varphi(n) \equiv \exists m \psi(n, m)$. Sea $Z = \{(n, m) : \psi(n, m)\}$. Como $\psi \in \Delta_0^0$, por el lema 2.2 se tiene que Z es Y -computable. Definamos entonces la función Y -computable $f(k) := \mu m Z(k, m)$, de lo cual se tiene que para cada $k \in \omega$:

$$\begin{aligned} k \in X &\iff \varphi(k) \iff \exists m \psi(k, m) \iff \exists m Z(k, m) \\ &\iff \mu m Z(k, m) \downarrow \iff f(k) \downarrow \iff k \in \text{dom}(f) \end{aligned}$$

Por lo que X es el dominio de una función Y -computable, de lo cual se tiene que X es c.e. en Y .

Ahora, para la necesidad supongamos que X es c.e. en Y , entonces existe $e \in \omega$ tal que $X = W_e^Y$. Sabemos por otra parte que el T-predicado de Kleene está definido como una fórmula $T \in \Delta_0^0$ con parámetro Y de modo que para cada $k \in \omega$:

$$k \in X \iff k \in W_e^Y \iff \mu p T(e, k, Y, p) \downarrow \iff \exists p T(e, k, Y, p)$$

Luego, si φ está dada por $\exists pT(e, k, Y, p)$, se sigue que $\varphi \in \Sigma_1^0$ y define a X .

2. Para la suficiencia, sea X definible por una fórmula $\varphi \in \Pi_1^0$ con parámetro Y . Como $\varphi \in \Pi_1^0$, entonces existen $\psi \in \Delta_0^0$ con parámetro Y y $n \in V_N$ tales que $\varphi \equiv \forall n\psi$. Notemos que:

$$\omega \setminus X = \{n : \neg\varphi(n)\} = \{n : \neg\forall m\psi(n, m)\} = \{n : \exists m\neg\psi(n, m)\}$$

Puesto que $\psi \in \Delta_0^0$, entonces $\neg\psi \in \Delta_0^0$ y luego $\exists m\neg\psi \in \Sigma_1^0$. Por tanto $\omega \setminus X$ es definible por una fórmula Σ_1^0 con parámetro Y , lo cual implica que $\omega \setminus X$ es c.e. en Y debido al punto anterior. Ya que $\omega \setminus X$ es c.e. en Y , se concluye que X es co-c.e. en Y .

Ahora, supongamos que X es co-c.e. en Y para la necesidad. Como X es co-c.e. en Y , entonces $\omega \setminus X$ es c.e. en Y . Debido al punto anterior sabemos que existe una fórmula $\varphi \in \Sigma_1^0$ con parámetro Y que define a $\omega \setminus X$. Como $\varphi \in \Sigma_1^0$, entonces existen $\psi \in \Delta_0^0$ con parámetro Y y $m \in V_N$ tales que $\varphi \equiv \exists m\psi$. Se tiene entonces que:

$$X = \omega \setminus (\omega \setminus X) = \{n : \neg\varphi\} = \{n : \neg\exists m\psi\} = \{n : \forall m\neg\psi\}$$

Ya que $\psi \in \Delta_0^0$, entonces $\neg\psi \in \Delta_0^0$, de lo cual se tiene que $\forall m\neg\psi \in \Pi_1^0$. Por lo tanto, se obtiene que X es definible por una fórmula Π_1^0 con parámetro Y .

3. Sabemos que X es computable en Y si y solo si X es c.e. y co-c.e. en Y , de lo cual por los dos puntos anteriores, se cumple si y solo si X es definible por una fórmula Σ_1^0 y por una fórmula Π_1^0 .

□

Definición 2.18 (Jerarquía Δ). Para cada $k \in \omega^+$, se define el conjunto $\Delta_k^0 := \Sigma_k^0 \cap \Pi_k^0$.

Lema 2.4. Sean $S \subseteq 2^\omega$ una ω -estructura, $k, m \in \omega^+$, $Y_1, \dots, Y_k \in S$ y $X \in \omega^m$. Si X es definible por una fórmula Δ_0^0 con parámetros $Y_1, \dots, Y_k$, entonces $X \leq_T Y_1 \oplus \dots \oplus Y_k$.

Demostración. Sea $\varphi(n_1, \dots, n_m) \in \Delta_0^0$ la fórmula con parámetros $Y_1, \dots, Y_m$ y $n_1, \dots, n_m \in V_N$ tal que $X = \{(n_1, \dots, n_m) : \varphi(n_1, \dots, n_m)\}$. Probemos por inducción sobre el conjunto Δ_0^0 .

■ (Base) Si $\varphi \in \text{Atom}$, entonces tenemos dos casos:

- Si φ está dada por $(t_1 = t_2)$ o $(t_1 < t_2)$ para algunos $t_1, t_2 \in \text{Term}$, ya que la suma, producto, la relación “menor que” y la relación identidad son computables en ω , se sigue que X es computable. Ya que $X \leq_T \emptyset$ y $\emptyset \leq_T Y_1 \oplus \dots \oplus Y_k$, entonces $X \leq_T Y_1 \oplus \dots \oplus Y_k$.

- Si φ está dada por $(t \in Y_p)$ para algún $t \in \text{Term}$ y $p \in \{1, \dots, k\}$, ya que la suma y producto son computables en ω , entonces se tiene claramente que $X \leq_T Y_p$. Ahora, puesto que $Y_p \leq_T Y_1 \oplus \dots \oplus Y_k$, entonces se sigue que $X \leq_T Y_1 \oplus \dots \oplus Y_k$.
- (Paso inductivo) Supongamos que $\psi(n_1, \dots, n_m), \theta(n_1, \dots, n_m) \in \Delta_0^0$ fórmulas con parámetros $Y_1, \dots, Y_k$ y $n_1, \dots, n_m \in V_N$ definen conjuntos $U, W \subseteq \omega^k$ respectivamente tales que $U, W \leq_T Y_1 \oplus \dots \oplus Y_k$. Entonces tenemos tres casos:

- Supongamos que φ está dada por $\neg\psi$. Ya que $U \leq_T Y_1 \oplus \dots \oplus Y_k$, entonces por propiedades de la computabilidad relativa, se tiene que $\omega \setminus U \leq_T Y_1 \oplus \dots \oplus Y_k$. Puesto que:

$$X = \{(n_1, \dots, n_m) : \varphi(n_1, \dots, n_m)\} = \{(n_1, \dots, n_m) : \neg\psi(n_1, \dots, n_m)\} = \omega \setminus U$$

se sigue que $X \leq_T Y_1 \oplus \dots \oplus Y_k$.

- Supongamos que φ está dada por $(\psi \wedge \theta)$. Ya que $U, W \leq_T Y_1 \oplus \dots \oplus Y_k$, entonces $U \cap W \leq_T Y_1 \oplus \dots \oplus Y_k$. Puesto que:

$$\begin{aligned} X &= \{(n_1, \dots, n_m) : \varphi(n_1, \dots, n_m)\} \\ &= \{(n_1, \dots, n_m) : (\psi(n_1, \dots, n_m) \wedge \theta(n_1, \dots, n_m))\} = U \cap W \end{aligned}$$

se sigue entonces que $X \leq_T Y_1 \oplus \dots \oplus Y_k$.

- Supongamos que φ está dada por $(\exists n < t) \psi$ o $(\forall n < t) \psi$ para algunos $n \in V_N$ y $t \in \text{Term}$ con $n \notin \text{Var}[t]$. Sabemos para cualquier caso que como la suma y producto son computables, las búsquedas acotadas también son computables en $Y_1 \oplus \dots \oplus Y_k$, por lo que $X \leq_T Y_1 \oplus \dots \oplus Y_k$.

Por lo tanto, podemos afirmar que $X \leq_T Y_1 \oplus \dots \oplus Y_k$. □

Teorema 2.5. Sean $S \in 2^\omega$ una ω -estructura, $k \in \omega^+$ y $Y_1, \dots, Y_k \in S$. Entonces toda fórmula Δ_0^0 con parámetro $Y_1 \oplus \dots \oplus Y_k$ es equivalente a una fórmula igualmente Δ_0^0 con parámetros $Y_1, \dots, Y_n$.

Demostración. Probemos por inducción sobre el número de parámetros.

- (Base) Trivialmente se cumple para el caso $k = 1$.
- (Paso inductivo) Sea $m \in \omega^+$ tal que toda fórmula Δ_0^0 con parámetro $Y_1 \oplus \dots \oplus Y_m$ es equivalente a una fórmula igualmente Δ_0^0 con parámetros $Y_1, \dots, Y_m$.

Sea $\varphi \in \Delta_0^0$ con parámetro $Y_1 \oplus \cdots \oplus Y_{m+1}$. Notemos que cada aparición de $Y_1 \oplus \cdots \oplus Y_{m+1}$ es de la forma $(t \in Y_1 \oplus \cdots \oplus Y_{m+1})$ para algún $t \in \text{Term}$. Por la definición de la yunta se tiene que esta fórmula es equivalente a:

$$\begin{aligned} & (\exists n < t + 1) (((n \in Y_1 \oplus \cdots \oplus Y_m) \wedge (t = (n + n))) \\ & \vee ((n \in Y_{m+1}) \wedge (t = ((n + n) + 1)))) \end{aligned}$$

la cual es Δ_0^0 . Puesto que la fórmula $((n \in Y_1 \oplus \cdots \oplus Y_m) \wedge (t = (n + n)))$ es Δ_0^0 , entonces por hipótesis existe una fórmula $\theta(Y_1, \dots, Y_m) \in \Delta_0^0$ equivalente. Se tiene entonces que:

$$\begin{aligned} (t \in Y_1 \oplus \cdots \oplus Y_{m+1}) & \equiv (\exists n < t + 1) (\theta(Y_1, \dots, Y_m) \\ & \vee ((n \in Y_{m+1}) \wedge (t = ((n + n) + 1)))) \end{aligned}$$

la cual sigue siendo Δ_0^0 . Por lo tanto, al reemplazar cada aparición de la subfórmula $(t \in Y_1 \oplus \cdots \oplus Y_{m+1})$ por tal fórmula equivalente en φ , se obtiene que φ es equivalente a una fórmula Δ_0^0 con parámetros $Y_1, \dots, Y_{m+1}$.

□

Teorema 2.6 (Post generalizado). Sean S una ω -estructura, $k \in \omega^+$ y $X, Y_1, \dots, Y_k \in S$.

1. X es c.e. en $Y_1 \oplus \cdots \oplus Y_k$ si y solo si X es definible por una fórmula Σ_1^0 con parámetros $Y_1, \dots, Y_k$.
2. X es co-c.e. en $Y_1 \oplus \cdots \oplus Y_k$ si y solo si X es definible por una fórmula Π_1^0 con parámetros $Y_1, \dots, Y_k$.
3. X es computable en $Y_1 \oplus \cdots \oplus Y_k$ si y solo si X es definible por una fórmula Δ_1^0 con parámetros $Y_1, \dots, Y_k$.

Demostración. Es únicamente necesario probar el primer punto, pues las demostraciones de los dos restantes son análogas a las de sus correspondientes del teorema 2.3.

Para la necesidad, por el teorema de Post (2.3), se tiene que si X es c.e. en $Y_1 \oplus \cdots \oplus Y_k$, entonces X es definible por una fórmula Σ_1^0 con parámetro $Y_1 \oplus \cdots \oplus Y_k$, es decir, existen una fórmula $\varphi(Y_1 \oplus \cdots \oplus Y_k) \in \Delta_0^0$ y $n \in V_N$ tales que X es definida por $\exists n \varphi(Y_1 \oplus \cdots \oplus Y_k)$. Por el teorema 2.5 sabemos que existe $\theta(Y_1, \dots, Y_k) \in \Delta_0^0$ tal que $\varphi \equiv \theta$, y luego $\exists n \varphi \equiv \exists n \theta$. Por lo tanto, X es definible por una fórmula Σ_1^0 con parámetros $Y_1, \dots, Y_k$.

Ahora, para la suficiencia, sea X definible por una fórmula Σ_1^0 con parámetros $Y_1, \dots, Y_k$, es decir, existen $m \in V_N$ y $\varphi(n, m) \in \Delta_0^0$ con parámetros $Y_1, \dots, Y_k$ tales que $X = \{n : \exists m \varphi(n, m)\}$. Por el lema 2.4 sabemos que el conjunto $Z \subseteq \omega^2$ dado por $Z =$

$\{(n, m) : \varphi(n, m)\}$ es $Y_1 \oplus \dots \oplus Y_k$ -computable. Sea $f(n) = \mu m Z(n, m)$ una función $Y_1 \oplus \dots \oplus Y_k$ -computable, entonces se sigue que para cada $p \in \omega$:

$$\begin{aligned} p \in X &\iff \exists m \varphi(p, m) \iff \exists m Z(p, m) \\ &\iff \mu m Z(p, m) \downarrow \iff f(p) \downarrow \iff p \in \text{dom}(f) \end{aligned}$$

De lo cual se implica que $X = \text{dom}(f)$. Puesto que X es dominio de una función $Y_1 \oplus \dots \oplus Y_k$ -computable, entonces se concluye que X es c.e. en $Y_1 \oplus \dots \oplus Y_k$. $\square$

Definición 2.19 (Axiomas base de la aritmética de segundo orden). Definimos a los *axiomas básicos* de L_2 , denotados por PA^- , como las cerraduras universales de las siguientes fórmulas:

1. $\neg(n + 1 = 0)$
2. $((n + 1) = (m + 1)) \rightarrow (n = m)$
3. $((n + (m + 1)) = ((n + m) + 1))$
4. $((n \cdot 0) = 0)$
5. $((n \cdot (m + 1)) = ((n \cdot m) + n))$
6. $\neg(n < 0)$
7. $((n < (m + 1)) \rightarrow ((n < m) \vee (n = m)))$
8. $((n + 0) = n)$

Definición 2.20 (Esquema de inducción). Sea $\Gamma \subseteq \text{Form}$ no vacío. Se define el *esquema de Γ -inducción*, denotado por $\Gamma\text{-IND}$, como el esquema de cerraduras universales:

$$((\varphi(0) \wedge \forall n (\varphi(n) \rightarrow \varphi(n + 1))) \rightarrow \forall n \varphi(n))$$

donde $\varphi \in \Gamma$.

Definición 2.21 (Esquema de Δ_1^0 -comprensión). Se define el *esquema de Δ_1^0 -comprensión*, denotado por $\Delta_1^0\text{-COM}$, como el esquema de cerraduras universales:

$$(\forall n (\varphi(n) \leftrightarrow \psi(n)) \rightarrow \exists X \forall n ((n \in X) \leftrightarrow \varphi(n)))$$

donde $\varphi(n) \in \Sigma_1^0$, $\psi(n) \in \Pi_1^0$ y $\text{Free}[\varphi] = \text{Free}[\psi] = \{n\}$.

Definición 2.22 (ω -modelo). Sean $\Gamma \subseteq \text{Sent}$ no vacío y $S \subseteq 2^\omega$ una ω -estructura. S se denomina ω -modelo de Γ si $S \models \varphi$ para cada $\varphi \in \Gamma$.

Capítulo 3

El subsistema RCA_0

Definición 3.1 (RCA_0). Se define al subsistema de L_2 de *axiomas de comprensión recursiva*, o RCA_0 , como el conjunto de axiomas:

$$\text{RCA}_0 := \text{PA}^- \cup \Sigma_1^0\text{-IND} \cup \Delta_1^0\text{-COM}$$

Teorema 3.1. Sea $S \subseteq 2^\omega$ una ω -estructura. S es un ω -modelo de RCA_0 si y solo si es un ideal de Turing.

Demostración. Para la necesidad debemos demostrar lo siguiente:

1. $S \neq \emptyset$

Consideremos la fórmula φ dada por $\neg(n = n)$. Notemos que $\varphi \in \Delta_0^0$ y luego φ es Σ_1^0 y Π_1^0 . Por lo tanto, por $\Delta_1^0\text{-COM}$ sabemos que existe el conjunto $X \in S$ definido por φ . Se tiene entonces lo siguiente:

$$X = \{n : \varphi(n)\} = \{n : \neg(n = n)\} = \emptyset$$

Por lo tanto, $\emptyset \in S$, y luego $S \neq \emptyset$.

2. Para cada $X, Y \in S$ se satisface que $X \oplus Y \in S$.

Consideremos a la fórmula φ dada por:

$$(\exists m < n + 1) (((m \in X) \wedge (n = (m + m))) \vee ((m \in Y) \wedge (n = ((m + m) + 1))))$$

Notemos que $\varphi \in \Delta_0^0$, y entonces por $\Delta_1^0\text{-COM}$ existe $Z \in S$ definido por φ . Se obtiene entonces lo siguiente:

$$\begin{aligned}
Z &= \{n : \varphi(n)\} \\
&= \{n : (\exists m < n+1) (((m \in X) \wedge (n = (m+m))) \\
&\quad \vee ((m \in Y) \wedge (n = ((m+m)+1))))\} \\
&= \{n : ((\exists m < n+1) ((m \in X) \wedge (n = (m+m))) \\
&\quad \vee (\exists m < n+1) ((m \in Y) \wedge (n = ((m+m)+1))))\} \\
&= \{n : (\exists m < n+1) ((m \in X) \wedge (n = (m+m)))\} \\
&\quad \cup \{n : (\exists m < n+1) ((m \in Y) \wedge (n = ((m+m)+1)))\} \\
&= \{2m : m \in X\} \cup \{2m+1 : m \in Y\} \\
&= X \oplus Y
\end{aligned}$$

Por lo tanto, $X \oplus Y \in S$.

3. para cada $X, Y \in 2^\omega$, si $X \leq_T Y$ y $Y \in S$, entonces $X \in S$.

Ya que $X \leq_T Y$, entonces por el teorema de Post (2.3) sabemos que existe una fórmula φ con parámetro Y la cual es Σ_1^0 y Π_1^0 y que define a X . Puesto que $Y \in S$, entonces por Δ_1^0 -COM se tiene que existe $Z \in S$ definida por φ . Por lo tanto, $X = Z \in S$.

Probemos ahora la suficiencia. Observemos que puesto que S es una ω -estructura, entonces se satisfacen directamente PA^- y Σ_1^0 -IND. Luego, es únicamente necesario demostrar que se satisface el esquema Δ_1^0 -COM.

Sea $\varphi(n) \in \Delta_1^0$ con $\text{Free}[\varphi] = \{n\}$, entonces demostremos que existe $X \in S$ definido por φ . Debido a que $S \neq \emptyset$, podemos suponer que $\varphi(n, Y_1, \dots, Y_k)$ para algunos parámetros $Y_1, \dots, Y_k \in S$ y $k \in \omega^+$. Consideremos al conjunto $X \in 2^\omega$ definido por φ , entonces por el teorema de Post generalizado (2.6) sabemos que $X \leq_T Y_1 \oplus \dots \oplus Y_k$. Ya que $Y_1, \dots, Y_k \in S$, entonces $Y_1 \oplus \dots \oplus Y_k \in S$, y por lo tanto, $X \in S$. $\square$

3.1. Teoría de números elemental

De ahora en adelante, todos los siguientes teoremas serán demostrados dentro del subsistema RCA_0 , a menos que se diga lo contrario.

Teorema 3.2 (Propiedades básicas de la aritmética). Se satisfacen las siguientes cerraduras universales.

1. $((n+m)+p) = (n+(m+p))$

2. $((0 + n) = n)$
3. $((1 + n) = n)$
4. $((n + m) = (m + n))$
5. $((n \cdot (m + p)) = ((n \cdot m) + (n \cdot p)))$
6. $((n \cdot m) \cdot p = (n \cdot (m \cdot p)))$
7. $((n + m) \cdot p = ((n \cdot p) + (m \cdot p)))$
8. $((0 \cdot n) = 0)$
9. $((1 \cdot n) = n)$
10. $((n \cdot m) = (m \cdot n))$
11. $((n < m) \wedge (m < p)) \rightarrow (n < p)$
12. $((n < m) \leftrightarrow ((n + 1) < (m + 1)))$
13. $(\neg(n = 0) \rightarrow (0 < n))$
14. $((n < m) \vee (n = m) \vee (m < n))$
15. $\neg(n < n)$
16. $((n < m) \leftrightarrow ((n + p) < (m + p)))$
17. $(n < ((n + m) + 1))$
18. $((n + p) = (m + p)) \rightarrow (n = m)$
19. $((\neg(p = 0) \wedge (n < m)) \rightarrow ((n \cdot p) < (m \cdot p)))$
20. $((\neg(p = 0) \wedge ((n \cdot p) < (m \cdot p))) \rightarrow (n < m))$
21. $((\neg(p = 0) \wedge ((n \cdot p) = (m \cdot p))) \rightarrow (n = m))$
22. $((n < m) \rightarrow (\exists p < m) ((n + p) + 1 = n))$
23. $(\neg(n = 0) \rightarrow (\exists m < n) ((m + 1) = n))$

[2] Demostraciones

Demostrar las propiedades del teorema anterior.

Notación 3.1. Realizaremos la siguiente notación:

1. Escribiremos $(t_1 \leq t_2)$ en lugar de la fórmula $(t_1 < t_2 + 1)$ donde $t_1, t_2 \in \text{Term}$.
2. Escribiremos $(t \notin X)$ en lugar de la fórmula $\neg(t \in X)$ donde $t \in \text{Term}$.
3. Escribiremos $(t_1 \neq t_2)$ en lugar de la fórmula $\neg(t_1 = t_2)$ donde $t_1, t_2 \in \text{Term}$.

Definición 3.2 (Mínimo). Sea X un conjunto. Se dice que X tiene un *elemento mínimo* si se satisface la siguiente fórmula:

$$\exists k \forall m ((k \in X) \wedge ((m \in X) \rightarrow (k \leq m)))$$

Teorema 3.3 (Principio del buen orden). Todo conjunto X no vacío, es decir tal que se satisface $\exists n (n \in X)$, tiene un elemento mínimo.

Demostración. Demostremos por contradicción. Consideremos a X un conjunto no vacío, es decir tal que se satisface $\exists n (n \in X)$, para el cual se cumple lo siguiente:

$$\forall k \exists m ((k \notin X) \vee ((m \in X) \wedge (m < k))) \quad (3.1)$$

Sea $\varphi(p)$ la fórmula dada por $(\forall k \leq p) (k \notin X)$, la cual es Δ_0^0 . Mostremos por Σ_1^0 -IND que se satisface la fórmula $\forall p \varphi(p)$.

- (Base) Si $p = 0$, entonces debemos mostrar que $(0 \notin X)$. Supongamos por el contrario que $(0 \in X)$, entonces por la fórmula (3.1) se sigue que existe m tal que $(m < 0)$ y $(m \in X)$. Puesto que por los axiomas PA^- se tiene que para cada n se satisface que $\neg(n < 0)$, entonces llegamos a una contradicción. Por lo tanto, necesariamente $0 \notin X$.
- (Paso inductivo) Sea n tal que se satisface $\varphi(n)$, entonces demostremos que se satisface $\varphi(n+1)$.

Supongamos que $((n+1) \in X)$, entonces por la fórmula (3.1) se sigue que existe m tal que $(m \in X)$ y $m < (n+1)$, es decir, $(m \leq n)$. Ya que se satisface $\varphi(n)$ por hipótesis, se tiene entonces que $(m \notin X)$, lo cual contradice el hecho que $(m \in X)$ como se dedujo anteriormente. Por lo tanto, necesariamente $((n+1) \notin X)$.

Se sigue entonces por Σ_1^0 -IND que:

$$\forall p \varphi(p) \equiv \forall p (\forall k \leq p) (k \notin X) \equiv \forall p (p \notin X)$$

lo cual contradice el hecho que X era no vacío.

En conclusión, por contradicción, se prueba el teorema. □

Definición 3.3 (Cota superior). Sea X un conjunto. X se dice estar *acotado superiormente* si se satisface la siguiente fórmula:

$$\exists n \forall m ((m \in X) \rightarrow (m \leq n))$$

Definición 3.4 (Máximo). Sea X un conjunto. Se dice que X tiene un *elemento máximo* si se satisface la siguiente fórmula:

$$\exists k \forall m ((k \in X) \wedge ((m \in X) \rightarrow (m \leq k)))$$

Teorema 3.4 (Principio del máximo). Todo conjunto X no vacío y acotado superiormente tiene un elemento máximo.

Demostración. Defínase la fórmula $\varphi(n)$ dada por:

$$((\forall k \leq n) (k \notin X) \vee (\exists p \leq n) (\forall k \leq n) ((k \in X) \rightarrow (k \leq p))) \quad (3.2)$$

la cual es Δ_0^0 . Observemos que esta fórmula se interpreta como que dado n se satisface que todos los elementos menores a n no pertenecen a X o el conjunto $X \cap \{0, \dots, n\}$ tiene máximo. Probemos entonces por Σ_1^0 -IND que se satisface la fórmula $\forall n \varphi(n)$.

1. (Base) Para $n = 0$ se tienen dos casos: $(0 \in X)$ o $(0 \notin X)$. Si se cumple $(0 \notin X)$, entonces directamente se satisface la primera parte de la fórmula (3.2). Si se satisface $(0 \in X)$, entonces tomando $p = 0$ se satisface la segunda parte de la fórmula (3.2). En cualquier caso obtenemos que se satisface $\varphi(0)$.
2. (Paso inductivo) Sea n tal que $\varphi(n)$. Entonces demostremos que se satisface que $\varphi(n+1)$.

Ya que se satisface $\varphi(n)$, entonces tenemos tres casos:

- Si se satisface $((n+1) \in X)$, entonces se satisface la segunda parte de la fórmula $\varphi(n+1)$ tomando $p = (n+1)$.
- Si se satisface $((n+1) \notin X)$ y para cada k con $(k \leq n)$ se cumple que $(k \notin X)$, entonces claramente se cumple la primer parte de la fórmula $\varphi(n+1)$.
- Si se satisface $((n+1) \notin X)$ y existe un elemento máximo de los elementos de X acotados por n , entonces directamente se satisface la segunda parte de la fórmula $\varphi(n+1)$.

Se sigue entonces que en cualquier caso se satisface $\varphi(n+1)$. Luego, por Σ_1^0 -IND se cumple la fórmula $\forall n \varphi(n)$.

Ya que X es acotado superiormente, se deduce que existe m tal que para cada k , si $k \in X$, entonces $k \leq m$. Por lo tanto, se satisface particularmente $\varphi(m)$.

Ahora, puesto que X es no vacío, entonces no se satisface la primera parte de $\varphi(m)$, de lo cual se sigue que existe p con $(p \leq m)$ tal que es máximo de los elementos de X acotados por m . Como m ya es cota superior de X , entonces tal p es un máximo de X .

□

3.2. Sistemas numéricos

3.2.1. Los números enteros

En esta sección desarrollaremos la construcción de los números enteros, racionales y reales que son necesarios para poder abordar el estudio de los teoremas del Análisis Real que analizaremos en cada uno de los subsistemas.

Fuera de RCA_0 , recordemos que la característica más importante de los números enteros es que existe la resta, y que para cada número entero m existen dos números naturales n, k tales que $m = n - k$. Con esta propiedad podemos codificar cada número entero con tal par de números naturales que se interpretan de la forma anterior. Por ejemplo, si tenemos el par $(1, 2)$, entonces lo interpretamos como el número entero $1 - 2 = -1$. Sin embargo, notemos que existen varios pares que nos darían el mismo número entero en ese sentido. Para el caso del -1 se tiene que los pares $(1, 2)$, $(2, 3)$, $(3, 4)$ y $(4, 5)$ se interpretarían de la misma forma. En ese caso, decimos que todos estos pares son *equivalentes* para representar al número -1 . En general, para cada par de pares (n, m) y (p, q) , estos son equivalentes si se interpretan como el mismo número entero, es decir, si $n - m = p - q$, o equivalentemente $n + q = p + m$. Con esta condición podemos luego definir una relación de equivalencia que nos codifique la *igualdad* entre números enteros en RCA_0 .

Definición 3.5 (Igualdad en $\mathbb{Z}$). Se define la relación $=_{\mathbb{Z}} \subseteq \mathbb{N}^2 \times \mathbb{N}^2$ como sigue para cada $n, m, p, q \in \mathbb{N}$:

$$\langle n, m \rangle =_{\mathbb{Z}} \langle p, q \rangle \iff n + q = p + m$$

Notemos que esta relación puede definirse por medio de una fórmula Δ_0^0 , por lo que existe debido a $\Delta_1^0\text{-COM}$.

Ya que vimos que existen elementos diferentes que son *iguales* mediante esta relación, entonces es necesario demostrar que esta relación es de equivalencia para que toda la construcción sea coherente.

Teorema 3.5. La relación $=_{\mathbb{Z}}$ es de equivalencia.

Demostración. Demostremos entonces que $=_{\mathbb{Z}}$ es reflexiva, simétrica y transitiva. Sean $n, m, p, q, r, s \in \mathbb{N}$.

1. Reflexividad

Puesto que $n + m = m + n$, entonces se sigue directamente $\langle n, m \rangle =_{\mathbb{Z}} \langle n, m \rangle$. Por lo tanto, $=_{\mathbb{Z}}$ es reflexiva.

2. Simetría

Supongamos que $\langle n, m \rangle =_{\mathbb{Z}} \langle p, q \rangle$, entonces por definición se satisface que $n + q = p + m$, o equivalentemente, $p + m = n + q$. De lo anterior se sigue que $\langle p, q \rangle =_{\mathbb{Z}} \langle n, m \rangle$. Por lo tanto, $=_{\mathbb{Z}}$ es simétrica.

3. Transitividad

Supongamos que $\langle n, m \rangle =_{\mathbb{Z}} \langle p, q \rangle$ y $\langle p, q \rangle =_{\mathbb{Z}} \langle r, s \rangle$, entonces se satisface que $n + q = p + m$ y $p + s = r + q$. Sumando ambas igualdades obtenemos lo siguiente:

$$\begin{aligned} (n + q) + (p + s) &= (p + m) + (r + q) \\ (n + s) + (p + q) &= (r + m) + (p + q) \\ n + s &= r + m \end{aligned}$$

Lo cual implica que $\langle n, m \rangle =_{\mathbb{Z}} \langle r, s \rangle$. Por lo tanto, $=_{\mathbb{Z}}$ es transitiva

En conclusión, la relación $=_{\mathbb{Z}}$ es de equivalencia □

Notación 3.2. Sean $A \subseteq \mathbb{N}$ y $R \subseteq A^2$ de equivalencia, entonces para cada $x \in A$ denotaremos por $[x]_R$ a la clase de equivalencia de x , es decir:

$$[x]_R = \{y \in A : xRy\}$$

Todo esto nos implica que dado cualquier $x \in \mathbb{N}^2$, su clase de equivalencia $[x]_{=_{\mathbb{Z}}}$ es el conjunto de todos los pares de naturales que se interpretan como el mismo número entero. Más aún, puesto que $=_{\mathbb{Z}}$ es un conjunto que existe en RCA_0 , entonces la clase de equivalencia de cualquier par de naturales existe ya que puede definirse fácilmente por una fórmula Δ_0^0 .

Ya que tenemos el concepto de igualdad en los enteros, podemos definir lo que es la suma y producto en ellos. Fuera de RCA_0 , consideremos dos números enteros $x = n - m$ y $y = p - q$, entonces su suma y producto satisface lo siguiente:

$$\begin{aligned} x + y &= (n - m) + (p - q) = (n + p) - (m + q) \\ xy &= (n - m)(p - q) = np - nq - mp + mq = (np + mq) - (nq + mp) \end{aligned}$$

De esta forma, podríamos definir la suma y producto de enteros como los pares $(n + p, m + q)$ y $(np + mq, nq + mp)$ respectivamente. Con esto definimos sus equivalentes en RCA_0 .

Definición 3.6. Definimos las siguientes funciones:

$$\begin{aligned} f : \mathbb{N}^2 \times \mathbb{N}^2 &\rightarrow \mathbb{N} & g : \mathbb{N}^2 \times \mathbb{N}^2 &\rightarrow \mathbb{N} \\ (\langle n, m \rangle, \langle p, q \rangle) &\mapsto \langle n + p, m + q \rangle & (\langle n, m \rangle, \langle p, q \rangle) &\mapsto \langle np + mq, nq + mp \rangle \end{aligned}$$

Por el momento no les denominamos suma ni producto debido a que necesitaremos redefinirlas cuando establezcamos propiamente el conjunto de números enteros.

En cambio, demostremos que estas dos operaciones se “comportan bien”, es decir, que son independientes del representante, son conmutativas, asociativas y distributivas.

Lema 3.6. Para cada $x, y, z, w \in \mathbb{N}^2$ se satisface que si $x =_{\mathbb{Z}} z$ y $y =_{\mathbb{Z}} w$, entonces $f(x, y) =_{\mathbb{Z}} f(z, w)$ y $g(x, y) =_{\mathbb{Z}} g(z, w)$.

Demostración. Sean $n, m, p, q, r, s, t, u \in \mathbb{N}$ tales que $x = \langle n, m \rangle$, $y = \langle p, q \rangle$, $z = \langle r, s \rangle$ y $w = \langle t, u \rangle$.

1. Mostremos que $f(x, y) =_{\mathbb{Z}} f(z, w)$.

Puesto que $x =_{\mathbb{Z}} z$ y $y =_{\mathbb{Z}} w$, entonces $n + s = r + m$ y $p + u = t + q$. Luego:

$$\begin{aligned} (n + s) + (p + u) &= (r + m) + (t + q) \\ (n + p) + (s + u) &= (r + t) + (m + q) \\ \langle n + p, m + q \rangle &=_{\mathbb{Z}} \langle r + t, s + u \rangle \\ f(x, y) &=_{\mathbb{Z}} f(z, w) \end{aligned}$$

2. Mostremos que $g(x, y) =_{\mathbb{Z}} g(z, w)$.

Puesto que $x =_{\mathbb{Z}} z$ y $y =_{\mathbb{Z}} w$, entonces $n + s = r + m$ y $p + u = t + q$. Luego:

$$\begin{aligned}
p + u &= t + q \\
n(p + u) &= n(t + q) \\
np + nu &= nt + nq \\
np + nu + mq &= nt + nq + mq \\
np + mq + nu &= nq + mq + nt \\
np + mq + nu + su &= nq + mq + nt + su \\
np + mq + (n + s)u &= nq + mq + nt + su \\
np + mq + (r + m)u &= nq + mq + nt + su \\
np + mq + ru + mu &= nq + mq + nt + su \\
np + mq + ru + mu + mt &= nq + mq + mt + nt + su \\
np + mq + ru + mu + mt &= nq + m(t + q) + nt + su \\
np + mq + ru + mu + mt &= nq + m(p + u) + nt + su \\
np + mq + ru + mu + mt &= nq + mp + mu + nt + su \\
np + mq + ru + mt &= nq + mp + nt + su \\
np + mq + ru + mt + st &= nq + mp + nt + st + su \\
np + mq + ru + mt + st &= nq + mp + (n + s)t + su \\
np + mq + ru + mt + st &= nq + mp + (r + m)t + su \\
np + mq + ru + mt + st &= nq + mp + rt + mt + su \\
np + mq + ru + st &= nq + mp + rt + su \\
(np + mq) + (ru + st) &= (nq + mp) + (rt + su) \\
\langle np + mq, nq + mp \rangle &=_{\mathbb{Z}} \langle rt + su, ru + st \rangle \\
g(x, y) &=_{\mathbb{Z}} g(z, w)
\end{aligned}$$

□

Lema 3.7. Para cada $x, y, z \in \mathbb{N}^2$ se satisface lo siguiente:

1. $f(x, y) = f(y, x)$
2. $g(x, y) = g(y, x)$
3. $f(x, f(y, z)) = f(f(x, y), z)$

$$4. g(x, g(y, z)) = g(g(x, y), z)$$

$$5. g(x, f(y, z)) = f(g(x, y), g(x, z))$$

Demostración. Sean $n, m, p, q, r, s \in \mathbb{N}$ tales que $x = \langle n, m \rangle$, $y = \langle p, q \rangle$ y $z = \langle r, s \rangle$.

$$1. f(x, y) = f(y, x)$$

Se tiene lo siguiente:

$$f(x, y) = \langle n + p, m + q \rangle = \langle p + n, q + m \rangle = f(y, x)$$

$$2. g(x, y) = g(y, x)$$

Se satisface lo siguiente:

$$g(x, y) = \langle np + mq, nq + mp \rangle = \langle pn + qm, pm + qn \rangle = g(y, x)$$

$$3. f(x, f(y, z)) = f(f(x, y), z)$$

Se satisface lo siguiente:

$$\begin{aligned} f(x, f(y, z)) &= f(x, \langle p + r, q + s \rangle) \\ &= \langle n + (p + r), m + (q + s) \rangle \\ &= \langle (n + p) + r, (m + q) + s \rangle \\ &= f(\langle n + p, m + q \rangle, z) \\ &= f(f(x, y), z) \end{aligned}$$

$$4. g(x, g(y, z)) = g(g(x, y), z)$$

Se satisface lo siguiente:

$$\begin{aligned} g(x, g(y, z)) &= g(x, \langle pr + qs, ps + qr \rangle) \\ &= \langle n(pr + qs) + m(ps + qr), n(ps + qr) + m(pr + qs) \rangle \\ &= \langle npr + nqs + mps + mqr, nps + nqr + mpr + mqs \rangle \\ &= \langle (np + mq)r + (nq + mp)s, (np + mq)s + (nq + mp)r \rangle \\ &= g(\langle np + mq, nq + mp \rangle, z) \\ &= g(g(x, y), z) \end{aligned}$$

$$5. g(x, f(y, z)) = f(g(x, y), g(x, z))$$

Se satisface lo siguiente:

$$\begin{aligned}
 g(x, f(y, z)) &= g(x, \langle p + r, q + s \rangle) \\
 &= \langle n(p + r) + m(q + s), n(q + s) + m(p + r) \rangle \\
 &= \langle np + nr + mq + ms, nq + ns + mp + mr \rangle \\
 &= \langle (np + mq) + (nr + ms), (nq + mp) + (ns + mr) \rangle \\
 &= f(\langle np + mq, nq + mp \rangle, \langle nr + ms, ns + mr \rangle) \\
 &= f(g(x, y), g(x, z))
 \end{aligned}$$

□

Lema 3.8. Para cada $n \in \mathbb{N}$ se satisface que $\langle 0, 0 \rangle =_{\mathbb{Z}} \langle n, n \rangle$.

Demostración. Se tiene directamente puesto que $n + 0 = 0 + n$. □

Con esto podemos ahora definir propiamente lo que son los números enteros en RCA_0 . Aunque lo más usual en teoría de conjuntos es usar cada clase de equivalencia como cada número entero y luego el conjunto de números enteros se defina como el conjunto cociente $\mathbb{N}^2 / =_{\mathbb{Z}}$, en RCA_0 no existen los conjuntos de conjuntos de números naturales, por lo que no podemos irnos directamente por esa definición. En cambio, de cada clase de equivalencia podríamos elegir un único elemento como representante y luego los números enteros serían tal conjunto de representantes, el cual sí puede existir ya que es un conjunto de naturales, lo cual sí está permitido. Puesto que ya sabemos que dado un conjunto no vacío podemos determinar su elemento mínimo, entonces de cada clase utilizaremos su elemento mínimo como representante.

Definición 3.7 (Números enteros). Definimos la función $Z : \mathbb{N}^2 \rightarrow \mathbb{N}$ tal que para cada $x \in \mathbb{N}^2$, $Z(x) = \min[x]_{=_{\mathbb{Z}}}$. Se define al conjunto de *números enteros* como el conjunto $\mathbb{Z} \subseteq \mathbb{N}$ tal que:

$$\mathbb{Z} := \text{ran}(Z)$$

es decir, un *número entero* es cualquier natural $x \in \mathbb{N}$ tal que existe $n \in \mathbb{N}^2$ para el cual $x = \min[n]_{=_{\mathbb{Z}}}$.

Notemos que $\mathbb{Z}$ existe por $\Delta_1^0\text{-COM}$. Además, Z está bien definida porque para cada $x \in \mathbb{N}^2$ se satisface que $x \in [x]_{=_{\mathbb{Z}}}$ y luego su mínimo existe. Otra observación es que claramente para cada $x, y \in \mathbb{N}^2$ tales que $x =_{\mathbb{Z}} y$ se satisface que $Z(x) = Z(y)$, pues x y y forman parte de la misma clase y luego tienen el mismo mínimo.

Ahora podemos también definir adecuadamente la suma y producto de $\mathbb{Z}$, con lo que podemos demostrar que $\mathbb{Z}$ es un anillo con unidad.

Definición 3.8 (Suma y producto en $\mathbb{Z}$). Se define la *suma* y *producto* en $\mathbb{Z}$ como las siguientes funciones:

$$\begin{aligned} +_{\mathbb{Z}} : \mathbb{Z}^2 &\rightarrow \mathbb{Z} & \cdot_{\mathbb{Z}} : \mathbb{Z}^2 &\rightarrow \mathbb{Z} \\ (x, y) &\mapsto Z(f(x, y)) & (x, y) &\mapsto Z(g(x, y)) \end{aligned}$$

donde f y g son las funciones de la definición 3.6.

Teorema 3.9. El conjunto de enteros $\mathbb{Z}$ es un anillo conmutativo con unidad con las operaciones $+_{\mathbb{Z}}$ y $\cdot_{\mathbb{Z}}$.

Demostración. Sean $x, y, z \in \mathbb{Z}$, entonces mostremos cada una de las siguientes propiedades con ayuda de los lemas 3.6, 3.7 y 3.8:

1. Conmutatividad para la suma

$$x +_{\mathbb{Z}} y = Z(f(x, y)) = Z(f(y, x)) = y +_{\mathbb{Z}} x$$

2. Asociatividad para la suma

$$\begin{aligned} x +_{\mathbb{Z}} (y +_{\mathbb{Z}} z) &= Z(f(x, Z(f(y, z)))) \\ &= Z(f(x, f(y, z))) \\ &= Z(f(f(x, y), z)) \\ &= Z(f(Z(f(x, y)), z)) \\ &= (x +_{\mathbb{Z}} y) +_{\mathbb{Z}} z \end{aligned}$$

3. Existencia del neutro aditivo

Definamos al entero $0_{\mathbb{Z}} := Z(\langle 0, 0 \rangle)$, entonces si $x = \langle n, m \rangle$ para algunos $n, m \in \mathbb{N}$, se tiene que:

$$\begin{aligned} x +_{\mathbb{Z}} 0_{\mathbb{Z}} &= Z(f(x, 0_{\mathbb{Z}})) \\ &= Z(f(\langle n, m \rangle, Z(\langle 0, 0 \rangle))) \\ &= Z(f(\langle n, m \rangle, \langle 0, 0 \rangle)) \\ &= Z(\langle n, m \rangle) \\ &= Z(x) = x \end{aligned}$$

4. Existencia de inversos aditivos

Si $x = \langle n, m \rangle$ para algunos $n, m \in \mathbb{N}$, entonces definamos $-x := Z(\langle m, n \rangle)$. Luego:

$$\begin{aligned}
 x +_{\mathbb{Z}} (-x) &= Z(f(x, -x)) \\
 &= Z(f(\langle n, m \rangle, Z(\langle m, n \rangle))) \\
 &= Z(f(\langle n, m \rangle, \langle m, n \rangle)) \\
 &= Z(\langle n + m, n + m \rangle) \\
 &= Z(\langle 0, 0 \rangle) = 0_{\mathbb{Z}}
 \end{aligned}$$

5. Conmutatividad para el producto

$$x \cdot_{\mathbb{Z}} y = Z(g(x, y)) = Z(g(y, x)) = y \cdot_{\mathbb{Z}} x$$

6. Asociatividad para el producto

$$\begin{aligned}
 x \cdot_{\mathbb{Z}} (y \cdot_{\mathbb{Z}} z) &= Z(g(x, Z(g(y, z)))) \\
 &= Z(g(x, g(y, z))) \\
 &= Z(g(g(x, y), z)) \\
 &= Z(g(Z(g(x, y)), z)) \\
 &= (x \cdot_{\mathbb{Z}} y) \cdot_{\mathbb{Z}} z
 \end{aligned}$$

7. Existencia del neutro multiplicativo

Definamos al entero $1_{\mathbb{Z}} := Z(\langle 1, 0 \rangle)$, entonces si $x = \langle n, m \rangle$ para algunos $n, m \in \mathbb{N}$, se tiene que:

$$\begin{aligned}
 x \cdot_{\mathbb{Z}} 1_{\mathbb{Z}} &= Z(g(x, 1_{\mathbb{Z}})) \\
 &= Z(g(\langle n, m \rangle, Z(\langle 1, 0 \rangle))) \\
 &= Z(g(\langle n, m \rangle, \langle 1, 0 \rangle)) \\
 &= Z(\langle n \cdot 1 + m \cdot 0, n \cdot 0 + m \cdot 1 \rangle) \\
 &= Z(\langle n, m \rangle) \\
 &= Z(x) = x
 \end{aligned}$$

8. Distributividad

$$\begin{aligned}
x \cdot_{\mathbb{Z}} (y +_{\mathbb{Z}} z) &= Z(g(x, Z(f(y, z)))) \\
&= Z(g(x, f(y, z))) \\
&= Z(f(g(x, y), g(x, z))) \\
&= Z(f(Z(g(x, y)), Z(g(x, z)))) \\
&= (x \cdot_{\mathbb{Z}} y) +_{\mathbb{Z}} (x \cdot_{\mathbb{Z}} z)
\end{aligned}$$

Por lo tanto, $\mathbb{Z}$ es un anillo conmutativo con unidad con las operaciones $+_{\mathbb{Z}}$ y $\cdot_{\mathbb{Z}}$. $\square$

Ahora, para definir a los números racionales es fundamental tener el concepto de orden en $\mathbb{Z}$. Fuera de RCA_0 , si tenemos dos números enteros $x = n - m$ y $y = p - q$ para algunos naturales n, m, p y q , sabemos que $x < y$ si y solo si $n - m < p - q$, o equivalentemente, $n + q < p + m$. De esta forma podemos también extender el concepto de orden a $\mathbb{Z}$ en RCA_0 como tal relación.

Definición 3.9 (Orden en $\mathbb{Z}$). Se define la *relación de orden* en $\mathbb{Z}$ como la relación $<_{\mathbb{Z}} \subseteq \mathbb{Z}^2$ tal que para cada $x = \langle n, m \rangle, y = \langle p, q \rangle \in \mathbb{Z}$:

$$x <_{\mathbb{Z}} y \iff n + q < p + m$$

Se tiene que $<_{\mathbb{Z}}$ existe por $\Delta_1^0\text{-COM}$, y además es un orden lineal.

Teorema 3.10. La relación de orden $<_{\mathbb{Z}}$ es un orden lineal en $\mathbb{Z}$.

Demostración. Sean $x = \langle n, m \rangle, y = \langle p, q \rangle, z = \langle r, s \rangle \in \mathbb{Z}$, entonces mostremos que $<_{\mathbb{Z}}$ es irreflexiva, transitiva y tricotómica.

1. Irreflexividad

Mostremos que $\neg(x <_{\mathbb{Z}} x)$. Puesto que $n + m = m + n$, entonces se satisface que $\neg(n + m < m + n)$, por lo que $\neg(x <_{\mathbb{Z}} x)$.

2. Transitividad

Supongamos que $x <_{\mathbb{Z}} y$ y $y <_{\mathbb{Z}} z$, entonces $n + q < p + m$ y $p + s < r + q$, por lo que se tiene lo siguiente:

$$\begin{aligned}
(n + q) + (p + s) &< (p + m) + (r + q) \\
(n + s) + (p + q) &< (r + m) + (p + q) \\
n + s &< r + m
\end{aligned}$$

Por lo tanto, $x <_{\mathbb{Z}} z$.

3. Tricotomía

Como $n, m, p, q \in \mathbb{N}$, y el orden en $\mathbb{N}$ es tricotómico, entonces se satisface una y solo una de las siguientes afirmaciones: $n + q = p + m$, $n + q < p + m$ o $n + q > p + m$.

Si $n + q = p + m$, entonces $x =_{\mathbb{Z}} y$, lo cual implicaría que $x = y$ ya que al ser enteros, serían representantes de la misma clase de equivalencia.

Si $n + q < p + m$, entonces $x <_{\mathbb{Z}} y$ por definición, y si $n + q > p + m$, también por definición $y <_{\mathbb{Z}} x$.

Por lo tanto, $<_{\mathbb{Z}}$ es tricotómica.

Por lo tanto, $<_{\mathbb{Z}}$ es un orden lineal en $\mathbb{Z}$. □

Para finalizar la construcción de $\mathbb{Z}$, mostremos que los números naturales se pueden encajar en $\mathbb{Z}$, lo cual podemos entender como que existe un subconjunto de $\mathbb{Z}$ que se comporta de manera idéntica como lo haría $\mathbb{N}$ en cuanto a la suma, producto y orden. Formalmente eso significa que debe existir una función inyectiva de $\mathbb{N}$ a $\mathbb{Z}$ que conserve la suma, producto y orden de $\mathbb{N}$ a $\mathbb{Z}$.

Puesto que nosotros ya interpretábamos cada par $\langle n, m \rangle$ de naturales como la resta $n - m$, entonces los números naturales serían los enteros tales que su segunda componente es 0. Por ejemplo, el número 2 en $\mathbb{Z}$ sería el representante de la clase de $\langle 2, 0 \rangle$, pues $2 = 2 - 0$.

Aseguramos que esta traducción de naturales a enteros es aquella función inyectiva que nos permite encajar los naturales en los enteros.

Teorema 3.11 (Encaje de $\mathbb{N}$ en $\mathbb{Z}$). Defínase la función $N : \mathbb{N} \rightarrow \mathbb{Z}$ tal que para cada $n \in \mathbb{N}$, $N(n) := Z(\langle n, 0 \rangle)$. Entonces N es inyectiva y satisface las siguientes propiedades:

1. Para cada $n, m \in \mathbb{N}$, $N(n + m) = N(n) +_{\mathbb{Z}} N(m)$.
2. Para cada $n, m \in \mathbb{N}$, $N(nm) = N(n) \cdot_{\mathbb{Z}} N(m)$.
3. Para cada $n, m \in \mathbb{N}$, $n < m$ si y solo si $N(n) <_{\mathbb{Z}} N(m)$.

Demostración. Sean $n, m \in \mathbb{N}$, entonces:

1. Inyectividad de N

Supongamos que $N(n) = N(m)$, entonces se tiene que $Z(\langle n, 0 \rangle) = Z(\langle m, 0 \rangle)$, lo cual significa que $\langle n, 0 \rangle =_{\mathbb{Z}} \langle m, 0 \rangle$, es decir que $n + 0 = 0 + m$, o equivalentemente, $n = m$. Por lo tanto, N es inyectiva.

2. Conservación de la suma

Se tiene lo siguiente:

$$\begin{aligned}
 N(n + m) &= Z(\langle n + m, 0 \rangle) \\
 &= Z(f(\langle n, 0 \rangle, \langle m, 0 \rangle)) \\
 &= Z(f(Z(\langle n, 0 \rangle), Z(\langle m, 0 \rangle))) \\
 &= Z(f(N(n), N(m))) \\
 &= N(n) +_{\mathbb{Z}} N(m)
 \end{aligned}$$

3. Conservación del producto

Se tiene lo siguiente:

$$\begin{aligned}
 N(nm) &= Z(\langle nm, 0 \rangle) \\
 &= Z(\langle nm + 0 \cdot 0, n \cdot 0 + 0 \cdot m \rangle) \\
 &= Z(g(\langle n, 0 \rangle, \langle m, 0 \rangle)) \\
 &= Z(g(Z(\langle n, 0 \rangle), Z(\langle m, 0 \rangle))) \\
 &= Z(g(N(n), N(m))) \\
 &= N(n) \cdot_{\mathbb{Z}} N(m)
 \end{aligned}$$

4. Conservación del orden

Sean $N(n) = \langle p, q \rangle$ y $N(m) = \langle r, s \rangle$ para algunos $p, q, r, s \in \mathbb{N}$, entonces sabemos que $n + q = p$ y $m + s = r$ por definición de N , entonces:

$$\begin{aligned}
 n < m &\iff n + q < m + q \\
 &\iff p < m + q \\
 &\iff p + s < m + q + s \\
 &\iff p + s < r + q \\
 &\iff \langle p, q \rangle <_{\mathbb{Z}} \langle r, s \rangle \\
 &\iff N(n) <_{\mathbb{Z}} N(m)
 \end{aligned}$$

□

Con esto hemos mostrado que al trabajar con los enteros, también podemos hacer referencia a los naturales si es que necesitamos tratar con ellos. Aunque cuando hablamos de los naturales en $\mathbb{Z}$ no necesariamente estamos realizando las mismas operaciones en el propio $\mathbb{N}$, es decir, no necesariamente $2_{\mathbb{Z}} +_{\mathbb{Z}} 3_{\mathbb{Z}} = 2 + 3$, la inyección N nos permite traducir cada expresión de un conjunto a otro de manera sencilla, indicándonos que podemos ignorar esa diferencia entre $\mathbb{N}$ y $\text{ran}(N)$, y en cambio tratarlos como el mismo conjunto. Por eso mismo, no haremos distinción simbólica entre $+_{\mathbb{Z}}$ y la suma en $\mathbb{N}$ al trabajar en $\mathbb{Z}$ debido a ese encaje.

3.2.2. Los números racionales

Ya que hemos construido los números enteros, ya directamente podemos definir lo que son los números racionales, lo cuales son los más fundamentales para el estudio del Análisis Real que realizaremos.

Fuera de RCA_0 , los números racionales se caracterizan por tener *división*, y en general cualquier número racional q puede expresarse como un cociente entre números enteros $q = n/m$, donde $m > 0$. De manera similar podemos codificar un número racional como un par de números enteros interpretados de la forma anterior. Por ejemplo, el número $2/3$ sería el par $(2, 3)$. Sin embargo, el par no es único, pues $2/3$ también puede representarse con los pares $(4, 6)$, $(6, 9)$, $(8, 12)$, etc. Entonces todos estos pares se deberían considerar *equivalentes* al interpretarse como el mismo racional. En general, para cada par de pares (p, q) y (r, s) que representan a los racionales p/q y r/s , estos son equivalentes si y solo si $p/q = r/s$, o equivalentemente $ps = rq$. De esta manera podemos definir la relación de equivalencia entre pares de enteros en RCA_0 que codifica la igualdad entre racionales.

Definición 3.10 (Igualdad en $\mathbb{Q}$). Se define la relación $=_{\mathbb{Q}} \subseteq (\mathbb{Z} \times \mathbb{Z}^+) \times (\mathbb{Z} \times \mathbb{Z}^+)$ como sigue para cada $n, m, p, q \in \mathbb{N}$:

$$\langle n, m \rangle =_{\mathbb{Q}} \langle p, q \rangle \iff nq = pm$$

Aquí $\mathbb{Z}^+$ se entiende como el conjunto de enteros positivos, es decir el conjunto $\mathbb{Z}^+ = \{n : (n \in \mathbb{Z}) \wedge (n >_{\mathbb{Z}} 0)\}$. Se tiene también que esta relación existe por Δ_1^0 -COM.

De manera análoga al teorema 3.5 se demuestra que $=_{\mathbb{Q}}$ es de equivalencia, por lo que para cada $q \in \mathbb{Z} \times \mathbb{Z}^+$ existe su clase de equivalencia $[q]_{=_{\mathbb{Q}}}$ por Δ_1^0 -COM. Entonces, de manera similar definiremos a los números racionales como el conjunto de elementos mínimos de cada clase de equivalencia.

Definición 3.11 (Números racionales). Defínase la función $Q : \mathbb{Z} \times \mathbb{Z}^+ \rightarrow \mathbb{N}$ tal que para cada $q \in \mathbb{Z} \times \mathbb{Z}^+$, $Q(q) = \min[q]_{=_{\mathbb{Q}}}$. Entonces definimos al *conjunto de números racionales* como el subconjunto $\mathbb{Q} \subseteq \mathbb{N}$ tal que:

$$\mathbb{Q} := \text{ran}(Q)$$

es decir, un *número racional* es cualquier natural $q \in \mathbb{N}$ tal que existe $n \in \mathbb{Z} \times \mathbb{Z}^+$ para el cual $q = \min[n]_{=_{\mathbb{Q}}}$.

De manera análoga podemos definir las operaciones en $\mathbb{Q}$ por medio de dos funciones auxiliares que codifican la suma y producto de racionales usando los pares de enteros. Fuera de

RCA_0 , si tenemos dos racionales $x = n/m$ y $y = p/q$, entonces obtenemos lo siguiente:

$$\begin{aligned} x + y &= \frac{n}{m} + \frac{p}{q} = \frac{nq + mp}{mq} \\ xy &= \left(\frac{n}{m}\right)\left(\frac{p}{q}\right) = \frac{np}{mq} \end{aligned}$$

Por lo que la suma y producto estarían codificados como los pares $(nq + mp, mq)$ y (np, mq) respectivamente, lo cual podemos llevar a RCA_0 de la siguiente forma.

Definición 3.12. Definimos las siguientes funciones:

$$\begin{aligned} f : (\mathbb{Z} \times \mathbb{Z}^+) \times (\mathbb{Z} \times \mathbb{Z}^+) &\rightarrow \mathbb{N} & g : (\mathbb{Z} \times \mathbb{Z}^+) \times (\mathbb{Z} \times \mathbb{Z}^+) &\rightarrow \mathbb{N} \\ (\langle n, m \rangle, \langle p, q \rangle) &\mapsto \langle nq + mp, mq \rangle & (\langle n, m \rangle, \langle p, q \rangle) &\mapsto \langle np, mq \rangle \end{aligned}$$

Al igual que en caso de los enteros, es necesario asegurar que estas operaciones son independientes del representante bajo la igualdad en $\mathbb{Q}$.

Lema 3.12. Para cada $x, y, z, w \in \mathbb{Z} \times \mathbb{Z}^+$ se satisface que si $x =_{\mathbb{Q}} z$ y $y =_{\mathbb{Q}} w$, entonces $f(x, y) =_{\mathbb{Q}} f(z, w)$ y $g(x, y) =_{\mathbb{Q}} g(z, w)$.

Demostración. Sean $n, p, r, t \in \mathbb{Z}$ y $m, q, s, u \in \mathbb{Z}^+$ tales que $x = \langle n, m \rangle$, $y = \langle p, q \rangle$, $z = \langle r, s \rangle$ y $w = \langle t, u \rangle$ donde además $x =_{\mathbb{Q}} z$ y $y =_{\mathbb{Q}} w$. Se tiene entonces que $ns = rm$ y $pu = tq$.

1. Para f se tiene lo siguiente:

$$\begin{aligned} (nq + mp)(su) &= nqsu + mpsu \\ &= rmqu + tqms \\ &= (ru + st)(mq) \end{aligned}$$

Por lo que $f(x, y) =_{\mathbb{Q}} f(z, w)$.

2. Para g se tiene lo siguiente:

$$(np)(su) = (ns)(pu) = (rm)(tq)$$

Por lo que $g(x, y) =_{\mathbb{Q}} g(z, w)$.

□

Una observación a realizar es que en los racionales se cumple la cancelación en la división, es decir, si nosotros tenemos el racional np/nq donde $n \neq 0$, entonces $np/nq = p/q$. Traducido a la construcción que estamos realizando se obtiene el siguiente teorema.

Lema 3.13. Para cada $p \in \mathbb{Z}$ y cada $n, q \in \mathbb{Z}^+$ se satisface que $\langle np, nq \rangle =_{\mathbb{Q}} \langle p, q \rangle$.

Demostración. Se sigue directamente puesto que $(np)q = p(nq)$. $\square$

Con esto podemos demostrar que estas funciones se comportan bien, lo cual significa que son conmutativas, asociativas y distributivas.

Lema 3.14. Para cada $x, y, z \in \mathbb{Z} \times \mathbb{Z}^+$ se satisface lo siguiente:

1. $f(x, y) = f(y, x)$
2. $g(x, y) = g(y, x)$
3. $f(x, f(y, z)) = f(f(x, y), z)$
4. $g(x, g(y, z)) = g(g(x, y), z)$
5. $g(x, f(y, z)) =_{\mathbb{Q}} f(g(x, y), g(x, z))$

Demostración. Las demostraciones de los primeros cuatro puntos son análogas a lo que se realiza en las demostraciones del teorema 3.7. La única necesaria a demostrar es la del último punto que es la distributividad.

Sean $n, p, r \in \mathbb{Z}$ y $m, q, s \in \mathbb{Z}^+$ tales que $x = \langle n, m \rangle$, $y = \langle p, q \rangle$, $z = \langle r, s \rangle$, entonces obtenemos lo siguiente usando el lema 3.13:

$$\begin{aligned}
 g(x, f(y, z)) &= g(x, \langle ps + qr, qs \rangle) \\
 &= \langle n(ps + qr), m(qs) \rangle \\
 &= \langle nps + nqr, mqs \rangle \\
 &=_{\mathbb{Q}} \langle m(nps + nqr), m^2qs \rangle \\
 &= \langle (np)(ms) + (mq)(nr), (mq)(ms) \rangle \\
 &= f(\langle np, mq \rangle, \langle nr, ms \rangle) \\
 &= f(g(x, y), g(x, z))
 \end{aligned}$$

$\square$

Lema 3.15. Para cada $n \in \mathbb{Z}^+$ se satisface que $\langle 0, 1 \rangle =_{\mathbb{Q}} \langle 0, n \rangle$ y $\langle 1, 1 \rangle =_{\mathbb{Q}} \langle n, n \rangle$.

Demostración. Se sigue directamente de las igualdades $0 \cdot n = 0$ y $n = n$. $\square$

Podemos entonces ahora definir las operaciones de suma y producto en $\mathbb{Q}$ de manera coherente.

Definición 3.13 (Suma y producto en $\mathbb{Q}$). Se define la *suma* y *producto* en $\mathbb{Q}$ como las siguientes funciones:

$$\begin{aligned} +_{\mathbb{Q}} : \mathbb{Q}\mathbb{Q}^2 &\rightarrow \mathbb{Q} & \cdot_{\mathbb{Q}} : \mathbb{Q}\mathbb{Q}^2 &\rightarrow \mathbb{Q} \\ (x, y) &\mapsto Q(f(x, y)) & (x, y) &\mapsto Q(g(x, y)) \end{aligned}$$

donde f y g son las funciones de la definición 3.12.

Se puede entonces probar una propiedad fundamental la cual es que $\mathbb{Q}$ es un campo.

Teorema 3.16. El conjunto de números racionales $\mathbb{Q}$ es un campo con las operaciones $+_{\mathbb{Q}}$ y $\cdot_{\mathbb{Q}}$.

Demostración. Solo es necesario demostrar la existencia de neutros e inversos (aditivos y multiplicativos) en $\mathbb{Q}$, pues las otras propiedades de campo son análogas a las de la demostración del teorema 3.9.

1. Neutro aditivo

Sea $x = \langle p, q \rangle \in \mathbb{Q}$ para algunos $p \in \mathbb{Z}$ y $q \in \mathbb{Z}^+$. Entonces si definimos $0_{\mathbb{Q}} := Q(\langle 0, 1 \rangle)$, se obtiene lo siguiente:

$$\begin{aligned} x +_{\mathbb{Q}} 0_{\mathbb{Q}} &= Q(f(x, 0_{\mathbb{Q}})) \\ &= Q(f(\langle p, q \rangle, Q(\langle 0, 1 \rangle))) \\ &= Q(f(p, q), \langle 0, 1 \rangle) \\ &= Q(p \cdot 1 + q \cdot 0, q \cdot 1) \\ &= Q(\langle p, q \rangle) \\ &= Q(x) = x \end{aligned}$$

2. Neutro multiplicativo

Sea $x = \langle p, q \rangle \in \mathbb{Q}$ para algunos $p \in \mathbb{Z}$ y $q \in \mathbb{Z}^+$. Entonces si definimos $1_{\mathbb{Q}} := Q(\langle 1, 1 \rangle)$, se obtiene lo siguiente:

$$\begin{aligned} x \cdot_{\mathbb{Q}} 1_{\mathbb{Q}} &= Q(g(x, 1_{\mathbb{Q}})) \\ &= Q(g(\langle p, q \rangle, Q(\langle 1, 1 \rangle))) \\ &= Q(g(p, q), \langle 1, 1 \rangle) \\ &= Q(\langle p, q \rangle) \\ &= Q(x) = x \end{aligned}$$

3. Inverso aditivo

Sea $x = \langle p, q \rangle \in \mathbb{Q}$ para algunos $p \in \mathbb{Z}$ y $q \in \mathbb{Z}^+$. Entonces si definimos $-x := Q(\langle -p, q \rangle)$, se obtiene lo siguiente:

$$\begin{aligned}
 x +_{\mathbb{Q}} (-x) &= Q(f(x, -x)) \\
 &= Q(f(\langle p, q \rangle, Q(\langle -p, q \rangle))) \\
 &= Q(f(p, q), \langle -p, q \rangle) \\
 &= Q(pq + q(-p), q^2) \\
 &= Q(\langle 0, q^2 \rangle) \\
 &= Q(\langle 0, 1 \rangle) = 0_{\mathbb{Q}}
 \end{aligned}$$

4. Inverso multiplicativo

Sea $x = \langle p, q \rangle \in \mathbb{Q}$ para algunos $p \in \mathbb{Z}$ y $q \in \mathbb{Z}^+$. Si $x \neq 0_{\mathbb{Q}}$, entonces $p \neq 0$, luego tenemos dos casos: $p > 0$ o $p < 0$.

Si $p > 0$, entonces definimos $x^{-1} := Q(\langle q, p \rangle)$. Luego:

$$\begin{aligned}
 x \cdot_{\mathbb{Q}} x^{-1} &= Q(g(x, x^{-1})) \\
 &= Q(g(\langle p, q \rangle, Q(\langle q, p \rangle))) \\
 &= Q(g(p, q), \langle q, p \rangle) \\
 &= Q(\langle pq, pq \rangle) \\
 &= Q(\langle 1, 1 \rangle) = 1_{\mathbb{Q}}
 \end{aligned}$$

En cambio, si $p < 0$, entonces definimos $x^{-1} := Q(\langle -q, -p \rangle)$. Luego:

$$\begin{aligned}
 x \cdot_{\mathbb{Q}} x^{-1} &= Q(g(x, x^{-1})) \\
 &= Q(g(\langle p, q \rangle, Q(\langle -q, -p \rangle))) \\
 &= Q(g(p, q), \langle -q, -p \rangle) \\
 &= Q(\langle -pq, -pq \rangle) \\
 &= Q(\langle 1, 1 \rangle) = 1_{\mathbb{Q}}
 \end{aligned}$$

En cualquier caso existe $x^{-1} \in \mathbb{Q}$ tal que $x \cdot_{\mathbb{Q}} x^{-1} = 1_{\mathbb{Q}}$

Por lo tanto, el conjunto $\mathbb{Q}$ es un campo con las operaciones $+_{\mathbb{Q}}$ y $\cdot_{\mathbb{Q}}$. □

Para terminar con la construcción de los racionales definamos el orden en ellos. Sean $x = n/m$ y $y = p/q$. Puesto que m y p se toman como enteros positivos, entonces se tiene que $x < y$ si y solo si $n/m < p/q$, o equivalentemente $nq < pm$. Entonces de esta manera podemos ahora caracterizar el orden de $\mathbb{Q}$ en RCA_0 .

Definición 3.14 (Orden en $\mathbb{Q}$). Se define la *relación de orden* en $\mathbb{Q}$ como la relación binaria $<_{\mathbb{Q}} \subseteq \mathbb{Q}^2$ tal que para cada $x = \langle n, m \rangle, y = \langle p, q \rangle \in \mathbb{Q}$ para algunos $n, m, p, q \in \mathbb{Z}$:

$$x <_{\mathbb{Q}} y \iff nq < pm$$

Teorema 3.17. La relación $<_{\mathbb{Q}}$ es un orden lineal.

Demostración. Sean $x = \langle n, m \rangle, y = \langle p, q \rangle, z = \langle r, s \rangle \in \mathbb{Q}$ para algunos $n, p, r \in \mathbb{Z}$ y $m, q, s \in \mathbb{Z}^+$. Mostremos entonces que $<_{\mathbb{Q}}$ es irreflexiva, transitiva y tricotómica.

1. Irreflexividad

Puesto que el orden en $\mathbb{Z}$ es irreflexiva, sabemos entonces que $\neg(nm < nm)$, es decir, $\neg(x <_{\mathbb{Q}} x)$.

2. Transitividad

Supongamos que $x <_{\mathbb{Q}} y$ y $y <_{\mathbb{Q}} z$, entonces $nq < pm$ y $ps < rq$. Puesto que $s > 0$, entonces $nqs < pms$. Como $m > 0$, entonces $psm < rqm$. Por transitividad del orden en $\mathbb{Z}$ tenemos que $nqs < rqm$, y como $q > 0$, entonces $ns < rm$. Por lo tanto, $x <_{\mathbb{Q}} z$.

3. Tricotomía

Por la tricotomía en $\mathbb{Z}$ sabemos que se satisface una y solo una de las siguientes afirmaciones: $nq = pm$, $nq < pm$ o $nq > pm$.

Si $nq = pm$, entonces $x =_{\mathbb{Q}} y$, y puesto que $x, y \in \mathbb{Q}$, entonces $x = y$.

Si $nq < pm$, entonces $x <_{\mathbb{Q}} y$; y si $nq > pm$, entonces $y <_{\mathbb{Q}} x$.

Por lo tanto, $<_{\mathbb{Q}}$ es un orden lineal. □

Finalmente podemos demostrar igualmente que podemos encajar $\mathbb{Z}$ en $\mathbb{Q}$ mediante el encaje $n \mapsto Q(\langle n, 1 \rangle)$, pues se interpreta que $n = n/1$.

Teorema 3.18 (Encaje de $\mathbb{Z}$ en $\mathbb{Q}$). Defínase la función $Z : \mathbb{Z} \rightarrow \mathbb{Q}$ tal que para cada $n \in \mathbb{Z}$, $Z(n) := Q(\langle n, 1 \rangle)$. Entonces Z es inyectiva y satisface las siguientes propiedades:

1. Para cada $n, m \in \mathbb{Z}$, $Z(n + m) = Z(n) +_{\mathbb{Q}} Z(m)$.
2. Para cada $n, m \in \mathbb{Z}$, $Z(nm) = Z(n) \cdot_{\mathbb{Q}} Z(m)$.
3. Para cada $n, m \in \mathbb{Z}$, $n < m$ si y solo si $Z(n) <_{\mathbb{Q}} Z(m)$.

Demostración. Sean $p, q \in \mathbb{Z}$, entonces:

1. Inyectividad de Z

Supongamos que $Z(p) = Z(q)$, entonces se tiene que $Q(\langle p, 1 \rangle) = Q(\langle q, 1 \rangle)$, lo cual significa que $\langle p, 1 \rangle =_{\mathbb{Q}} \langle q, 1 \rangle$, es decir que $p \cdot 1 = q \cdot 1$, o equivalentemente, $p = q$. Por lo tanto, Z es inyectiva.

2. Conservación de la suma

Se tiene lo siguiente:

$$\begin{aligned}
 Z(p + q) &= Q(\langle p + q, 1 \rangle) \\
 &= Q(f(\langle p, 1 \rangle, \langle q, 1 \rangle)) \\
 &= Q(f(Q(\langle p, 1 \rangle), Q(\langle q, 1 \rangle))) \\
 &= Q(f(Z(p), Z(q))) \\
 &= Z(p) +_{\mathbb{Q}} Z(q)
 \end{aligned}$$

3. Conservación del producto

Se tiene lo siguiente:

$$\begin{aligned}
 Z(pq) &= Q(\langle pq, 1 \rangle) \\
 &= Q(g(\langle p, 1 \rangle, \langle q, 1 \rangle)) \\
 &= Q(g(Q(\langle p, 1 \rangle), Q(\langle q, 1 \rangle))) \\
 &= Q(g(Z(p), Z(q))) \\
 &= Z(p) \cdot_{\mathbb{Q}} Z(q)
 \end{aligned}$$

4. Conservación del orden

Sean $Z(p) = \langle n, m \rangle$ y $Z(q) = \langle r, s \rangle$ para algunos $n, r \in \mathbb{Z}$ y $m, s \in \mathbb{Z}^+$, entonces sabemos que $pm = n$ y $qs = r$ por definición de Z , entonces:

$$\begin{aligned}
 p < q &\iff pm < qm \\
 &\iff n < mq \\
 &\iff ns < mqs \\
 &\iff ns < mr \\
 &\iff \langle n, m \rangle <_{\mathbb{Q}} \langle r, s \rangle \\
 &\iff Z(p) <_{\mathbb{Q}} Z(q)
 \end{aligned}$$

□

3.2.3. Los números reales

Con los racionales a nuestra disposición, el siguiente paso es definir qué significa ser un número real. Aquí aparece una primera dificultad: los reales no pueden formar un conjunto de RCA_0 , pues cualquier construcción de $\mathbb{R}$ produce un conjunto incontable y, por lo tanto, imposible de codificar como subconjunto de $\mathbb{N}$. Aun cuando existan modelos incontables de RCA_0 , su estructura interna es *contable*, así que no hay manera de representar a $\mathbb{R}$ como un conjunto dentro del sistema. Lo que haremos, entonces, será definir qué es *un* número real: cada real individual sí podrá codificarse, aunque la totalidad de ellos no.

La construcción que seguiremos es la de Cauchy. Para motivarla, recordemos (fuera de RCA_0) que $\mathbb{Q}$, con la métrica inducida por el valor absoluto, no es un espacio métrico completo: existen sucesiones de Cauchy de racionales que no convergen en $\mathbb{Q}$. Por ejemplo, la sucesión $\{x_n\}_{n \in \omega}$ definida por $x_0 = 1$ y $x_{n+1} = x_n/2 + 1/x_n$ es, a partir de su segundo término, decreciente y acotada inferiormente por $\sqrt{2}$, y por lo tanto convergente en $\mathbb{R}$. Como toda sucesión convergente en $\mathbb{R}$ es de Cauchy, se trata de una sucesión de Cauchy de números racionales; sin embargo, $\lim_{n \rightarrow \infty} x_n = \sqrt{2} \notin \mathbb{Q}$.

La idea es entonces construir $\mathbb{R}$ *completando* a $\mathbb{Q}$: consideramos todas las sucesiones de Cauchy de racionales e identificamos aquellas que deberían tener el mismo límite. Como ese límite no necesariamente existe en $\mathbb{Q}$, no podemos compararlo directamente; lo que sí podemos hacer es examinar la sucesión resta $\{x_n - y_n\}_{n \in \omega}$, la cual debería converger a 0. Así, diremos que dos sucesiones de Cauchy $\{x_n\}_{n \in \omega}, \{y_n\}_{n \in \omega} \subseteq \mathbb{Q}$ son *equivalentes* si para cada $\varepsilon > 0$ existe $N \in \omega$ tal que $|x_n - y_n| < \varepsilon$ para cada $n \geq N$.

Esta definición, sin embargo, tiene un costo: vista como fórmula es Π_2^0 , y trabajar en RCA_0 con una noción de igualdad tan compleja resulta poco conveniente, por lo que nos interesa simplificarla. Observemos qué dice realmente la definición: la resta se “acerca” al cero tanto como queramos (el papel de ε) a partir de cierto índice (el papel de N), de modo que N es, implícitamente, una función $N(\varepsilon)$. Una primera simplificación es no usar un ε arbitrario, sino una sucesión concreta de valores que decaiga a cero; una buena propuesta son los números 2^{1-n} con $n \in \omega$, que decaen rápidamente a 0 conforme n crece. Con esto, N pasa a ser una función de n . Aun así, la función $N(n)$ puede comportarse de manera errática: puede ser mayor que n para algunos valores y menor para otros, de modo que determinarla puede ser complicado y variar de una pareja de sucesiones a otra. La segunda simplificación es entonces eliminar por completo la búsqueda de N imponiendo $N(n) = n$.

Llegamos así a la siguiente definición: dos sucesiones $\{x_n\}_{n \in \omega}, \{y_n\}_{n \in \omega} \subseteq \mathbb{Q}$ son equivalentes si para cada $n \in \omega$ se satisface que $|x_n - y_n| < 2^{1-n}$. Ahora bien, esta condición obliga a que la sucesión resta decaiga a un ritmo exponencial, y no toda sucesión de Cauchy decae tan rápido. Para que la noción de equivalencia sea coherente, basta restringirnos desde el inicio a sucesiones de Cauchy que decaigan a ese mismo ritmo: pedimos que para cada $k, n, m \in \omega$, si $n, m \geq k$,

entonces $|x_n - x_m| < 2^{-k}$. Esta condición se simplifica tomando $n = k$ y escribiendo $m = k + p$ con $p \in \omega$: trabajaremos entonces con las sucesiones $\{x_n\}_{n \in \omega} \subseteq \mathbb{Q}$ tales que para cada $k, p \in \omega$, $|x_k - x_{k+p}| < 2^{-k}$. Con todo esto podemos definir qué significa ser un número real en RCA_0 .

Definición 3.15 (Sucesiones). Una *sucesión de números racionales* es una función $f : \mathbb{N} \rightarrow \mathbb{Q}$. Se denotará a la sucesión f por $\{x_k\}_{k \in \mathbb{N}}$ si para cada $k \in \mathbb{N}$, $f(k) = x_k$.

Definición 3.16 (Números reales). Un *número real* es cualquier sucesión de racionales $\{x_k\}_{k \in \mathbb{N}}$ tal que para cada $k, p \in \mathbb{N}$, $|x_k - x_{k+p}| < 2^{-k}$.

Definición 3.17 (Igualdad entre números reales). Sean $x = \{x_k\}_{k \in \mathbb{N}}$ y $y = \{y_k\}_{k \in \mathbb{N}}$ números reales. Decimos que x y y son *iguales*, denotado por $x =_{\mathbb{R}} y$, si para cada k se satisface que $|x_k - y_k| < 2^{-k+1}$.

Aunque esta relación $=_{\mathbb{R}}$ no es un conjunto que exista en RCA_0 , ya que es una relación entre conjuntos y no entre números, sí podemos verificar que satisface las propiedades de una relación de equivalencia.

Teorema 3.19. Se demuestra lo siguiente:

1. Para cada número real x se satisface que $x =_{\mathbb{R}} x$.
2. Para cada par de reales x y y se satisface que $x =_{\mathbb{R}} y$ implica $y =_{\mathbb{R}} x$.
3. Para cada terna de reales x , y y z se satisface que $x =_{\mathbb{R}} y$ y $y =_{\mathbb{R}} z$ implica $x =_{\mathbb{R}} z$.

Demostración. Consideremos a los reales $x = \{x_k\}_{k \in \mathbb{N}}$, $y = \{y_k\}_{k \in \mathbb{N}}$ y $z = \{z_k\}_{k \in \mathbb{N}}$.

1. Se satisface claramente para cada k que:

$$|x_k - x_k| = 0 < 2^{-k+1}$$

Por lo que $x =_{\mathbb{R}} x$.

2. Si $x =_{\mathbb{R}} y$, entonces sabemos que para cada k se satisface que $|x_k - y_k| < 2^{-k+1}$. Directamente se sigue que para cada k , $|y_k - x_k| < 2^{-k+1}$, es decir, $y =_{\mathbb{R}} x$.
3. Si $x =_{\mathbb{R}} y$ y $y =_{\mathbb{R}} z$, entonces por definición para cada k , $|x_k - y_k| < 2^{-k+1}$ y $|y_k - z_k| < 2^{-k+1}$. Consideremos un valor fijo $p > 1$, entonces por la desigualdad del triángulo y al

ser x , y y z números reales se sigue que para cada k :

$$\begin{aligned}
 |x_k - z_k| &\leq |x_k - x_{k+p}| + |x_{k+p} - y_{k+p}| + |y_k - y_{k+p}| + |y_k - z_k| \\
 &< 2^{-k} + 2^{-k-p+1} + 2^{-k} + 2^{-k+1} \\
 &= 2^{-k+1}(2^{-1} + 2^{-p} + 2^{-1} + 1) \\
 &= 2^{-k+1}2^{-p+1} \\
 &< 2^{-k+1}
 \end{aligned}$$

□

Ahora definamos el orden dentro de los reales. Fuera de RCA_0 , pensemos en dos números reales $\{x_n\}_{n \in \omega}$ y $\{y_k\}_{k \in \omega}$. Consideremos que $x = \lim_{n \rightarrow \infty} x_n$ y $y = \lim_{n \rightarrow \infty} y_n$ son tales que $x \leq y$. Por la propuesta de definición que tenemos de convergencia en RCA_0 sabemos que para cada $n, p \in \omega$:

$$\begin{aligned}
 |x_n - x_{n+p}| &< 2^{-n} \\
 |y_n - y_{n+p}| &< 2^{-n}
 \end{aligned}$$

Tomando el límite cuando $p \rightarrow \infty$ notemos que se tiene lo siguiente para cada $n \in \omega$:

$$\begin{aligned}
 |x_n - x| &< 2^{-n} \\
 |y_n - y| &< 2^{-n}
 \end{aligned}$$

de lo cual se sigue que $x_n - 2^{-n} < x$ y $y < y_n + 2^{-n}$. Puesto que $x \leq y$ se tiene que para cada $n \in \omega$, $x_n - 2^{-n} < x \leq y < y_n + 2^{-n}$, o equivalentemente se satisface que $x_n < y_n + 2^{-n+1}$. Además, notemos que tomando el límite cuando $n \rightarrow \infty$ obtenemos que:

$$\lim_{n \rightarrow \infty} x_n \leq \lim_{n \rightarrow \infty} (y_n + 2^{-n+1}) \implies x \leq y$$

Por lo tanto, podemos concluir que $x \leq y$ si y solo si para cada $n \in \omega$ se satisface que $x_n < y_n + 2^{-n+1}$. De esta forma podemos definir la relación de orden de los reales en RCA_0 .

Definición 3.18 (Orden en $\mathbb{R}$). Sean $x = \{x_k\}_{k \in \mathbb{N}}$ y $y = \{y_k\}_{k \in \mathbb{N}}$ números reales.

- Denotaremos por $x \leq_{\mathbb{R}} y$ si para cada k se satisface que $x_k < y_k + 2^{-k+1}$.
- Denotaremos por $x <_{\mathbb{R}} y$ si no se satisface que $y \leq_{\mathbb{R}} x$.

Mostremos que de esta definición de orden se puede demostrar que se satisfacen las propiedades de una relación de orden lineal.

Teorema 3.20. Se demuestra lo siguiente:

1. Si x es un real, entonces se satisface que $x \leq_{\mathbb{R}} x$.
2. Si x, y y z son números reales, $x \leq_{\mathbb{R}} y$ y $y \leq_{\mathbb{R}} z$, entonces $x \leq_{\mathbb{R}} z$.
3. Si x y y son números reales, $x \leq_{\mathbb{R}} y$ y $y \leq_{\mathbb{R}} x$, entonces $x =_{\mathbb{R}} y$.
4. Si x y y son números reales, entonces $x \leq_{\mathbb{R}} y$ o $y \leq_{\mathbb{R}} x$.

Demostración. Consideremos a los números reales $x = \{x_k\}_{k \in \mathbb{N}}$, $y = \{y_k\}_{k \in \mathbb{N}}$ y $z = \{z_k\}_{k \in \mathbb{N}}$.

1. Puesto que para cada k se satisface que $2^{-k+1} > 0$ se sigue que $x_k < x_k + 2^{-k+1}$, es decir, $x \leq_{\mathbb{R}} x$.
2. Si $x \leq_{\mathbb{R}} y$ y $y \leq_{\mathbb{R}} z$, entonces sabemos que para cada k , $x_k < y_k + 2^{-k+1}$ y $y_k < z_k + 2^{-k+1}$. Consideremos un valor fijo $p > 1$, entonces para cada k se sigue lo siguiente:

$$\begin{aligned}
 x_k &< y_k + 2^{-k+1} \\
 &< y_k - y_{k+p} + y_{k+p} + 2^{-k+1} \\
 &< 2^{-k} + y_{k+p} + 2^{-k+1} \\
 &< 2^{-k} + z_{k+p} + 2^{-k-p+1} + 2^{-k+1} \\
 &< z_{k+p} - z_k + z_k + 2^{-k} + 2^{-k+1} + 2^{-k-p+1} \\
 &< 2^{-k} + z_k + 2^{-k} + 2^{-k+1} + 2^{-k-p+1} \\
 &= z_k + 2^{-k+1} 2^{-p+1} \\
 &< z_k + 2^{-k+1}
 \end{aligned}$$

Por lo tanto, $x \leq_{\mathbb{R}} z$.

3. Si $x \leq_{\mathbb{R}} y$ y $y \leq_{\mathbb{R}} x$, entonces para cada k , $x_k < y_k + 2^{-k+1}$ y $y_k < x_k + 2^{-k+1}$. De la segunda desigualdad obtenemos que $y_k - 2^{-k+1} < x_k$, por lo que $y_k - 2^{-k+1} < x_k < y_k + 2^{-k+1}$, es decir, $|x_k - y_k| < 2^{-k+1}$ para cada k . Por lo tanto, $x =_{\mathbb{R}} y$.

□